



DIPARTIMENTO DI INGEGNERIA INFORMATICA, MODELLISTICA,  
ELETTRONICA E SISTEMISTICA

Corso di Laurea in Ingegneria Informatica

**Tesi di Laurea**

**Controllo del TurtleBot3 e  
aggiramento ostacoli noti con ROS  
Noetic**

Relatore:

**Prof. Domenico Famularo**

Candidato:

**Francesca Vaccaro**

Correlatore:

**Dott. Ric. Antonello Venturino**

**Mat. 239641**

ANNO ACCADEMICO 2024/2025

*A mia madre, la prima persona a conoscere i miei successi e i miei  
fallimenti, il posto sicuro in cui rifugiarsi anche nei giorni grigi.*

*A mio padre, che nonostante tutto è sempre stato lì, pronto a dare e fare  
tutto il possibile per me.*

*A mia sorella, che ha reso leggeri anche i momenti più pesanti.*

*“Ma tra la partenza e il traguardo in mezzo c’è tutto il resto e tutto il resto è  
giorno dopo giorno e giorno dopo giorno è silenziosamente costruire.”*

*(Nicolò Fabi - Costruire)*

# Indice

|                                                |           |
|------------------------------------------------|-----------|
| <b>Indice</b>                                  | <b>2</b>  |
| <b>1 Introduzione</b>                          | <b>6</b>  |
| 1.1 Obiettivo dell'elaborato . . . . .         | 6         |
| 1.2 Contesto e motivazioni . . . . .           | 7         |
| 1.2.1 Contesto . . . . .                       | 8         |
| 1.2.2 Storia della robotica mobile . . . . .   | 10        |
| Timeline . . . . .                             | 16        |
| 1.2.3 Motivazione . . . . .                    | 16        |
| <b>2 Controllo di un robot mobile</b>          | <b>18</b> |
| 2.1 Tipologie di robot . . . . .               | 18        |
| 2.1.1 Robot a trazione differenziale . . . . . | 18        |
| 2.1.2 Front Wheel Steering . . . . .           | 19        |
| 2.1.3 Omnidirezionali . . . . .                | 20        |
| 2.2 Cinematica differenziale . . . . .         | 22        |
| 2.2.1 Sistemi di riferimento . . . . .         | 24        |
| Matrice di Rotazione . . . . .                 | 26        |
| 2.2.2 Cinematica diretta . . . . .             | 27        |
| 2.2.3 Cinematica inversa . . . . .             | 29        |
| 2.3 Legge di controllo . . . . .               | 31        |
| 2.3.1 Approccio sensor-based . . . . .         | 31        |
| Tipi di sensori . . . . .                      | 32        |
| 2.3.2 Approccio con path planning . . . . .    | 33        |
| RoadMap . . . . .                              | 33        |

|                                              |           |
|----------------------------------------------|-----------|
| Cell Decomposition . . . . .                 | 34        |
| Potential Field . . . . .                    | 35        |
| 2.3.3 Modello di controllo . . . . .         | 36        |
| Schema di controllo . . . . .                | 40        |
| <b>3 Pianificazione e navigazione</b>        | <b>42</b> |
| 3.1 Cos'è ROS? . . . . .                     | 42        |
| 3.1.1 Architettura . . . . .                 | 42        |
| File system . . . . .                        | 43        |
| Computation Graph Level . . . . .            | 45        |
| Community Level . . . . .                    | 46        |
| 3.1.2 Comunicazione . . . . .                | 46        |
| Topic . . . . .                              | 47        |
| Service . . . . .                            | 48        |
| Action . . . . .                             | 49        |
| 3.1.3 Strumenti . . . . .                    | 50        |
| 3.2 Stack ROS Navigation . . . . .           | 50        |
| 3.2.1 Requisiti hardware . . . . .           | 51        |
| 3.2.2 Nodo move_base . . . . .               | 52        |
| 3.3 Algoritmo DWA . . . . .                  | 55        |
| 3.3.1 Introduzione dell'algoritmo . . . . .  | 55        |
| Vincoli imposti dagli ostacoli . . . . .     | 57        |
| Finestra Dinamica(Dynamic Window) . . . . .  | 59        |
| Massimizzare la funzione obiettivo . . . . . | 60        |
| 3.3.2 Implementazione in ROS . . . . .       | 62        |
| <b>4 Esperimenti e risultati</b>             | <b>63</b> |
| 4.1 Setup della simulazione . . . . .        | 63        |
| 4.1.1 Robot utilizzato . . . . .             | 63        |
| 4.1.2 Creazione dell'ambiente . . . . .      | 67        |
| Progettazione ambiente su gazebo . . . . .   | 67        |
| Generazione delle mappe con RViz . . . . .   | 69        |

|       |                                                                   |     |
|-------|-------------------------------------------------------------------|-----|
| 4.1.3 | Documentazione necessaria e file di lancio . . . . .              | 73  |
|       | Start localization . . . . .                                      | 74  |
|       | Start navigation . . . . .                                        | 75  |
| 4.1.4 | Architettura stack di navigazione . . . . .                       | 75  |
| 4.2   | Traiettorie asservite e comportamento nei pressi degli ostacoli . | 76  |
| 4.2.1 | Primo Scenario . . . . .                                          | 78  |
|       | Mappe Gazebo e RViz . . . . .                                     | 78  |
|       | Metriche . . . . .                                                | 82  |
|       | Grafici simulazioni . . . . .                                     | 83  |
| 4.2.2 | Secondo Scenario . . . . .                                        | 84  |
|       | Mappe Gazebo e RViz . . . . .                                     | 85  |
|       | Metriche . . . . .                                                | 86  |
|       | Grafici simulazioni . . . . .                                     | 87  |
|       | Confronti . . . . .                                               | 88  |
| 4.2.3 | Terzo Scenario . . . . .                                          | 90  |
|       | Mappe Gazebo e RViz . . . . .                                     | 90  |
|       | Metriche . . . . .                                                | 92  |
|       | Grafici simulazioni . . . . .                                     | 92  |
|       | Confronti . . . . .                                               | 93  |
| 4.2.4 | Quarto Scenario . . . . .                                         | 94  |
|       | Mappe Gazebo e RViz . . . . .                                     | 95  |
|       | Metriche . . . . .                                                | 96  |
|       | Grafici simulazioni . . . . .                                     | 96  |
| 4.2.5 | Quinto Scenario . . . . .                                         | 97  |
|       | Mappe Gazebo e RViz . . . . .                                     | 98  |
|       | Considerazioni . . . . .                                          | 98  |
| 4.2.6 | Confronto e considerazioni . . . . .                              | 99  |
| 4.3   | Esperimenti in laboratorio . . . . .                              | 100 |
| 4.3.1 | Connessione del robot . . . . .                                   | 100 |
| 4.3.2 | Scenari creati . . . . .                                          | 101 |
|       | Primo scenario . . . . .                                          | 101 |

|                               |            |
|-------------------------------|------------|
| Secondo scenario . . . . .    | 103        |
| Terzo scenario . . . . .      | 106        |
| Quarto scenario . . . . .     | 108        |
| <b>5 Conclusioni</b>          | <b>110</b> |
| 5.1 Proposte future . . . . . | 111        |
| <b>Bibliografia</b>           | <b>112</b> |

# Capitolo 1

## Introduzione

### 1.1 Obiettivo dell’elaborato

Il seguente elaborato, è stato sviluppato con l’obiettivo di ***analizzare e descrivere*** le strategie di controllo che vengono applicate ad un robot mobile con struttura *waffle* a trazione differenziale(*differential drive*), all’interno di un ambiente presentante degli ostacoli noti, che dovranno essere evitati.

Verranno fatti, in particolare, una serie di test all’interno del framework ROS (*Robot Operating System*), il quale offre una moltitudine di strumenti utili adatti sia alle simulazioni virtuali, dove verranno prevalentemente utilizzati strumenti come Gazebo e RViz, ma anche per simulazioni destinate alla vita reale e quindi in laboratorio; in particolar modo nel progetto verrà utilizzato un approccio standard, il suddetto prevede l’uso dello stack di navigazione ROS “*move\_base*” che combina: un planner *globale* per la computazione e dunque il calcolo di un percorso che guidi il robot dal punto di partenza in cui si trova, sino all’obiettivo prestabilito, e poi vi sarà anche un planner di tipo *locale*, nella fattispecie un controllore DWA(*Dynamic Window Approach*), il quale andrà a generare i comandi di controllo del robot, per fare in modo che questo possa evitare gli ostacoli presenti nell’ambiente, come in precedenza introdotto, l’ambiente in questione sarà già noto al robot.

Si potrà avere modo di osservare diverse simulazioni, per capire come si comportano in circostanze differenti e su diverse prove(partendo da situazioni più

semplici, arrivando a situazioni sempre più complesse), e attraverso la produzione di *metriche* e *grafici* sarà possibile arrivare a delle considerazioni in relazione a come agisce effettivamente il controllore e come si rapporta con lo stack di navigazione.

## 1.2 Contesto e motivazioni

In questo capitolo introduttivo seguirà una breve trattazione relativa alla storia della robotica mobile, al suo sviluppo e alle possibili implementazioni che ne derivano; in questo modo sarà maggiormente comprensibile lo scopo dell'elaborato, e verranno alla luce i passi in avanti fatti nell'ambito della seguente disciplina durante il corso degli anni.

Si pensi che la prima volta che è stata utilizzata la parola “**robot**” può essere considerata relativamente recente, difatti fu rinvenuta nel dramma utopico fantascientifico “***R.U.R: Rossum's Universal Robots***” del 1920, scritto da Karel Čapek 1.1, il quale introduce la parola robot come derivante da una parola ceca *robota*, in questo contesto però i robot sono come degli umanoidi, che però non possiedono le caratteristiche che rendono l'uomo tale, sono creati per sollevare l'uomo dalle fatiche del lavoro fisico, ma con il tempo finiscono per prendere anche i tratti caratteriali dei loro creatori. Questa è una ovvia descrizione, seppur distopica, della paura che in quegli anni vi era in relazione al progresso tecnico-scientifico.

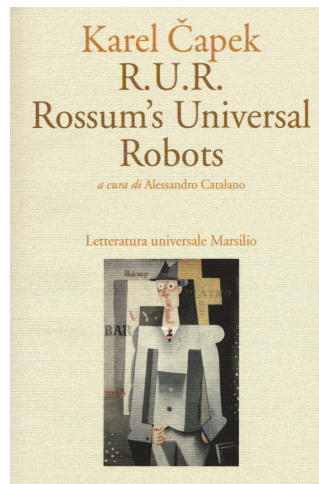


Figura 1.1: L'autore Karel Čapek, e una delle versioni dell'opera

### 1.2.1 Contesto

La robotica mobile, si occupa dello studio di robot e dispositivi che hanno la possibilità di muoversi nello spazio in cui si trovano, questo avviene grazie ad una serie di elementi, che dovranno tenere conto della storia degli spostamenti del robot nel tempo e nello spazio, ciò include sensori comunicanti con l'esterno e particolari strategie di controllo, ma anche grazie a elementi come ruote o cingoli che ne permettono l'effettivo movimento, cingoli e ruote sono solo esempi relativi al movimento sul suolo, difatti, dal momento che la robotica mobile nasce per semplificare azioni che all'uomo risulterebbero scomode, impossibili e a volte pericolose, troviamo anche dispositivi che possono muoversi in aria - ne sono un esempio i droni -, in acqua o addirittura in ambienti lontani dalla terra, come per esempio le superfici di altri pianeti.

Dunque il robot per potersi muovere in un ambiente, soprattutto se ostile e di conseguenza presentante degli ostacoli, dovrà mantenere informazioni relative alla posizione e all'orientamento, la navigazione e le strategie di controllo dovranno quindi poter andare di pari passo, così come l'acquisizione dei *dati* da parte dei sensori. Vi sono diverse modalità che vengono utilizzate per controllare i robot [1], una di queste è quella in cui il dispositivo viene *teleoperato* da personale esterno, e i livelli di supervisione possono essere molteplici, si passa per esempio da una supervisione quasi totale, dove il robot manda segnali a

chi lo telecomanda, e questo dovrà prendere tutte le decisioni, fino ad arrivare a modalità di supervisione quasi assente, in cui le decisioni sono “frutto” dei risultati, che vengono dati in output dai vari algoritmi installati all’interno del robot; la scelta del livello di controllo deriva anche dalle necessità e dalle circostanze, per esempio, guidare il ***Mars Rover*** dalla terra in tempo reale, sarebbe risultato troppo complesso, in quanto la distanza Terra-Marte avrebbe comportato una latenza nei segnali abbastanza cospicua, motivo per cui si è ricorsi ad altre soluzioni, come la pianificazione giornaliera delle attività che il robot deve svolgere basate sui dati raccolti durante la notte e “spediti” alla terra.

I robot trattati all’interno della robotica mobile però, non sono solamente veicoli dotati di ruote o cingoli, difatti vi sono anche dei dispositivi il quale scopo è quello di imitare quanto più possibile i movimenti biologici come la corsa, la camminata, il salto etc...

Chiaramente il fulcro di questo tipo di approccio, sono robot con fisionomie particolari, come quelli umanoidi, che come si evince dallo stesso nome riprendono le sembianze umane, bisogna dunque avere conoscenza di come i movimenti che si vogliono raggiungere avvengono e tentare di riprodurli al meglio, mantenendo “equilibrio”, stabilità e applicando la giusta cinematica. Nell’immagine riportata 1.2, è possibile osservare, in relazione al tipo di movimento, la principale forza di resistenza e l’applicazione della cinematica.

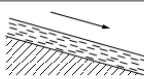
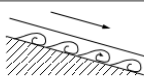
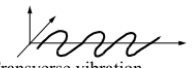
| Type of motion                                                                                      | Resistance to motion   | Basic kinematics of motion                                                                                                |
|-----------------------------------------------------------------------------------------------------|------------------------|---------------------------------------------------------------------------------------------------------------------------|
| Flow in a Channel  | Hydrodynamic forces    | Eddies                                  |
| Crawl              | Friction forces        | Longitudinal vibration                  |
| Sliding            | Friction forces        | Transverse vibration                    |
| Running            | Loss of kinetic energy | Periodic bouncing on a spring          |
| Walking            | Loss of kinetic energy | Rolling of a polygon (see figure 2.2)  |

Figura 1.2: Tabella dei movimenti per sistemi biologici. Fonte : “*Introduction to Autonomous Mobile Robots*”[2]

Come si evince dalla breve descrizione appena conclusa, lo studio di questa disciplina, è uno studio fortemente multidisciplinare, dal momento che alla costruzione, manutenzione e programmazione dello stesso dispositivo collaborano diverse scienze e discipline come la meccanica, l’elettronica e l’informatica, se poi il dispositivo sarà finalizzato per l’assistenza personale e dunque dovrà stare a contatto con le persone, allora entreranno in gioco anche discipline come le scienze sociali.

### 1.2.2 Storia della robotica mobile

Inizialmente lo sviluppo della robotica si è incentrato sulla produzione di *macchine autonome* che fossero fissate sul piano, ne sono un esempio i *bracci robotici* anche detti manipolatori, questi robot venivano e vengono utilizzati in contesti come quelli industriali, dove il loro compito è quello di assemblare, saldare e compiere anche operazioni maggiormente complesse; avere a disposizione questi dispositivi ha permesso dunque di automatizzare gran parte del lavoro. A livello cronologico uno dei primi manipolatori è stato il KUKA [2], un braccio robotico presentante sei giunti rotoidali 1.3; e con il passare del tempo se ne sono sviluppati altri, sempre più specializzati a compiere specifiche mansioni.



Figura 1.3: Foto storica del braccio robotico KUKA Famulus risalente al 1973.  
Fonte: KUKA AG, sezione “50 years Robotic Experience”, sito ufficiale KUKA

Si noti però come, avere un giunto fisso sul piano vada a ridurre le possibilità di movimento del robot, motivo per il quale si è sviluppata un’ulteriore branca della robotica che avesse come centro di trattazione dei dispositivi mobili, e dunque la *robotica mobile*, che a differenza di quella fissa necessita di una maggiore interazione con l’esterno e soprattutto di tenere conto della storia passata del movimento.

Uno dei primi progetti che ha avuto come protagonista un robot mobile, è stato “*Shakey*”(1966-1972) [3], questo progetto è stato mandato avanti dall’*Artificial Intelligence Center*; il robot in questione possedeva l’abilità di muoversi nell’ambiente andando, in maniera “limitata” a riconoscerlo e modellarlo, poteva inoltre svolgere attività di pianificazione, ricerca del percorso e riorganizzazione di alcuni oggetti, ovviamente il tutto con delle limitazioni date dalle tecnologie utilizzate. Sono state costruite due versioni di questo dispositivo, dove le principali differenze sono costituite dai sensori utilizzati

Il robot è composto da due motori “passo-passo” (*stepping motors*), e ognuno di essi gestisce una delle due ruote poste ai lati del veicolo, possiede poi

una telecamera e un telemetro ottico in una testa mobile, vi sono poi motori che controllano la messa a fuoco, le impostazioni del diaframma e l'angolo di inclinazione della testa.

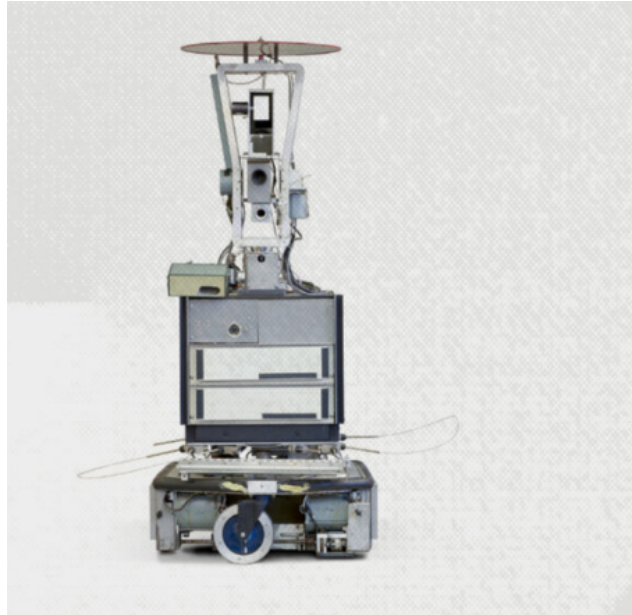


Figura 1.4: Shakey, fonte: Nilsson, Nils J. “Shakey the robot.” (1984).

Nello stesso frangente di tempo, anche l'industria aerospaziale dava vita ad uno dei primi robot mobili che siano mai andati sulla luna, in particolare si parla del ***Rover Lunokhod 1*** 1.5, un robot mobile teleoperato dalla terra, mandato sulla luna con lo scopo di studiare e analizzare la superficie lunare e raccogliere campioni di suolo. A bordo di questo veicolo vennero installate una serie di apparecchiature scientifiche e tecniche, messe all'interno di un contenitore sigillato.

Per oltre 300 giorni il lunokhod si è mosso con successo superando diversi ostacoli che il satellite terrestre presentava, come valli e colline, sopravvivendo a condizioni di vuoto spaziale, radiazioni e sbalzi di temperature da -100 a -150°C. Il rover è stato controllato a distanza dal *Communication Center* in Crimea; durante la sua missione, in cui l'ultima sessione di comunicazione risale al 14 settembre 1971, il veicolo ha percorso oltre 80000  $km^2$ , trasmettendo alla terra più di 20000 immagini, 206 panorami, 25 analisi del suolo a raggi X e 500 analisi fisiche e meccaniche del suolo [4].



Figura 1.5: Il rover lunare sovietico *Lunokhod 1*. Fonte: Petar Milošević (utente PetarM) via Wikimedia Commons, licenza CC BY-SA 3.0.

Sempre in ambito spaziale, ma ben più recente vi è il ***Sojourner*** 1.6, veicolo facente parte della missione *Mars Pathfinder* della **NASA**, la sua importanza deriva dal fatto che è stato il primo robot mobile ad operare su un altro pianeta in modalità semi autonoma, ha permesso di identificare ciottoli sia sulla superficie che nelle rocce di tipo sedimentario e non vulcanico, questa scoperta è stata molto importante, difatti implica che l'acqua su Marte sia esistita in qualche modo [5].

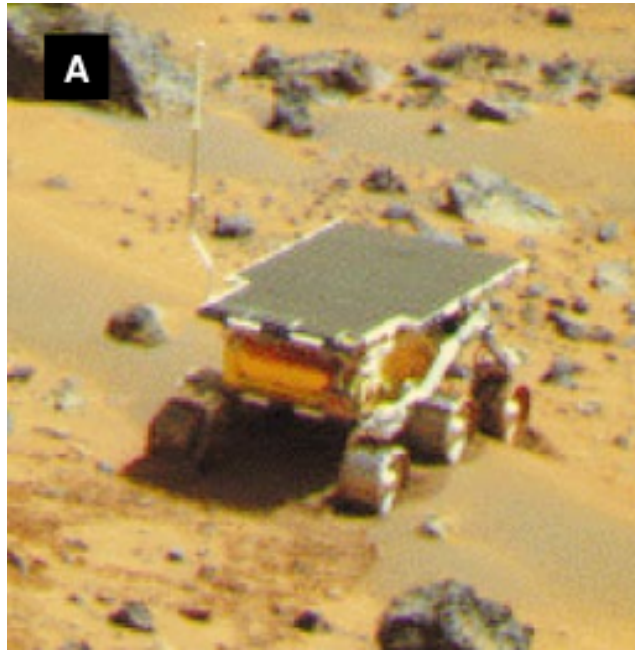


Figura 1.6: Il rover sojourner. Fonte: “Characterization of the Martian surface deposits by the Mars Pathfinder rover, Sojourner.”

Altri ambiti molto importanti sono stati quello domestico, come per esempio il **Roomba** del 2002 uno dei primi diffusi, ad oggi ne esistono diverse tipologie e con diverse funzioni, e quello medico come i robot **Robot UV e TUG**, che permettono di avere assistenza medica, ma vengono anche utilizzati per disinfezione e trasporto.

In particolare nelle immagini si può osservare: il Roomba 1.7, questo robot ha dato una svolta alla robotica di consumo, permetteva di dare assistenza domestica, come per esempio pulizia per tutta la casa, andando ad evitare gli ostacoli presenti; e poi una tipologia di robot TUG(A801) 1.8 molto recente che grazie alla sua tecnologia di navigazione ed evitamento degli ostacoli molto precisa permette di offrire una buona assistenza negli ospedali, può trasportare medicinali, controllare le porte e supporta una tecnologia di connessione 5G.



Figura 1.7: robort domestic roomba



Figura 1.8: robot tug A801

## Timeline

| Anno      | Robot/Eventi                 | Descrizione                                                        |
|-----------|------------------------------|--------------------------------------------------------------------|
| 1920      | <i>R.U.R.</i> di Karel Čapek | Primo utilizzo del termine "robot" in letteratura.                 |
| 1966–1972 | Shakey                       | Primo robot mobile intelligente capace di pianificazione autonoma. |
| 1970      | Lunokhod 1                   | Primo rover lunare teleoperato, missione sovietica.                |
| 1973      | KUKA Famulus                 | Uno dei primi manipolatori industriali con sei giunti rotoidali.   |
| 1997      | Sojourner (NASA)             | Primo rover mobile semi-autonomo su Marte.                         |
| 2002      | Roomba                       | Primo robot domestico mobile per la pulizia, ampiamente diffuso.   |
| 2004      | TUG (Aethon)                 | Robot mobile per il trasporto automatico in ospedali.              |
| 2020+     | Robot UV / Telepresenza      | Robot mobili per disinfezione e assistenza durante il COVID-19.    |

Tabella 1.1: Timeline storica della robotica mobile e dei principali robot

### 1.2.3 Motivazione

La diffusione, sempre maggiore, di robot mobili per le varie applicazioni sopra elencate, richiede lo sviluppo di tecnologie sempre migliori, a partire da quelle di navigazione e di controllo, che devono garantire la minimizzazione dell'errore, per mettere in sicurezza tutto il lavoro e rendere il tutto sempre più autonomo.

A questo proposito è dunque essenziale avere una buona integrazione tra pianificazione del percorso e controllo locale per evitare gli ostacoli.

La stesura di questo elaborato, nasce da un personale interesse verso la robotica mobile autonoma e dalla volontà di capire e studiare come i robot si muovono in uno spazio con degli ostacoli, affrontando il problema in maniera concreta

e dunque avvalendomi dei servizi che offre ROS, e di un vero robot su cui poter fare delle simulazioni reali, con l'obiettivo finale di confrontare approcci di navigazione, e ricavarne delle considerazioni.

# Capitolo 2

## Controllo di un robot mobile

Il seguente capitolo sarà incentrato sullo studio della *cinematica differenziale*, che governa e spiega come avviene il movimento del robot, e sulla *legge di controllo* che permette, invece, il raggiungimento dell'obiettivo prestabilito, avvalendosi di quelle che sono le informazioni relative alla comunicazione del robot con l'esterno, date da appositi sensori, ma non solo.

### 2.1 Tipologie di robot

Prima di passare all'effettiva trattazione delle leggi dietro al movimento del robot, è doveroso aprire una "parentesi" su quelle che sono le tipologie di dispositivi presi in considerazione; sarà proprio in base alla loro struttura fisica che si determineranno le leggi della cinematica.

#### 2.1.1 Robot a trazione differenziale

Il primo robot che viene trattato è di tipo *differential drive* 2.1, ovvero a trazione differenziale, in questo specifico caso le ruote poste su un lato del robot vengono controllate in maniera indipendente rispetto a quelle ubicate nel lato opposto.

Il moto effettivo del robot avverrà grazie al coordinamento tra le velocità angolari delle ruote.

In merito a ciò è possibile osservare che, quando le ruote poste sui due lati opposti presentano la stessa velocità angolare, allora il robot si muoverà su una linea retta; viceversa se hanno velocità angolari differenti si otterrà una rotazione (chiaramente il verso della rotazione dipende dalla velocità angolare che viene data alle ruote).

Nell'immagine le ruote sono quattro, ma esistono molti dispositivi di questa tipologia che ne presentano essenzialmente due, un esempio può essere il robot protagonista di questa tesi, ovvero un ***TurtleBot3*** che sia di tipo *burger* oppure come quello presente nelle simulazioni *waffle*. La configurazione appena descritta presenta diversi vantaggi, che dipendono anche dall'uso a cui è destinato il robot; un punto a favore è rappresentato dalla semplicità di implementazione del modello cinematico, difatti, nell'ottica del movimento, avere un robot di questo tipo permette, per esempio, di attuare manovre come la rotazione su se stesso. Tuttavia è limitato se parliamo di movimenti come quello laterale diretto, che implica uno spostamento laterale senza effettuare alcuna rotazione.

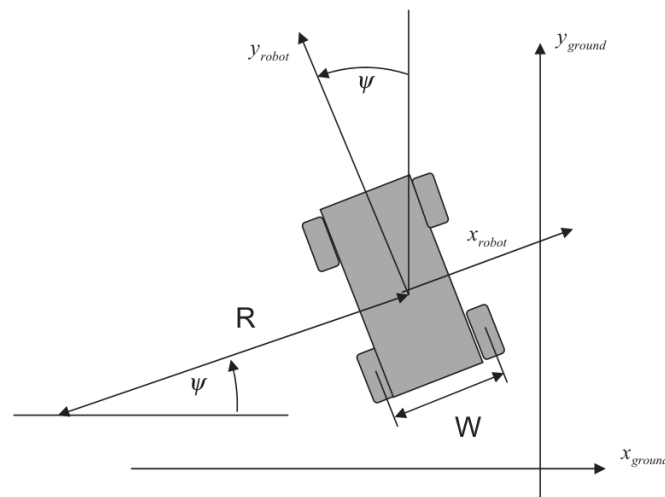


Figura 2.1: Rappresentazione schematica di un robot *Differential drive*, fonte : “Mobile robots: navigation, control and remote sensing” [1]

### 2.1.2 Front Wheel Steering

In questo caso si tratta di un modello di veicoli con ***sterzo anteriore*** 2.2, qui il veicolo viene generalmente azionato tramite delle ruote *posteriori* e la

sterzata avviene tramite un attuatore che fa girare le *ruote anteriori*.

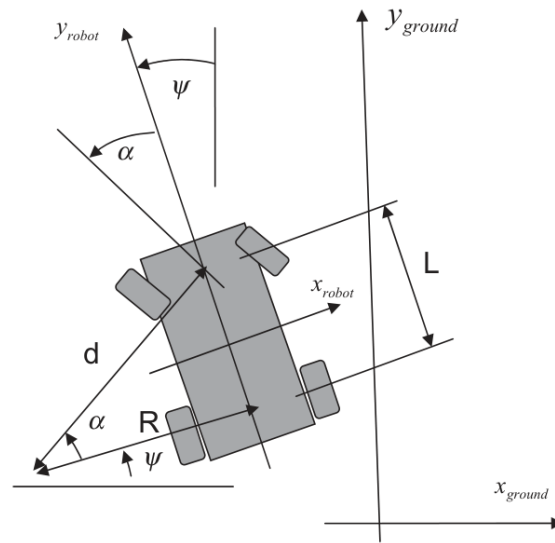


Figura 2.2: Rappresentazione schematica di un robot *Front-Wheel Steered Robot*, fonte : “Mobile robots: navigation, control and remote sensing” [1]

Un esempio di questi dispositivi sono le classiche automobili oppure quelle tipologie di tricicli che presentano le due ruote posteriori e la ruota anteriore sterzante.

Rispetto al modello presentato in precedenza, questo risulta essere maggiormente *manovrabile* in situazioni di elevata velocità, bisogna però considerare il fatto che richiede un meccanismo di sterzo più complesso e per quanto riguarda i movimenti, non ha la possibilità di girare sul posto.

### 2.1.3 Omnidirezionali

La particolarità dei robot *omnidirezionali* è che possono muoversi in qualsiasi direzione, indipendentemente dalla posa in cui si trovano in un determinato istante di tempo, questo avviene grazie a ruote come quelle Mecanum 2.3 o che siano sferiche/odometriche.

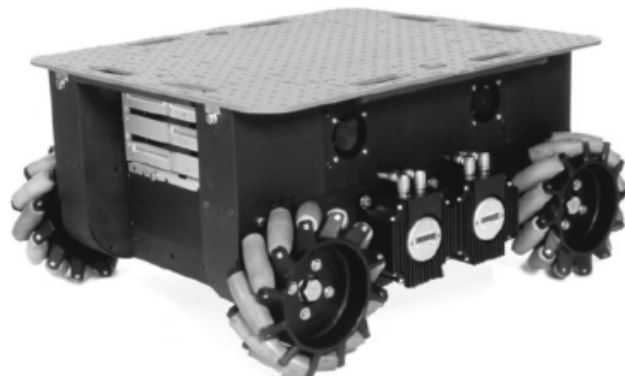


Figura 2.3: Rappresentazione schematica di un robot *omnidirezionale*, fonte: “Omnidirectional mobile robot-design and implementation.” (2007): 5518.[6]

Come è possibile osservare, le ruote sono molto ”particolari”, in questo caso specifico si tratta di ruote *Mecanum* 2.4 inventate nel 1973, la ruota è costituita da un *mozzo*<sup>1</sup> che porta un insieme di rulli liberi disposti a  $45^\circ$  rispetto alla circonferenza della ruota.

Questi rulli inclinati, fanno sì che una parte della forza esercitata nella direzione di rotazione della ruota, venga tradotta in una forza perpendicolare rispetto alla direzione della ruota stessa.

Verrà quindi prodotto, a seconda della direzione e della velocità di ciascuna ruota, un vettore di forza risultante in qualsiasi direzione desiderata. Ciò consente un libero movimento senza dover cambiare orientamento della ruota.

---

<sup>1</sup>Il mozzo è la parte centrale di una ruota o di un organo rotante che porta generalmente cuscinetti alloggiati al suo interno per permettere alla componente che verrà montata dentro di esso di ruotare.

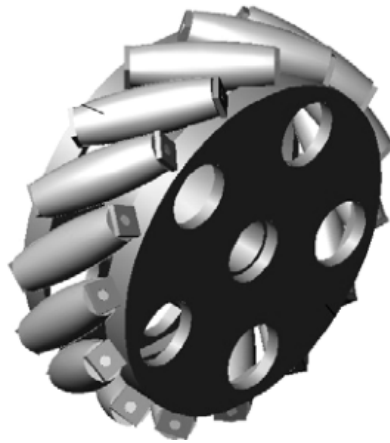


Figura 2.4: Ruota Mechanum *omnidirezionale*, fonte: “Omnidirectional mobile robot-design and implementation.” (2007): 5518.[6]

## 2.2 Cinematica differenziale

Questa sezione sarà dedicata all’analisi della cinematica del robot a trazione differenziale (*differential drive*) 2.1.1. Come sarà possibile notare, l’attenzione viene posta sul modello del dispositivo, poiché ogni sua caratteristica contribuisce alla produzione e generazione del moto: un differente schema di ruote, infatti, genererebbe comportamenti differenti per via dei vincoli che vengono imposti.

Nella sezione precedente si è discusso, seppur in modo introduttivo, e dunque non molto approfondito, dei vantaggi che ciascuna configurazione è in grado di offrire, e al contempo dei vincoli che limitano alcune azioni e alcuni movimenti. È di fondamentale importanza concentrarsi proprio su questi vincoli, che prendono il nome di *vincoli olonomi e anolonomi*<sup>2</sup>.

Quando si parla di un dispositivo che presenta le caratteristiche costruttive di un differential drive, un vincolo non olonomo (o *anolonomo*) è quello di non potersi muovere *lungo la direzione perpendicolare dell’asse delle ruote* come mostrato nell’immagine 2.5, matematicamente, se si considerano  $\dot{x}$  e  $\dot{y}$  le velo-

---

<sup>2</sup>Un vincolo è olonomo se può essere espresso da un’equazione che mette in relazione solo le coordinate generalizzate del sistema e, eventualmente, il tempo. Se un vincolo non può essere espresso in questa forma, è considerato anolonomo.

cità lineari del robot lungo gli assi e  $\theta$  l'orientamento di questo, il vincolo può essere espresso come:

$$\dot{x}\sin\theta - \dot{y}\cos\theta = 0 \quad (2.1)$$

In altri termini questa espressione ci dice che i due vettori sono **ortogonali** tra loro<sup>3</sup> dunque:

$$\begin{pmatrix} \dot{x} \\ \dot{y} \end{pmatrix} \cdot \begin{pmatrix} \sin(\theta) \\ -\cos(\theta) \end{pmatrix} = 0 \quad (2.2)$$

Questo particolare vettore, composto da  $\sin(\theta)$  e  $-\cos(\theta)$  non è altro che il vettore ortogonale al vettore che indica l'orientazione del robot, ergo l'asse di spostamento avanti-indietro.

Riferendosi a ciò che è stato in precedenza introdotto, relativamente alla differenza semantica tra le due tipologie di vincoli, questo è non olonomo in quanto non è possibile integrarlo per ottenere una relazione solo in termini di  $x, y, \theta$ , e dipende - come si può ben notare - anche dalle velocità e non solamente dalle coordinate.

Si noti bene che questo vincolo sarà fondamentale nella trattazione della cinematica, difatti risulterà utile in alcune operazioni che verranno fatte di seguito.

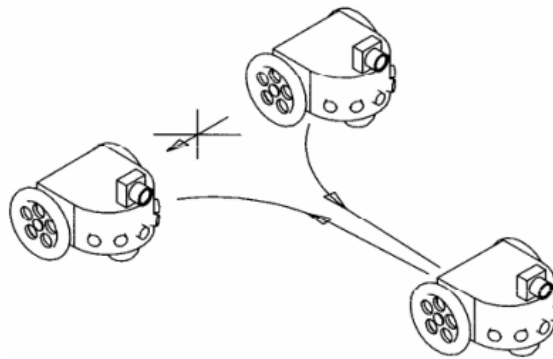


Figura 2.5: Movimento non permesso nel differential drive

---

<sup>3</sup>Due vettori sono ortogonali tra loro se il loro prodotto scalare risulta essere uguale a 0.

Compreso questo, un altro concetto immediatamente collegato è che derivare un modello per il movimento del robot è un processo *bottom-up*<sup>4</sup>, dunque ogni ruota contribuisce alla delineazione dell'equazione, motivo per cui per arrivare alla cinematica finale si passerà da quella delle singole ruote, prima però bisognerà andare a capire non solo il movimento delle ruote finalizzato ad uno spostamento del robot, ma anche e soprattutto come questo particolare spostamento influisca in un contesto più grande che è quello globale, ovvero l'ambiente in cui il robot viene inserito.

Non è banale infatti l'influenza dell'ambiente in queste circostanze, al contrario risulta addirittura fondamentale : un robot mobile infatti, a differenza di un robot fisso(escluse alcune casistiche che vedono i robot fissi comunicare con l'esterno in maniera assidua, come per prendere oggetti e portarli in altre posizioni, oppure casi in cui i robot devono comunicare tra loro per coordinarsi), deve sempre tenere conto dell'ambiente in cui si muove, specialmente se questo presenta degli ostacoli, in quanto si deve essere in grado di spostarsi nel modo più accurato possibile (l'accuratezza dipende anche e soprattutto da componenti presenti nel robot, che ne facilitano la comunicazione con l'esterno e sono i sensori), andando a *minimizzare l'errore* sempre più.

Il robot verrà trattato come un *corpo rigido* su ruote, che opera su un *piano orizzontale*. Quindi è importante definire il piano e le dimensioni che vengono prese in considerazione.

### 2.2.1 Sistemi di riferimento

La prima cosa rilevante da stabilire è il sistema di riferimento, sia quello globale, quindi relativo all'ambiente in cui si trova il robot, che quello locale del robot stesso, in modo da poter esprimere la relazione tra i due.

---

<sup>4</sup>Un processo bottom-up, parte dalle singole componenti o dettagli, per arrivare poi ad una visione più generale d'insieme, è il contrario dell'approccio top-down che invece parte dal generale per arrivare al particolare.

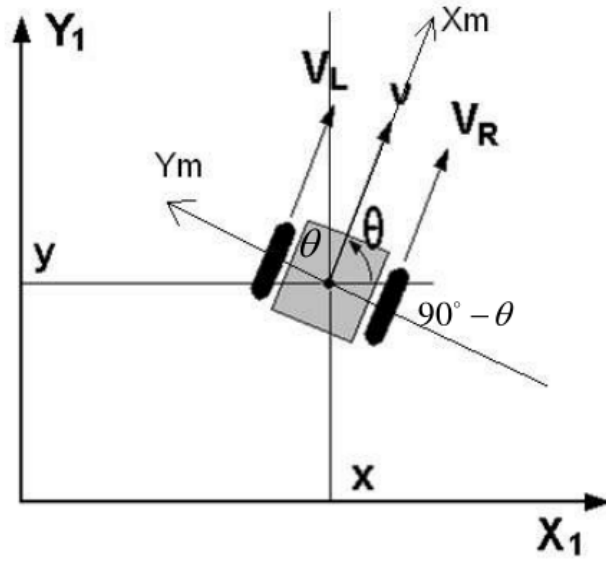


Figura 2.6: Movimento non permesso nel differential drive

Dalla figura 2.6 si distinguono i due sistemi di riferimento, quindi quello globale definito dagli assi  $X_I, Y_I$  e quello del robot, con assi  $X_R, Y_R$ , per specificare la posizione del robot nel piano si sceglie un punto P sul telaio del robot (questo punto può ad esempio trovarsi in un punto che risulti “comodo” come per esempio al centro dell’asse delle ruote), come punto di riferimento, la posizione di P sarà dunque indicata da  $x$  e  $y$  che corrispondono alle sue coordinate, e l’angolo che si forma tra i due sistemi viene indicato con  $\theta$ , in questo modo si può definire la **posa**<sup>5</sup> del robot come:

$$\zeta_I = \begin{pmatrix} x \\ y \\ \theta \end{pmatrix} \quad (2.3)$$

Una cosa molto importante da tenere a mente, è che la posa viene espressa in relazione a quello che è il sistema di riferimento globale, questo avviene in quanto quello a cui si è interessati è come si muove il robot nello spazio, quindi nell’ambiente e di conseguenza nel sistema globale.

Bisogna anche tenere conto che il robot si muoverà, il suo movimento quindi farà in modo che questa posa *cambi nel tempo*, dunque è bene indicarlo all’in-

<sup>5</sup>La posa è l’unione tra posizione e orientamento.

terno dell'espressione della posa(per questione di comodità potrebbe non essere indicato, ma è molto importante riconoscerne ed esplicitarne la dipendenza).

$$\zeta_I(t) = \begin{pmatrix} x(t) \\ y(t) \\ \theta(t) \end{pmatrix} \quad (2.4)$$

Il sistema di riferimento locale invece serve per descrivere i movimenti in termini di comandi alle ruote, dunque quando si parla di velocità angolari e lineari.

### Matrice di Rotazione

L'intento è quello di descrivere il movimento finale del robot, sulla base della cinematica delle sue componenti, ovvero le ruote, va dunque trovata una *trasformazione* che permetta di legare e mettere in relazione quei movimenti che si verificano nel contesto del sistema di riferimento globale, con il sistema di riferimento locale.

Proprio per questo motivo viene introdotto un elemento algebrico che prende il nome di **matrice di rotazione ortogonale**, questa risulta centrale anche nello studio della robotica fissa, e la si esprime nel seguente modo:

$$R(\theta) = \begin{pmatrix} \cos(\theta) & \sin(\theta) & 0 \\ -\sin(\theta) & \cos(\theta) & 0 \\ 0 & 0 & 1 \end{pmatrix} \quad (2.5)$$

La matrice riportata presenta una notevole importanza, in quanto serve come **operatore di cambio di base**: un vettore espresso in un sistema se moltiplicato per questa matrice verrà espresso in un altro sistema. È inoltre molto utilizzata anche in altri ambiti, in cui viene richiesta una rotazione che preveda la conservazione di distanze e angoli.

La sua forza deriva dalle proprietà che possiede, queste permetteranno lo svolgimento di determinate operazioni e in alcune casistiche le renderanno più semplici rispetto all'utilizzo di altri strumenti algebrici, le principali proprietà in questione sono:

- **ortogonalità** : le sue colonne e righe sono vettori ortonormali, si avrà che  $R^T R = 1$
- **determinante unitario**:  $\det(R) = 1$ , ciò implica che la matrice è sempre invertibile
- **conserva distanze e angoli**

A questo punto si può andare a ricavare la trasformazione per arrivare al vettore desiderato. Come detto, la posa esprime posizione e orientamento del robot, si ricerca però un movimento, quindi il vettore posa sarà derivato rispetto al tempo, questo sfrutta la legge fisica per cui derivando una posizione rispetto al tempo si ottiene la velocità istantanea, il vettore verrà indicato come:

$$\dot{\zeta}_I = \begin{pmatrix} \dot{x} \\ \dot{y} \\ \dot{\theta} \end{pmatrix} \quad (2.6)$$

Per quello che è stato detto in precedenza la trasformazione sarà scritta nel seguente modo:

$$\dot{\zeta}_R = R(\theta)\dot{\zeta}_I \quad (2.7)$$

Nelle sottosezioni immediatamente successive, sarà possibile osservare come come tutto ciò che è stato appena enunciato, possa essere applicato al robot posto al centro della trattazione.

## 2.2.2 Cinematica diretta

La **cinematica diretta** permette di passare dallo spazio dei giunti allo spazio fisico, dunque avendo in input la velocità delle ruote, si può ricavare come il robot si muove nello spazio, questo stesso approccio è centrale anche con i robot fissi e quindi inerenti alla robotica industriale, in questo specifico caso, quello che si può avere sono dei robot con varie tipologie di giunti e all'interno della

matrice di rotazione o rototraslazione basterà sostituire il valore dei giunti per capire la posa finale del robot.

Nella casistica che invece è centrale in questo progetto, non si vuole sapere solo il punto in cui il robot arriva, ma come evolve il movimento.

Dunque presupponendo di avere:

- P: centro di massa
- l: distanza delle ruote dal centro di massa
- $\dot{\varphi}_1$  e  $\dot{\varphi}_2$ : velocità angolari delle due ruote

Quello a cui si vuole arrivare con la cinematica diretta è il vettore  $\dot{\zeta}_I$  che è possibile esprimere in funzione delle caratteristiche presentate nell'elenco:

$$\dot{\zeta}_I = \begin{pmatrix} \dot{x} \\ \dot{y} \\ \dot{\theta} \end{pmatrix} = f(l, r, \theta, \dot{\varphi}_1, \dot{\varphi}_2) \quad (2.8)$$

Per raggiungere l'obiettivo, si utilizzano le informazioni relative alle ruote e la relazione di trasformazione specificata nella sottosezione precedente, facendo attenzione a invertire l'equazione.

Si parta dalle ruote, e dal presupposto di conoscere il raggio delle ruote ( $r$ ) e le velocità angolari ( $\dot{\varphi}_1$  e  $\dot{\varphi}_2$ ) e si calcolino le velocità lineari tangenziali nel seguente modo:

$$v_1 = r\dot{\varphi}_1, \quad v_2 = r\dot{\varphi}_2 \quad (2.9)$$

Da qui è possibile passare al calcolo della velocità lineare del robot nel frame locale R e della sua velocità angolare:

$$v = \frac{v_1 + v_2}{2} \Rightarrow \frac{r}{2} \cdot (\dot{\varphi}_1 + \dot{\varphi}_2) \quad (2.10)$$

$$\omega = \frac{v_2 - v_1}{2 \cdot l} \Rightarrow \frac{r}{2 \cdot l} \cdot (\dot{\varphi}_2 - \dot{\varphi}_1) \quad (2.11)$$

Arrivati a questo punto, tenendo conto del vincolo non olonomo 2.1, viene ricavato il vettore del movimento nel riferimento del robot:

$$\dot{\zeta}_R = \begin{pmatrix} v \\ 0 \\ \omega \end{pmatrix} = \begin{pmatrix} \frac{r}{2} \cdot (\dot{\varphi}_1 + \dot{\varphi}_2) \\ 0 \\ \frac{r}{2l} \cdot (\dot{\varphi}_2 - \dot{\varphi}_1) \end{pmatrix} \quad (2.12)$$

Per passare nel contesto globale bisogna applicare la trasformazione 2.7, ma invertita, si ricordi la proprietà per cui l'inversa della matrice di rotazione corrisponde alla sua trasposta, e sempre per le sue proprietà, la suddetta matrice è sempre invertibile, in quanto presenta un determinante unitario:

$$\dot{\zeta}_I = R(\theta)^T \dot{\zeta}_R \quad (2.13)$$

$$\begin{pmatrix} \dot{x} \\ \dot{y} \\ \dot{\theta} \end{pmatrix} = \begin{pmatrix} \cos(\theta) & -\sin(\theta) & 0 \\ \sin(\theta) & \cos(\theta) & 0 \\ 0 & 0 & 1 \end{pmatrix} \begin{pmatrix} v \\ 0 \\ \omega \end{pmatrix} \quad (2.14)$$

Risolvendo questo sistema si ottiene:

$$\begin{cases} \dot{x} = v \cdot \cos(\theta) = \frac{r}{2} \cdot (\dot{\varphi}_1 + \dot{\varphi}_2) \cdot \cos(\theta) \\ \dot{y} = v \cdot \sin(\theta) = \frac{r}{2} \cdot (\dot{\varphi}_1 + \dot{\varphi}_2) \cdot \sin(\theta) \\ \dot{\theta} = \frac{r}{2l} \cdot (\dot{\varphi}_2 - \dot{\varphi}_1) \end{cases} \quad (2.15)$$

### 2.2.3 Cinematica inversa

Differente è invece la **cinematica inversa**, questa permette di ricavare gli input da dare alle ruote, avendo la conoscenza della velocità che si vuole ottenere. Quindi si vanno a stabilire degli obiettivi da ottenere in termini di velocità lineare e angolare del robot, e tramite la cinematica inversa si ricavano quelle delle ruote del robot.

Come prima annunciato si passa dal dominio del riferimento globale a quello locale, da qui scriviamo la formula, per cui conosciamo  $\dot{\zeta}_I$  e vogliamo le singole velocità delle ruote (che siano esse quelle lineari o angolari, non ha importanza, è possibile ricavare le une dalle altre tramite il raggio delle ruote).

La trasformazione è:

$$\dot{\zeta}_R = R(\theta)^T \dot{\zeta}_I \quad (2.16)$$

Dove :

$$\dot{\zeta}_R = \begin{pmatrix} v \\ 0 \\ \omega \end{pmatrix} \quad (2.17)$$

Si ricordi che  $v$  e  $\omega$  dipendono dalle velocità dalle ruote 2.10 2.11.

L'inversa consiste nel risolvere rispetto a  $\dot{\varphi}_1$  e  $\dot{\varphi}_2$

Dunque:

$$\begin{cases} v = \frac{r}{2}(\dot{\varphi}_1 + \dot{\varphi}_2) \\ \omega = \frac{r}{2l}(\dot{\varphi}_2 - \dot{\varphi}_1) \end{cases} \Rightarrow \begin{cases} \frac{2v}{r} = \dot{\varphi}_1 + \dot{\varphi}_2 \\ \frac{2l\omega}{r} = \dot{\varphi}_2 - \dot{\varphi}_1 \end{cases} \quad (2.18)$$

Quindi scriviamo come equazione:

$$2\frac{v}{r} + 2l\frac{\omega}{r} = (\dot{\varphi}_1 + \dot{\varphi}_2) + (\dot{\varphi}_2 - \dot{\varphi}_1) = 2\dot{\varphi}_2 \quad (2.19)$$

Ricavo  $\dot{\varphi}_2$ :

$$2\dot{\varphi}_2 = 2\frac{v}{r} + 2\frac{l\omega}{r} \Rightarrow \dot{\varphi}_2 = \frac{(v + l\omega)}{r} \quad (2.20)$$

Ricavo anche  $\dot{\varphi}_1$

$$\dot{\varphi}_1 = \frac{v - l\omega}{r} \quad (2.21)$$

Grazie alla cinematica inversa si è arrivati alla conclusione per cui:

$$\dot{\varphi}_1 = \frac{(v - l\omega)}{r}, \quad \dot{\varphi}_2 = \frac{(v + l\omega)}{r} \quad (2.22)$$

In termini di velocità lineare invece:

$$v_1 = v - l\omega, \quad v_2 = v + l\omega \quad (2.23)$$

## 2.3 Legge di controllo

Dopo aver ricavato il modello cinematico e aver stabilito quindi come avviene il moto del robot, bisogna capire come questo si comporterà in relazione all'ambiente circostante, riuscendo ad arrivare al proprio obiettivo, senza però collidere con gli ostacoli che si troveranno lungo il percorso.

Come più volte ribadito è importante avere uno scambio accurato delle informazioni, tra il robot e l'ambiente, per fare ciò vengono utilizzati dei sensori, questi serviranno a "stabilire" quanto il percorso che il robot sta seguendo sia corretto e dunque presenti un errore molto basso, quindi bisogna andare a confrontare ciò che si sta facendo da quello che era stato prestabilito.

Il voler confrontare lo stato attuale con quello di riferimento generato e agire in base all'errore, può riportare alla mente l'utilizzo di un sistema ad anello chiuso. Un approccio puramente di feedback e quindi retroazionato, però, seppur risulti efficace per correggere gli errori non basta: nel controllo del robot si avrà anche la presenza di un approccio ad anello aperto, chiamato anche feedforward, il quale viene utilizzato per prevedere e compensare i movimenti. L'unione dei due approcci, come sarà possibile visionare nel corso della sezione, farà in modo di avere un controllo maggiormente robusto e preciso.

Prima di potersi addentrare nella derivazione della ***legge di controllo***, è utile individuare delle diverse tipologie di controllo che si possono avere [8]:

- approccio sensor-based control-based
- approccio che sfrutta la pianificazione del percorso

### 2.3.1 Approccio sensor-based

Questa modalità viene pensata per gli scenari in cui, l'ambiente non è conosciuto dal principio e presenta degli ostacoli in movimento.

Si noti bene che viene molto complesso calcolare un percorso da seguire già dalla partenza, se non si conosce come si muovono gli oggetti all'interno dell'ambiente, motivo per cui ci si basa sulle velocità e in particolare sul concetto

di *velocity obstacle (VO)* [9], per cui si cerca di capire e stabilire dove sono gli ostacoli in un determinato istante e dove saranno in futuro.

Quindi, bisogna avere la conoscenza sulle velocità del robot e degli ostacoli, con quelle degli ostacoli si va a definire uno spazio delle *"velocità proibite"*, ovvero le velocità che se mantenute porterebbero a delle eventuali collisioni, ad ogni intervallo di tempo i sensori aggiornano tutte queste informazioni e le tecniche di evitamento di ostacoli consistono nell'andare a scegliere velocità al di fuori dello spazio proibito.

## Tipi di sensori

Proprio in relazione a ciò che è stato detto fino ad ora di questo approccio, ma più in generale a quanto i sensori siano notevolmente importanti quando si parla di robotica, ecco una tabella relativa a quelli maggiormente utilizzati:

| General classification<br>(typical use)                                                         | Sensor<br>Sensor System      | PC or<br>EC | A or P |
|-------------------------------------------------------------------------------------------------|------------------------------|-------------|--------|
| Tactile sensors<br>(detection of physical contact or<br>closeness; security switches)           | Contact switches, bumpers    | EC          | P      |
|                                                                                                 | Optical barriers             | EC          | A      |
|                                                                                                 | Noncontact proximity sensors | EC          | A      |
| Wheel/motor sensors<br>(wheel/motor speed and position)                                         | Brush encoders               | PC          | P      |
|                                                                                                 | Potentiometers               | PC          | P      |
|                                                                                                 | Synchros, resolvers          | PC          | A      |
|                                                                                                 | Optical encoders             | PC          | A      |
|                                                                                                 | Magnetic encoders            | PC          | A      |
|                                                                                                 | Inductive encoders           | PC          | A      |
|                                                                                                 | Capacitive encoders          | PC          | A      |
| Heading sensors<br>(orientation of the robot in relation to<br>a fixed reference frame)         | Compass                      | EC          | P      |
|                                                                                                 | Gyroscopes                   | PC          | P      |
|                                                                                                 | Inclinometers                | EC          | A/P    |
| Acceleration sensor                                                                             | Accelerometer                | PC          | P      |
| Ground beacons<br>(localization in a fixed reference<br>frame)                                  | GPS                          | EC          | A      |
|                                                                                                 | Active optical or RF beacons | EC          | A      |
|                                                                                                 | Active ultrasonic beacons    | EC          | A      |
|                                                                                                 | Reflective beacons           | EC          | A      |
| Active ranging<br>(reflectivity, time-of-flight, and geo-<br>metric triangulation)              | Reflectivity sensors         | EC          | A      |
|                                                                                                 | Ultrasonic sensor            | EC          | A      |
|                                                                                                 | Laser rangefinder            | EC          | A      |
|                                                                                                 | Optical triangulation (1D)   | EC          | A      |
|                                                                                                 | Structured light (2D)        | EC          | A      |
| Motion/speed sensors<br>(speed relative to fixed or moving<br>objects)                          | Doppler radar                | EC          | A      |
|                                                                                                 | Doppler sound                | EC          | A      |
| Vision sensors<br>(visual ranging, whole-image analy-<br>sis, segmentation, object recognition) | CCD/CMOS camera(s)           | EC          | P      |
|                                                                                                 | Visual ranging packages      |             |        |
|                                                                                                 | Object tracking packages     |             |        |

A, active; P, passive; P/A, passive/active; PC, proprioceptive; EC, exteroceptive.

Figura 2.7: Tabella dei principali tipi di sensori, fonte: *"Introduction to Autonomous Mobile Robots"* [2]

### 2.3.2 Approccio con path planning

Il path planning invece è pensato per quelle situazioni in cui gli ostacoli sono statici, per quanto riguarda la conoscenza dell'ambiente, ci sono tecniche che permettono anche di avere una conoscenza minore.

Consiste nell'andare a determinare un percorso che il robot dovrà compiere, quindi attraverso la cinematica inversa verranno date tutte le direttive necessarie al veicolo per poter avanzare senza troppi problemi.

La domanda che può sorgere spontanea riguarda proprio la modalità con cui viene stabilito questo percorso, e quindi capire che criteri si seguono. Anche qui le possibilità sono diverse, poichè dipendono dalla tipologia di percorso che si cerca, ci sono tecniche che basano la loro costruzione sul voler minimizzare il tempo, altre per esempio vogliono minimizzare la distanza.

#### RoadMap

Tipologie di **roadmap** sono per esempio i **visibility graph** e i **diagrammi di Voronoi**, i primi 2.8 hanno come obiettivo quello di andare a rendere l'ambiente un vero e proprio grafo, andando a trasformare i vertici degli ostacoli in nodi, così come i punti di partenza e arrivo, in secondo luogo questi nodi saranno collegati tramite degli archi, che come vincolo avranno quello di non poter passare per gli ostacoli. In questo modo si avrà tra le mani un grafo e la scelta del percorso potrà essere fatta tramite algoritmi come quello di *Dijkstra*<sup>6</sup>, dovrà però essere passato anche un criterio di scelta, difatti in generale si può scegliere di minimizzare la distanza (si prendono tutti quegli archi la cui distanza totale è la minore), oppure il tempo, in questo ultimo caso bisognerebbe sfruttare la conoscenza sul modello cinematico, che permette di conoscere la velocità che un robot impiega quando si muove linearmente e quando ruota, e poi calcolare il numero di tratti lineari e di rotazione che ci sono, andando a valutarli per la distanza; per esempio se il tempo di rotazione

---

<sup>6</sup>Algoritmo inventato da Edsger Dijkstra, ha come scopo quello di trovare i cammini minimi in un grafo.

è maggiore, si predilige un percorso che presenta meno rotazioni e quindi cambi di orientamento.

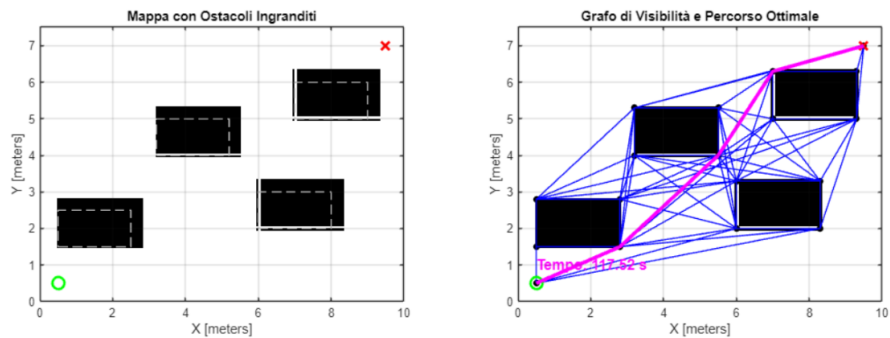


Figura 2.8: Visibility graph di un ambiente con ostacoli

I *diagrammi di Voronoi*, invece, sono un particolare tipo di decomposizione di uno spazio metrico, determinata dalle distanze rispetto ad un determinato insieme discreto di elementi dello spazio, e trovano applicazioni in tanti campi scientifici e ingegneristici.

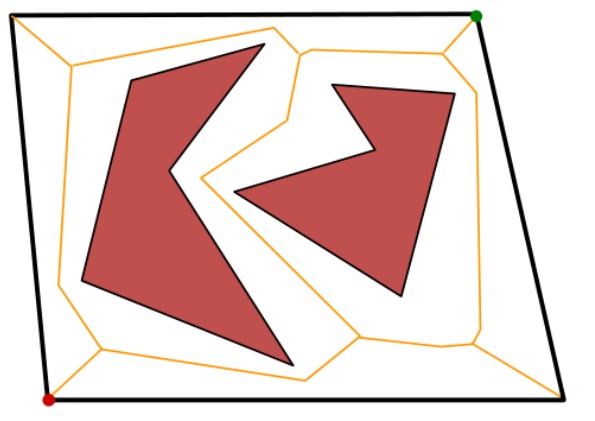


Figura 2.9: Diagrammi di Voronoi di un ambiente con ostacoli

Come si evince dall'immagine 2.9 però, spesso producono degli spazi troppo laschi, potrebbero essere adatti a robot, che per esempio non sono forniti di sensori troppo accurati.

### Cell Decomposition

Come il nome stesso suggerisce, quello che fa questa tecnica è **suddividere in celle** l'ambiente che si sta trattando, ovviamente vi sono diversi tipi di

suddivisione, che cambiano il tipo di percorso che si ottiene, e poi vengono associati dei grafi di connettività che rappresentino l'adiacenza tra le celle, come avviene nell'immagine sottostante 2.10.

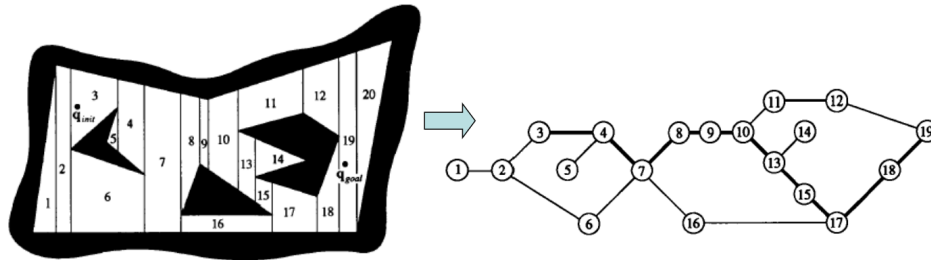


Figura 2.10: Cell decomposition

## Potential Field

Questo approccio è differente dai due precedentemente descritti, difatti si tratta di andare ad utilizzare dei ***campi potenziali***, in particolare sugli ostacoli verranno posti dei campi **localmente repulsivi**, mentre in corrispondenza dell'obiettivo, dunque nel punto dove si vuole fare arrivare il robot, verrà inserito un campo **globalmente attrattivo**, in questo modo il robot dovrebbe evitare gli ostacoli e arrivare al punto di destinazione.

Bisogna però prestare attenzione a come vengono definiti questi campi e a quali sono i coefficienti che vengono assegnati, difatti potrebbe accadere di avere un coefficiente di repulsione troppo alto, e se due ostacoli si trovano troppo vicini tra loro, questo potrebbe far cadere il robot in un minimo locale da cui non uscirebbe più (ci sono tuttavia delle tecniche che permettono di evitare/risolvere ciò).

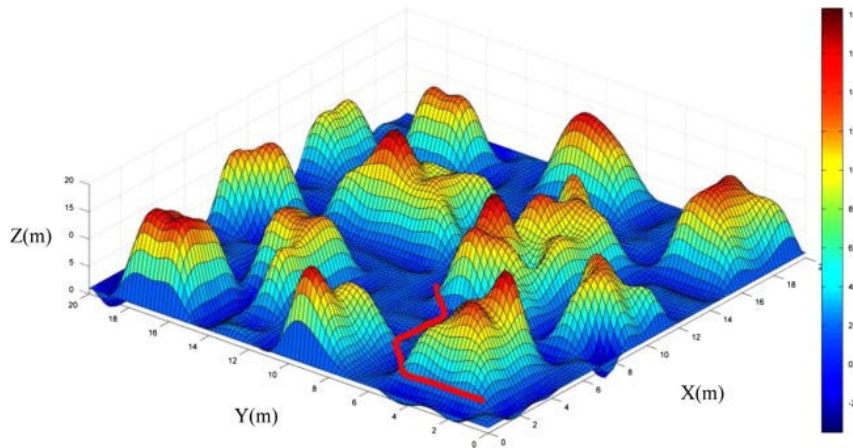


Figura 2.11: Immagine 3D di campi potenziali per il path planning, fonte: "Siddiqui, R. (2022, October 2). Path planning using potential field algorithm. Medium. Retrieved August 23, 2025, from <https://medium.com/@rymshasiddiqui/path-planning-using-potential-field-algorithm-a30ad12bdb08>"

### 2.3.3 Modello di controllo

Un primo approccio che si potrebbe pensare, per controllare il movimento del robot, è quello di andare a definire una traiettoria dall'inizio fino al punto di arrivo, in secondo luogo scomporre la traiettoria in diversi segmenti di movimento con una forma ben definita, quale può essere per esempio quella circolare oppure quella retta, a questo punto arrivati il problema del controllo consisterà nel pre-calcolare il percorso, in modo tale che sia fluido e che il robot possa attraversarlo senza problemi.

Questa tecnica, che prende il nome di *controllo ad anello aperto*, non prevede un *feedback* da parte del robot, e proprio per questo motivo presenta degli svantaggi notevoli, come per esempio il fatto che sarà complesso calcolare una traiettoria che risulti fluida, generalmente soprattutto se si parla di ostacoli, le traiettorie saranno molto "spezzate", e poi non avendo un riscontro da parte del robot, questo non si adatterà dinamicamente all'ambiente e quindi non sarà in grado di ricalcolare eventualmente dei percorsi.

Proprio per questi motivi si predilige un controllo in **feedback** o *retroazionato*, partendo proprio dalla generazione del modello cinematico. Sarà difatti richie-

sto un riferimento, ovvero una serie di posizioni, velocità e accelerazioni che vengano prese e generate come riferimento quando il robot è in movimento, da qui bisogna dare gli input agli elementi che permettono al robot di muoversi e quindi alle sue ruote, per farlo si sfrutta la conoscenza relativa alla *cinematica inversa* 2.2.3 e si generano i riferimenti.

Questi dovranno essere confrontati con quello che è lo stato del robot che viene misurato dai sensori 2.3.1, e si ottiene dunque una misura di errore data proprio dalla differenza tra i due stati, dunque per gestire l'errore c'è bisogno di avere un **controllore**.

I riferimenti che verranno fatti in questa sezione sono relativi all'immagine sottostante 2.12:

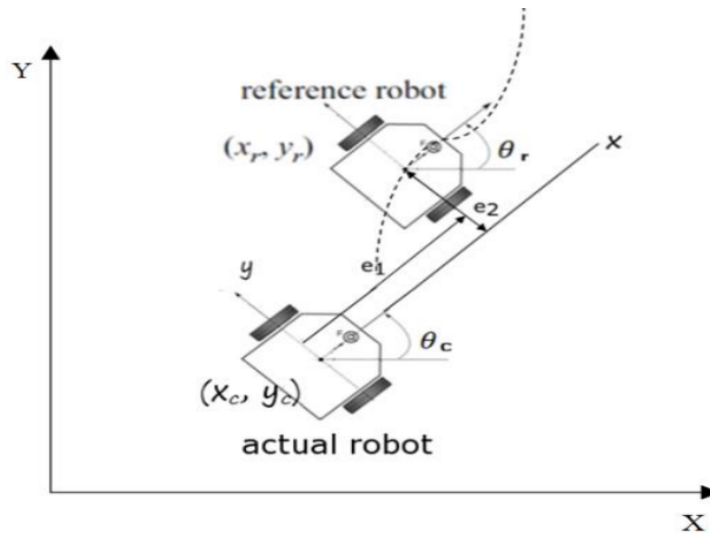


Figura 2.12: Traiettoria calcolata e stato del robot per evidenziare l'errore, fonte: "Differential-drive mobile robot control design based-on linear feedback control law." IOP Conference Series: Materials Science and Engineering. Vol. 190. No. 1. IOP Publishing, 2017 [8]

Il modello di errore lo si ottiene andando a sottrarre le coordinate della posa di riferimento:

$$q_r(t) = \begin{pmatrix} x_r(t) \\ y_r(t) \\ \theta_r(t) \end{pmatrix} \quad (2.24)$$

A quelle della posa del robot:

$$q_c(t) = \begin{pmatrix} x_c(t) \\ y_c(t) \\ \theta_c(t) \end{pmatrix} \quad (2.25)$$

Quello che si ottiene è dunque l'errore di posa del robot mobile che indichiamo con  $q_e(t)$  e corrisponde a:

$$q_e(t) = \begin{pmatrix} e_1(t) \\ e_2(t) \\ e_3(t) \end{pmatrix} \quad (2.26)$$

$q_e(t)$  è la trasformazione risultante tra la posizione di riferimento  $q_r(t)$  nel sistema di coordinate locali con la precedente posizione  $q_c(t)$  e l'asse delle X che è la direzione del robot data da  $\theta_c$ .

Dunque è possibile scriverla come:

$$q_e(t) = T_e(q_r(t) - q_c(t)) \quad (2.27)$$

Analogamente a quanto fatto con la trasformazione relativa al movimento nei paragrafi trattanti la cinematica differenziale, la trasformazione la applichiamo attraverso l'utilizzo dell'oggetto algebrico della matrice di rotazione, così facendo si ottiene la seguente forma:

$$q_e(t) = \begin{pmatrix} e_1(t) \\ e_2(t) \\ e_3(t) \end{pmatrix} = \begin{pmatrix} \cos\theta_c(t) & \sin\theta_c(t) & 0 \\ -\sin\theta_c(t) & \cos\theta_c(t) & 0 \\ 0 & 0 & 1 \end{pmatrix} \begin{pmatrix} x_r(t) - x_c(t) \\ y_r(t) - y_c(t) \\ \theta_r(t) - \theta_c(t) \end{pmatrix} \quad (2.28)$$

Da qui, vengano considerati  $v_r$  e  $\omega_r$  le velocità lineari e angolari di riferimento, mentre  $v_c$  e  $\omega_c$  quelle effettive misurate del robot, e poi venga anche considerata la presenza - più volte ribadita - del vincolo non olonomo relativo al differential drive, e si ottiene la seguente struttura:

$$\begin{pmatrix} \dot{e}_1(t) \\ \dot{e}_2(t) \\ \dot{e}_3(t) \end{pmatrix} = \begin{pmatrix} \cos e_3(t) & 0 \\ \sin e_3(t) & 0 \\ 0 & 1 \end{pmatrix} \begin{pmatrix} v_r(t) \\ \omega_r(t) \end{pmatrix} + \begin{pmatrix} -1 & e_2(t) \\ 0 & -e_1(t) \\ 0 & -1 \end{pmatrix} \begin{pmatrix} v_c(t) \\ \omega_c(t) \end{pmatrix} \quad (2.29)$$

A questo punto si vanno a combinare due modelli di controllo, quello **feed forward** e quello in **in retroazione**, quando parliamo del primo, implica che se si conosce il riferimento, dunque la traiettoria che si vuole seguire, si sa già che input dare al robot, e questo li esegue, sarebbe un buon modello anche solo, se non ci fossero errori, ma dal momento che ci sono errori nella posa e nei movimenti, entra in gioco il controllo in feedback e quindi in retroazione. Dopo questa breve digressione si può esprimere in formule quello che è stato appena detto:

$$v_c(t) = v_{ff}(t) - v_1(t) \quad (2.30)$$

[8]

$$v_c(t) = v_r(t) \cos e_3(t) - v_1(t)$$

$$\omega_c(t) = \omega_{ff}(t) - v_2(t)$$

$$\omega_c(t) = \omega_r(t) - v_2(t)$$

Sostituendo il tutto e riprendendo la precedente equazione si ottiene:

$$\begin{pmatrix} \dot{e}_1(t) \\ \dot{e}_2(t) \\ \dot{e}_3(t) \end{pmatrix} = \begin{pmatrix} 0 & \omega_c(t) & 0 \\ -\omega_c(t) & 0 & 0 \\ 0 & 0 & 0 \end{pmatrix} \begin{pmatrix} e_1(t) \\ e_2(t) \\ e_3(t) \end{pmatrix} + \begin{pmatrix} 0 \\ -\sin e_3(t) \\ 0 \end{pmatrix} v_r(t) + \begin{pmatrix} 1 & 0 \\ 0 & 0 \\ 0 & 1 \end{pmatrix} \begin{pmatrix} v_1(t) \\ v_2(t) \end{pmatrix} \quad (2.31)$$

C'è però un problema attorno a questa equazione, ovvero il fatto che non sia lineare e lo si capisce dai termini che la compongono, quindi non può essere così utilizzata, motivo per cui si ricorre ad una linearizzazione.

Per farlo si sceglie un **punto operativo**, e si linearizza attorno ad esso, che in questo caso è  $(e_1 = e_2 = e_3 = 0, v_1 = v_2 = 0)$ , il che significa che il robot in questo punto è esattamente nella traiettoria di riferimento e che i correttori

non stanno applicando azione, dal momento che l'errore è nullo, inoltre il punto è di **equilibrio** nel sistema degli errori.

Quello che si ottiene è:

$$\Delta \dot{e} = \begin{pmatrix} 0 & \omega_r(t) & 0 \\ -\omega_r(t) & 0 & v_r(t) \\ 0 & 0 & 0 \end{pmatrix} \cdot \Delta e(t) = \begin{pmatrix} 1 & 0 \\ 0 & 0 \\ 0 & 1 \end{pmatrix} \cdot \Delta v(t) \quad (2.32)$$

## Schema di controllo

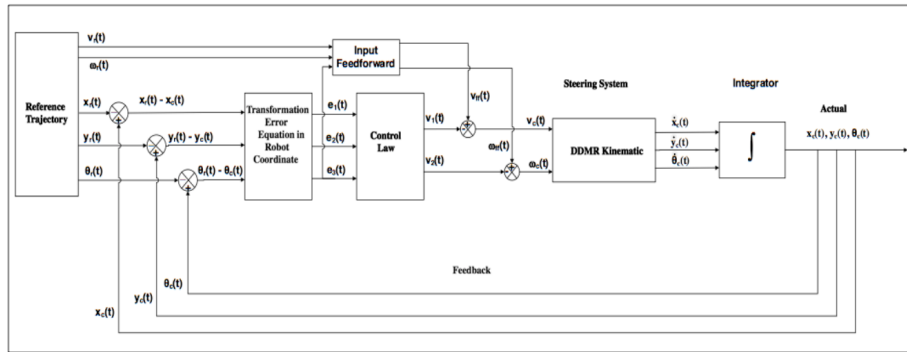


Figura 2.13: Schema di controllo retroazionato, fonte: *"Differential-drive mobile robot control design based-on linear feedback control law."* IOP Conference Series: Materials Science and Engineering. Vol. 190. No. 1. IOP Publishing, 2017 [8]

Il controllore consiste in tre stati:  $x(t)$ ,  $y(t)$  e  $\theta(t)$ , e poi nei due ingressi che corrispondono alle velocità, quindi quella lineare e quella angolare ( $v(t)$ ,  $\omega(t)$ ).

L'equazione generale di controllo nello spazio di stato lineare è:

$$v(t) = K \cdot e(t) \quad (2.33)$$

La  $K$  indica la matrice dei guadagni, perchè questo controllore agisce come proporzionale, anche se a differenza del classico (se parlassimo di PID questo sarebbe molto vicino al concetto di  $P$ , e quindi di controllore proporzionale, il  $P$  però non presenta una matrice  $K$  bensì un valore  $k_p$ ), ha una forma multivariabile e geometrica.

Da qui la legge di controllo diventa :

$$\begin{pmatrix} v_1(t) \\ v_2(t) \end{pmatrix} = \begin{pmatrix} -k_1 & 0 & 0 \\ 0 & -\text{sign}(v_r(t))k_2 & -k_3 \end{pmatrix} \begin{pmatrix} e_1(t) \\ e_2(t) \\ e_3(t) \end{pmatrix} \quad (2.34)$$

Come si può notare, in questa formulazione la velocità lineare  $v$  e quella angolare  $\omega$  prendono due simboli differenti, diventando  $v_1$  e  $v_2$ . Inoltre i parametri  $k_1$ ,  $k_2$  e  $k_3$  rappresentano quelli che sono i guadagni del controllore, costituenti il vettore di guadagno.

Una domanda che potrebbe sorgere spontanea riguarda i valori che i tre guadagni assumeranno, perchè ovviamente non potranno assumere valori casuali, ma saranno determinati confrontando le caratteristiche polinomiali *reali* del sistema con quelle *desiderate*.

I polinomi caratteristici li consideriamo in questa forma:

$$(s + 2\zeta\omega_n)(s^2 + 2\zeta\omega_n s + \omega_n^2) \quad (2.35)$$

e

$$s^3 + (4\zeta\omega_n)s^2 + (4\zeta^2\omega_n^2 + \omega_n^2)s + 2\zeta\omega_n^3 \quad (2.36)$$

Questo particolare parametro che si ritrova in ambo le equazioni  $\zeta$ , rappresenta il **coefficiente di smorzamento** ed è definito nel seguente dominio  $\zeta \in (0, 1)$ , mentre  $\omega_n$  è la pulsazione naturale.

Quando si progetta un controllore quello che si cerca di fare è andare a rispettare delle caratteristiche per il sistema, come il tempo di salita, la sovraelongazione, la sottoelongazione ecc.. Difatti in questo caso è stato aggiunto un polo in  $s = -2\zeta\omega_n$ , nel seguente modo è stato possibile ridurre il tempo di salita, e allo stesso tempo diminuire la sovraelongazione (o overshoot) del sistema.

Tutto questo è stato possibile anche visualizzarlo nello schema a blocchi in figura 2.13.

# Capitolo 3

## Pianificazione e navigazione

Il seguente capitolo, avrà l'obiettivo di addentrarsi maggiormente nello scopo del progetto, introducendo il framework utilizzato, e andando ad analizzare il funzionamento, prima in modo generale e poi soffermandosi in particolare sull'algoritmo di navigazione, da un punto di vista prettamente teorico.

### 3.1 Cos'è ROS?

Ros, acronimo di *Robot Operating System*, è un *middleware*<sup>1</sup> opensource, che ad oggi viene convenzionalmente utilizzato in molti robot, e permette una comunicazione fluida con essi, mettendo a disposizione ambienti simulativi e altri strumenti utili.

Attraverso questo framework i vari dispositivi potranno comunicare tra loro e scambiare dati, anche se sono scritti in linguaggi diversi.

#### 3.1.1 Architettura

Per apprendere al meglio il funzionamento dell'architettura di ROS, bisogna sapere che esistono tre livelli concettuali che la definiscono. [10].

---

<sup>1</sup>Un middleware è un software che agisce da intermediario tra sistemi eterogenei, quali possono essere per esempio sistemi software e hardware, nel caso di ROS, separa l'ambiente di software dall'hardware del robot, andando a "nascondere" quelle infrastrutture che potrebbero risultare "tediose".

## File system

Il primo livello è quello del *file system*, qui viene evidenziato come i file sono organizzati in memoria 3.1.

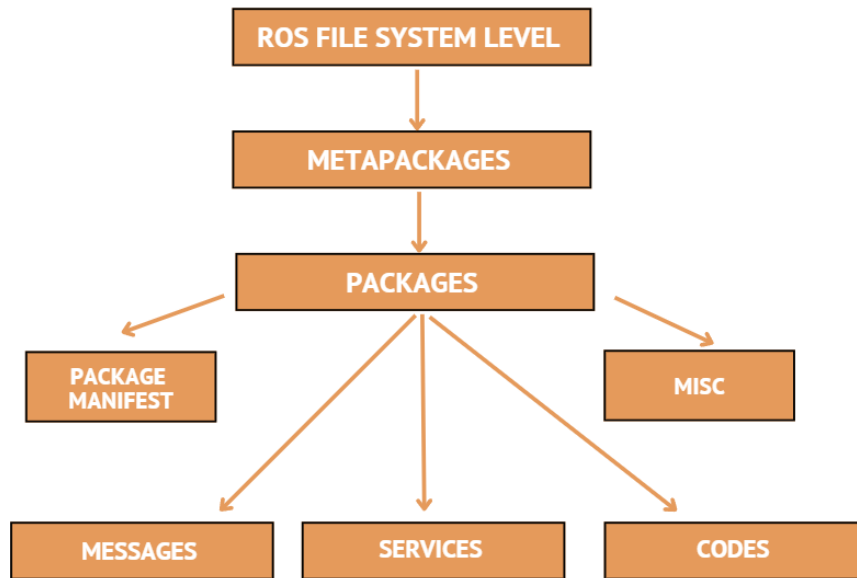


Figura 3.1: Architettura file system

Dall'immagine si può notare che il file system è basato su un'organizzazione in *pacchetti*: il pacchetto è l'unità fondamentale e contiene tutto l'occorrente per svolgere una funzione, all'interno di ogni pacchetto si possono trovare:

| Nome             | Formato  | Funzione                                                                               | Descrizione                                                                             |
|------------------|----------|----------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------|
| Package Manifest | .xml     | Serve al sistema per sapere come compilare ed interagire con altri pacchetti           | Contiene tutte le informazioni utili sul pacchetto                                      |
| Messages         | .msg     | Servono per la descrizione della struttura dei dati che i nodi inviano oppure ricevono | Definiscono i tipi di messaggi scambiati sui topic                                      |
| Services         | .srv     | Definiscono le chiamate a servizio                                                     | Ciascun servizio è composto da richiesta e risposta, rispettivamente request e response |
| Codes            | .cpp/.py | Servono per fare eseguire determinate operazioni                                       | Sono script o file sorgente scritti in linguaggi come Python e C++                      |
| Misc             | altri    | Servono per varie cose, come configurazioni, descrizioni ecc...                        | Possono essere launch file, URDF/XACRO, YAML ecc...                                     |

Tabella 3.1: Struttura dei file in un pacchetto ROS

Inoltre, più pacchetti se hanno una finalità comune sono raggruppati in un ***metapacchetto***, un esempio è il turtlebot3 utilizzato nelle simulazioni ai fini di questo elaborato.

## Computation Graph Level

Questo livello è composto da un grafo di calcolo, basato su una connessione di tipo *peer-to-peer*<sup>2</sup>, che elabora tutte le informazioni insieme.

In particolare la struttura è formata da:

- nodi: processi o eseguibili che eseguono dei calcoli o portano a termine delle attività, ciascun nodo può essere eseguito con frequenza diversa, scritto in linguaggi diversi e risiedere in differenti computer.
- topic: canali di comunicazione asincrona, e sono uno dei metodi che ROS utilizza per la comunicazione.
- master: svolge il compito di nodo intermedio che aiuta la connessione tra gli altri nodi, esso possiede infatti tutte le informazioni relative ai nodi presenti nell'ambiente di ROS.
- messaggi: contenuto dei topic che vengono scambiati.
- servizi: come i topic, anche questi sono dei canali di comunicazione, ma sono di tipo sincrono.
- action: queste sono simili ai servizi, ma vengono utilizzate per gestire operazioni di lunga durata e che richiedono feedback.
- parameter server: database chiave-valore centrale, un nodo infatti può mettere qui dentro una variabile e deciderne il livello di privacy.
- bags: vengono utilizzati per registrare e riprodurre i ROS topic. Un esempio è *rosbag* (il quale è stato utilizzato nell'elaborato per gestire le simulazioni), che permette di registrare i dati relativi ai sensori, alle metriche e ai parametri del robot.

---

<sup>2</sup>In questa tipologia di connessione, non vi è un server centrale e dei client, ma tutti i nodi possono agire sia da server che da client, la presenza del master serve solo per la fase iniziale di scoperta degli altri nodi.

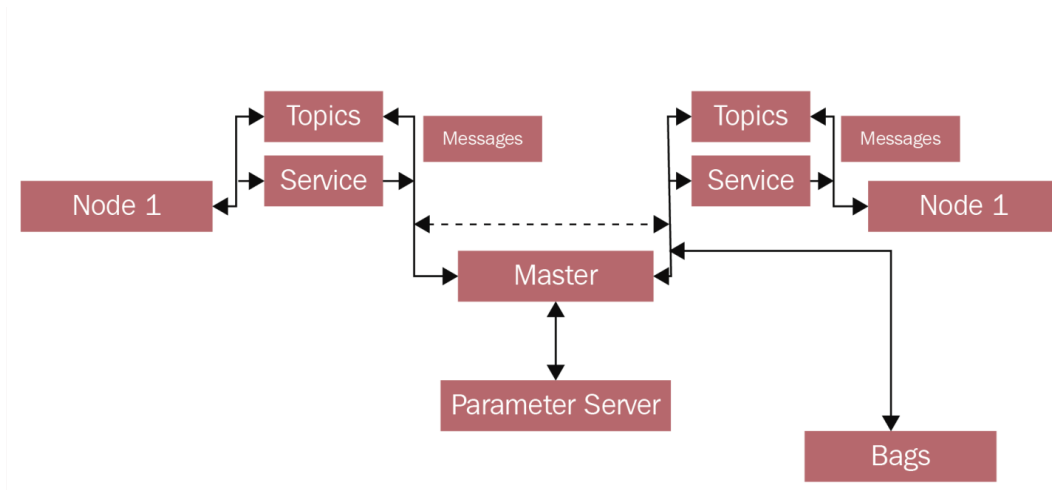


Figura 3.2: Livello di calcolo, fonte : *ROS Robotics projects. Packt Publishing Ltd, 2017*.<sup>[10]</sup>

### Community Level

Questo livello è composto da risorse messe a disposizione da e per gli utenti, dal momento che come già detto ROS è un framework opensource.

L'immagine mostra alcune delle risorse messe a disposizione :



Figura 3.3: Risorse, fonte: *ROS Robotics projects. Packt Publishing Ltd, 2017*.<sup>[10]</sup>

### 3.1.2 Comunicazione

Per mettere in comunicazione tra loro dei nodi, vi sono diverse modalità, nella sottosezione precedente ne sono state introdotte alcune, è utile andare ad analizzare come queste funzionano e come si comportano.

## Topic

Il **topic** può essere inteso come un vero e proprio *canale di comunicazione*, che permette lo scambio di messaggi tra i nodi attraverso il protocollo **Publish/Subscribe**.

Una comunicazione che utilizza questo tipo di protocollo è un modello *uno a molti*, dove vi è un **publisher**, ovvero un nodo che pubblica un determinato messaggio, e poi n **subscribers** che sono i nodi i quali ricevono il messaggio. Il messaggio viene comunicato e inviato tramite uno specifico topic, che agisce da canale per inoltrarlo.

Prima però di poter andare ad attivare qualsiasi nodo, è molto importante avviare il ROS Master, una volta che questo è attivo, il publisher può andare a connettersi con il master e passargli i dettagli relativi al topic di pubblicazione, quando un nodo subscriber viene attivato, si connette anch'esso al master e come anche il publisher gli passa i dati necessari, in questo modo i due nodi potranno connettersi e il master non avrà più un ruolo centrale nella loro comunicazione che avverrà con lo scambio del messaggio.

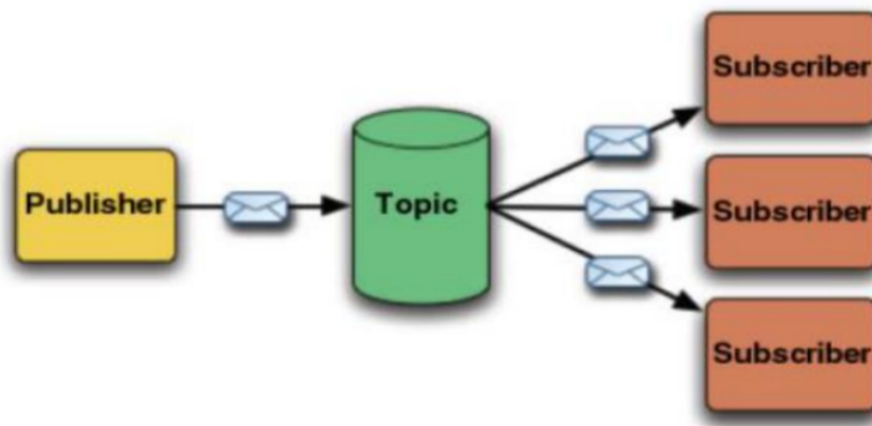


Figura 3.4: Schema publish-subscribe

Il fatto che la comunicazione sia **asincrona**, significa che il client non rimane bloccato finché non riceve risposta da parte del server, ma continua ad inviare i dati e le informazioni.

Quando un nodo viene pubblicato può essere ricercato nella lista dei topic da terminale nel seguente modo:

```
$ rostopic list | grep /nome_topic  
$ rostopic info /nome_topic  
$ rostopic echo /nome_topic
```

## Service

I **service** sono un meccanismo di comunicazione di tipo *sincrono*: il client resta bloccato fino a che il server non risponde alla sua richiesta.

In particolare sono basati sul meccanismo *request/response*, dunque un nodo (il client) manda una richiesta ad un nodo (il server) e aspetta una risposta.

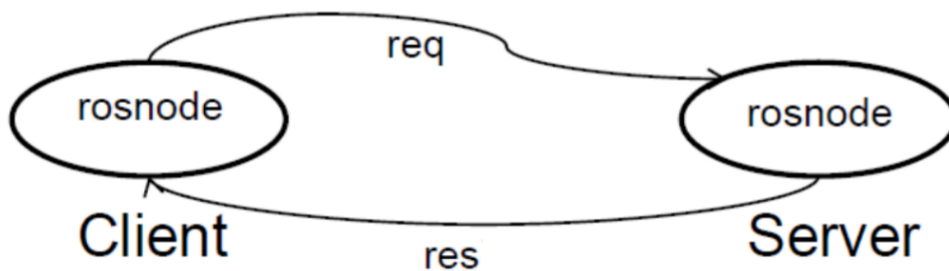


Figura 3.5: Schema comunicazione request-response

Come per i topic, anche per i service è possibile andare a vedere la lista di quelli attivi

```
$ rosservice list  
$ rostopic info /nome_service  
$ rostopic call /nome_service
```

Per chiamare un servizio:

```
$ rosservice call /nome_service
```

## Action

Le **actions** sono simili a delle chiamate *asincrone* ai servizi, la differenza sta nel fatto che quando un nodo chiama un'azione non deve aspettare che l'azione sia completata (definizione di comunicazione sincrona).

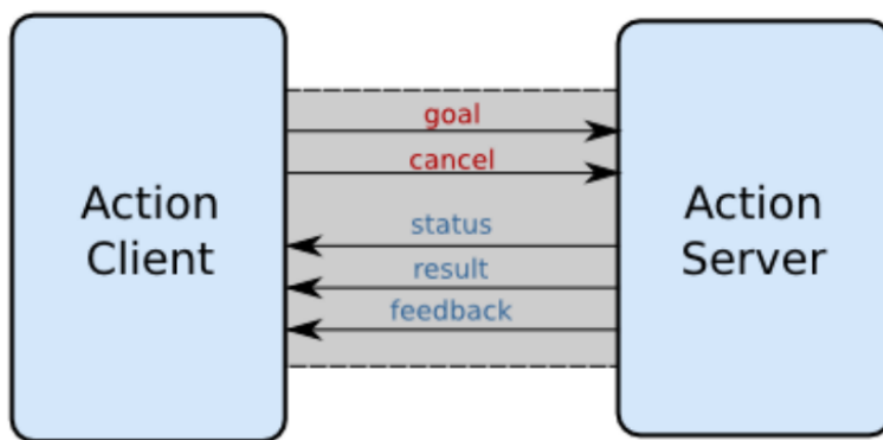


Figura 3.6: Schema di funzionamento delle actions

Si consideri per esempio di avere un Action Server di nome `turtlebot3`, allora richiedendo la lista dei topic si avrà:

```
/turtlebot3/goal  
/turtlebot3/cancel  
/turtlebot3/status  
/turtlebot3/result  
/turtlebot3/feedback
```

Il messaggio di un action server, come si può notare dall'immagine 3.6, è composto da tre parti, ovvero il goal (l'obiettivo, necessario per mandare un obiettivo al server), il risultato che viene mandato quando l'azione è stata

completata e il feedback che è un messaggio generato e inviato dal server durante lo svolgimento dell'azione, per informare su come sta andando l'azione.

### 3.1.3 Strumenti

| Strumento               | Funzione                                                                                                          |
|-------------------------|-------------------------------------------------------------------------------------------------------------------|
| Gazebo                  | Simulatore 3D di robot e ambienti, utilizzato per testare vari algoritmi senza dover utilizzare il modello reale. |
| RViz                    | Strumento di visualizzazione che mostra lo stato del robot, sensori, mappe e traiettorie pianificate.             |
| Rqt                     | Interfaccia grafica modulare con plugin per monitorare nodi, grafi delle comunicazioni, parametri e topic.        |
| rosbag                  | Strumento per registrare e riprodurre messaggi ROS, utile per il debug e la ripetizione di scenari.               |
| tf / tf2                | Libreria per la gestione delle trasformazioni di coordinate tra i vari frame del robot e dell'ambiente.           |
| move_base               | Pacchetto per la navigazione autonoma, che integra local path planning (es. DWA) e global planner.                |
| slam_toolbox / gmapping | Strumenti per la costruzione e aggiornamento di mappe mediante SLAM.                                              |

Tabella 3.2: Strumenti principali di ROS e loro funzione

## 3.2 Stack ROS Navigation

Lo **stack di navigazione** di ROS è un toolkit messo a disposizione dall'omonimo framework per la navigazione autonoma dei veicoli, in maniera specifica di quelli a trazione differenziale, e fornisce un set completo per la localizzazione, la pianificazione del percorso e l'utilizzo della mappa. [12]

Questo stack permette ai veicoli di muoversi liberamente in ambienti complessi che comprendono diversi ostacoli, e raggiungere gli obiettivi stabiliti, attraverso l'implementazione di algoritmi come SLAM, A\*, Dijkstra, AMCL ecc....

### 3.2.1 Requisiti hardware

Ci sono dei requisiti hardware che il robot deve rispettare se si vogliono avere dei buoni risultati applicando lo stack di navigazione, questi sono :

1. il pacchetto di navigazione lavora meglio quando viene utilizzato un robot a trazione differenziale, difatti è ottimizzato per questo tipo di robot e in generale per quelli omnidirezionali semplici; questo per via della relativa semplicità del modello cinematico, difatti in questi modelli le velocità vengono date sotto forma di velocità lineare e velocità angolare, che sono lo standard per la maggior parte degli approcci.
2. il robot dovrebbe montare su esso un laser planare, grazie al quale possa andare a ricostruire l'ambiente che lo circonda
3. anche la forma della base influisce sulla rendita, si predilige infatti una forma quadrata o circolare, ciò non esclude che non possa lavorare con altre forme, ma non ne verranno garantite le prestazioni.

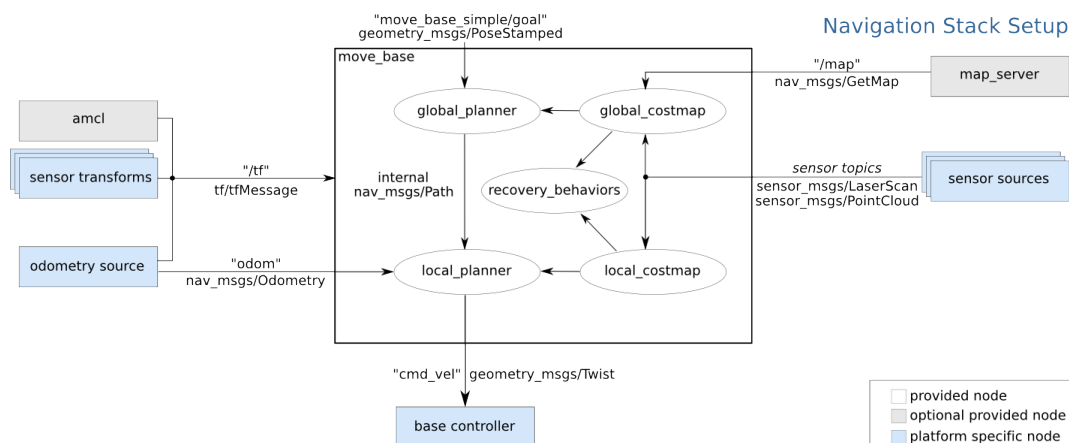


Figura 3.7: Navigation stack setup, fonte : <http://wiki.ros.org/navigation/Tutorials/RobotSetup>

Quelli presenti nel diagramma 3.7 sono dei blocchi funzionali che servono per interfacciarsi con lo stack di navigazione, alcuni di questi blocchi sono :

1. Sorgente di odometria (odometry source): i dati di odometria di un robot indicano la posa che ha il robot rispetto alla posa iniziale, la maggior parte di questi dati viene fornita dagli encoder delle ruote, dalle telecamere

e da IMU. La pubblicazione nello stack avviene sotto forma di messaggio del tipo **nav\_msgs/Odometry**, i messaggi di odometria sono un input **obbligatorio** per lo stack.

2. Sorgente sensoriale (sensor source): questi sono dei dati che vengono forniti per esempio dallo scanner laser o dalla point cloud e servono per mappare in modo opportuno l'ambiente circostante, insieme ai dati di odometria vengono utilizzati per costruire le cost map locali e globali. I dati sono del tipo **sensor\_msgs/LaserScan** o **sensor\_msgs/PointCloud**.
3. Trasformazione dei sensori(sensor transforms/tf): questo viene utilizzato per gestire la trasformazione tra sistemi di riferimento in modo dinamico e nel tempo, la pubblicazione avviene tramite ROS **tf**.
4. **base\_controller** : converte l'output dello stack di navigazione, il quale è sotto forma di messaggio di tipo: **geometry\_msgs/Twist** nelle velocità dei motori del robot.

### 3.2.2 Nodo **move\_base**

Il nodo **move\_base** viene dall'omonimo package, la cui funzione è quella di guidare il robot dalla sua posa di partenza fino all'obiettivo, con l'aiuto degli altri nodi di navigazione.

Questo nodo è un'implementazione del SimpleActionServer che prende la posa dell'obiettivo (goal) la quale si trova all'interno del messaggio **geometry\_msgs/PoseStamped**, goal inviato per esempio da un SimpleActionClient, in seguito il nodo **move\_base** lo sottoscrive da un topic che prende il nome di **move\_base\_simple/goal** che è l'input dello stack di navigazione. Dopo aver ricevuto la posa, il nodo si collega ad altri componenti come *global\_planner*, *local\_planner*, *recovery\_behavior*, *global\_costmap*, *local\_costmap*, e genera come output un comando di velocità (come già detto in precedenza viene espresso sotto forma di **geometry\_msgs/Twist**) e lo invia al base controller per muovere il veicolo verso il suo obiettivo.

Si è parlato di pacchetti come *global\_planner*, *local\_costmap* ecc... e, dal momento che questi saranno centrali anche nella parte prettamente pratica, è bene andare ad approfondire alcuni loro aspetti.

| Pacchetto              | Descrizione e Utilizzo                                                                                                                                                                                                                                                                                                                                     |
|------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| global_planner         | Offre librerie e nodi per pianificare il percorso migliore partendo dalla posa di partenza del robot fino all'obiettivo evitando eventuali ostacoli e rispettando i vincoli dell'ambiente. Al suo interno si possono trovare algoritmi come A*, Dijkstra e altri ancora                                                                                    |
| local_planner          | Guida il robot lungo una porzione del percorso globale, e utilizza i dati di odometria e sensori per riuscire a dare al controllo delle velocità adeguate.                                                                                                                                                                                                 |
| rotate_recovery        | Aiuta il robot ad uscire dagli ostacoli, attraverso una rotazione di 360°.                                                                                                                                                                                                                                                                                 |
| clear_costmap_recovery | Ha lo stesso obiettivo del precedente pacchetto, ma lo attua andando a cancellare la costmap corrente e ripristinandola alla mappa statica.                                                                                                                                                                                                                |
| costmap2D              | Mappa l'ambiente, e lo fa per esempio, attraverso delle mappe o griglie di occupazione 2D o 3D : rappresentano l'ambiente come griglia di celle, e ogni cella ha una determinata probabilità di essere libera oppure occupata, per la costruzione della mappa vengono utilizzati i dati ricevuti dai sensori.                                              |
| map_server             | Salva e carica la mappa generata dal pacchetto di costmap2D.                                                                                                                                                                                                                                                                                               |
| AMCL                   | Serve per andare a localizzare il robot nella mappa attraverso un metodo probabilistico, difatti implementa un approccio adattivo di Monte Carlo, e utilizza un filtro di particelle per tenere traccia del robot in una mappa conosciuta, come accennato nei requisiti hardware questo pacchetto funziona solamente su mappe costruite con il laser scan. |
| gmapping               | Implementa l'algoritmo di Fast Slam per costruire la mappa di occupazione 2D, combinando i dati odometrici e quelli del laser                                                                                                                                                                                                                              |

Tabella 3.3: Pacchetti dello stack di navigazione

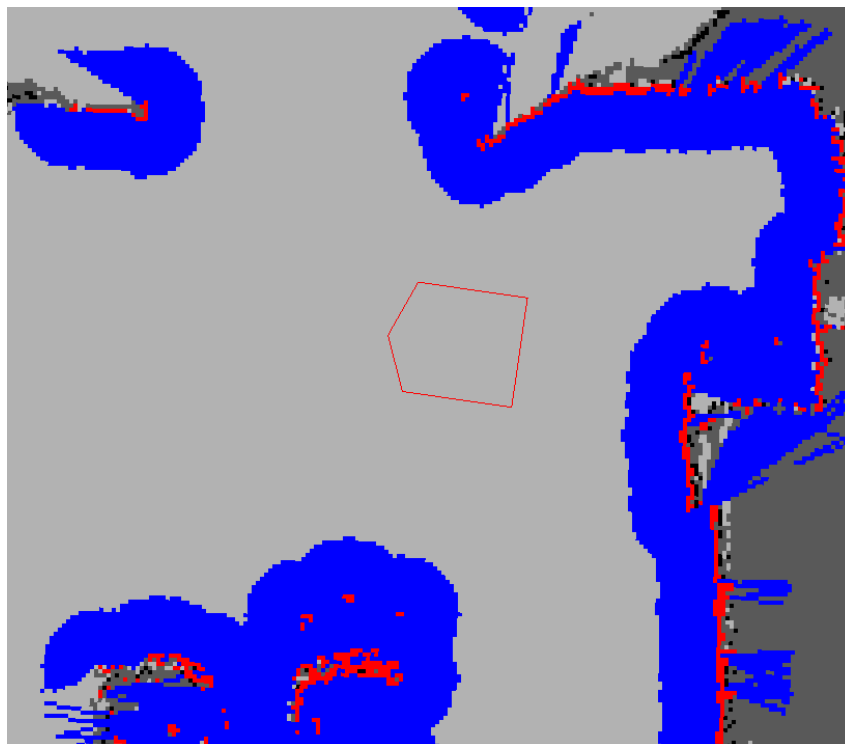


Figura 3.8: Esempio di costmap2D su rviz, fonte: [http://wiki.ros.org/costmap\\_2d?distro=noetic](http://wiki.ros.org/costmap_2d?distro=noetic)

## 3.3 Algoritmo DWA

### 3.3.1 Introduzione dell'algoritmo

L'algoritmo *DWA (Dynamic Window Approach)* è un algoritmo di *local path planning* 2.3.2 per i robot mobili, sviluppata da *Fox, Burgard e Thrun* nel 1997 [11].

Il suo scopo è quello di scegliere in tempo reale la velocità lineare e angolare da applicare al robot, ovviamente andando a tenere conto dei vari vincoli : quelli cinematici e quindi relativi alle massime velocità lineari e angolari, quelli degli ostacoli che dovrà evitare e infine quelli determinati dal percorso come per esempio la partenza e l'arrivo.

La particolarità di questo approccio è che non lavora nello spazio delle traiettorie come succede con diversi approcci e algoritmi di path planning, alcuni dei quali descritti in precedenza, ma lavora nello **spazio delle velocità**( $v$ ,

$\omega$ ), spazio che risulta filtrato dalle limitazioni del robot (vincoli cinematici e dinamici).

In aggiunta a queste restrizioni, si hanno anche quelle relative alla *sicurezza* : vengono prese soltanto le velocità che risultano sicure rispetto agli ostacoli.

Proprio questi primi controlli costituiscono il **primo passo dell'algoritmo**.

Il **secondo passo** consiste nell'andare a scegliere tra le velocità rimaste nello spazio di ricerca, quella che massimizza la *funzione obiettivo*.

Una importante considerazione che bisogna fare prima di procedere nella spiegazione dell'algoritmo, è quella relativa all'approssimazione della traiettoria del robot a moto differenziale come un insieme di archi di circonferenza.

Si prenda come punto di partenza la cinematica già introdotta nel corso del precedente capitolo 2.2, per cui:

$$\dot{x} = v \cdot \cos(\theta), \quad \dot{y} = v \cdot \sin(\theta), \quad \dot{\theta} = \omega \quad (3.1)$$

Quindi, dando una serie di comandi al robot, si possono verificare diversi scenari di movimento:

- $v = 0, \quad \omega = 0$ , il robot è fermo.
- $v > 0, \quad \omega = 0$ , il robot si muove linearmente.
- $v > 0, \quad \omega > 0$  o  $v = 0, \quad \omega > 0$ , il robot durante il movimento forma degli archi di circonferenza/rotazioni.

Di conseguenza si può affermare che quando le due velocità sono mantenute costanti la traiettoria è un arco di circonferenza (o una retta) il cui raggio è  $R = \frac{v}{\omega}$ , ogni coppia  $(v, \omega)$  può essere approssimata ad un arco circolare per un breve intervallo temporale.

Ed è proprio questo che fa il DWA, esplora quello che è lo spazio di ricerca, andando a tradurre ogni coppia di valori in una *traiettoria circolare percorribile dal robot*. Per generare una traiettoria fino ad un obiettivo, per i successivi  $n$  intervalli di tempo (che vanno da  $t_0$  a  $t_n$ ) il robot dovrà determinare la coppia di velocità, sotto il presupposto che la traiettoria risultante non vada a collidere

con un ostacolo. Un'assunzione che viene fatta dall'algoritmo per rendere tutto computazionalmente trattabile è che venga considerato soltanto il primo intervallo di tempo, gli altri  $n-1$  sono reputati costanti (non avremo dunque un'accelerazione tra  $t_1$  e  $t_n$ ).

### Vincoli imposti dagli ostacoli

Gli ostacoli creano dei vincoli restrittivi sulle velocità rotazionali e lineari, questo perché se si ha un ostacolo in prossimità, la massima velocità di una curvatura dipende anche dalla distanza che la curvatura ha rispetto al prossimo ostacolo. Una velocità viene considerata ammissibile se il robot è capace di fermarsi prima di raggiungere l'ostacolo.

A questo punto venga definito l'insieme delle **velocità ammissibili**, traducendo in linguaggio formale ciò che è stato appena affermato:

$$V_a = \{(v, \omega) | v \leq \sqrt{2 \cdot \text{dist}(v, \omega) \cdot \dot{v}} \quad \wedge \quad \omega \leq \sqrt{2 \cdot \text{dist}(v, \omega) \cdot \dot{\omega}}\} \quad (3.2)$$

Dove:

- $v$  e  $\omega$  sono rispettivamente le velocità lineari e angolari.
- $\text{dist}(v, \omega)$  corrisponde alla distanza dall'ostacolo più vicino.
- $\dot{v}$  e  $\dot{\omega}$  sono le massime decelerazioni lineari e angolari, quindi rappresentano le massime velocità di frenata.

Per poter visualizzare questo spazio si prenda un ambiente che faccia da esempio:

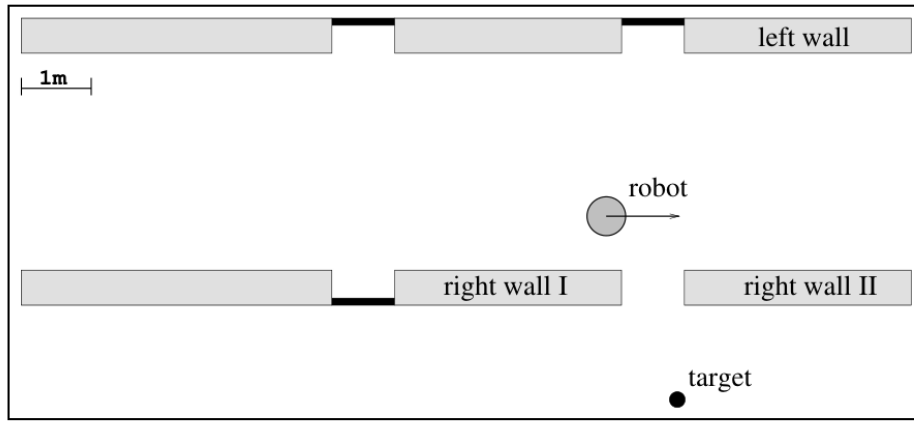


Figura 3.9: Situazione di partenza in un ambiente con ostacoli, fonte: *"The dynamic window approach to collision avoidance."* (1997) [11]

La figura che segue 3.10 mostra le velocità ammissibili in questa situazione, date le accelerazioni  $\dot{v} = 50 \frac{cm}{s^2}$  e  $\dot{\omega} = 60 \frac{deg}{s^2}$

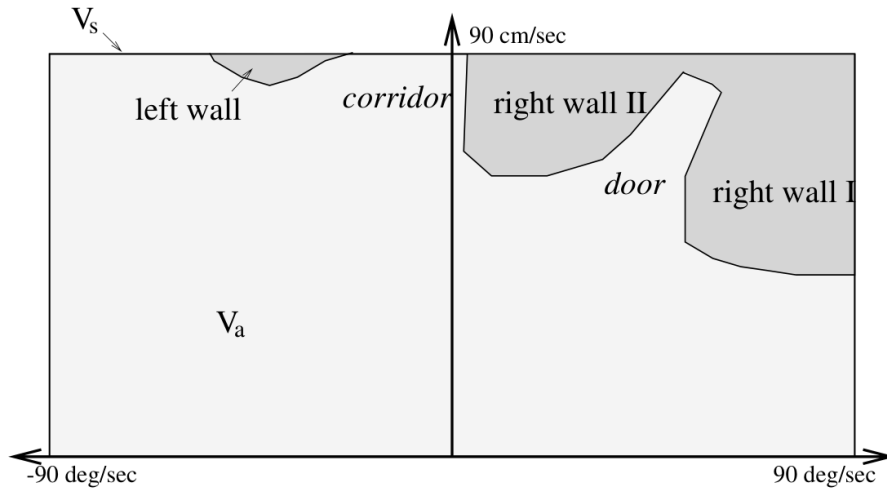


Figura 3.10: Spazio di velocità, fonte : *"The dynamic window approach to collision avoidance."* (1997) [11]

Le velocità non ammissibili sono evidenziate nelle aree più scure, queste causerebbero una curva che porterebbe alla collisione con le pareti; le zone non ammissibili vengono ricavate a partire da informazioni di prossimità date dai vari sensori che il robot possiede 2.3.1.

### Finestra Dinamica(Dynamic Window)

A questo punto, è stato ristretto lo spazio delle velocità, introducendo lo spazio delle velocità ammissibili che elimina le combinazioni che porterebbero ad una collisione, ora però bisogna pensare anche al fatto che il robot non può istantaneamente assumere una qualsiasi velocità di  $V_a$ , questo perché ci sono dei limiti fisici dati dal robot stesso, come i limiti di accelerazione dei motori, dunque si ha bisogno di una finestra dinamica che verrà indicata con  $V_d$ .

In formule la finestra viene definita come :

$$V_d = \{(v, \omega) \mid v \in [v_a - \dot{v} \cdot t, v_a + \dot{v} \cdot t] \quad \wedge \quad \omega \in [\omega_a - \dot{\omega} \cdot t, \omega_a + \dot{\omega} \cdot t]\} \quad (3.3)$$

Sempre assumendo di osservare lo stesso esempio preso precedentemente in considerazione, si evidenzi la finestra dinamica anche graficamente :

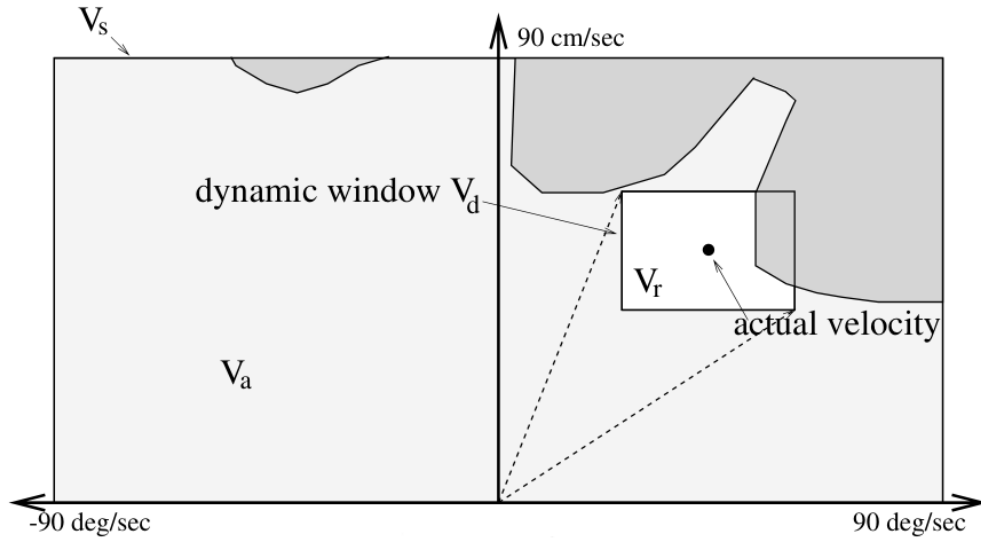


Figura 3.11: Finestra dinamica, fonte: "The dynamic window approach to collision avoidance." (1997) [11]

Qui l'intervallo di tempo è di  $0.25 \text{ s}$ , e nell'immagine si può notare che le due frecce tratteggiate che puntano agli angoli del rettangolo denotano un bound per la curvatura.

Unendo tutte le restrizioni questo è quello che si ottiene :

$$V_r = V_s \cap V_a \cap V_d \quad (3.4)$$

L'insieme risultante racchiude tutte le velocità sicure e raggiungibili, tra queste l'algoritmo dovrà scegliere la migliore secondo una funzione obiettivo che considera più criteri.

### Massimizzare la funzione obiettivo

La funzione obiettivo è costituita da diversi criteri che permettono una buona navigazione del robot.

Il **primo criterio** da prendere in considerazione è l'***orientamento desiderato (target heading)*** ovvero l'allineamento con l'obiettivo che si vuole raggiungere, non basta infatti che il robot eviti gli ostacoli, ma serve che sia direzionato verso il punto di arrivo.

Per calcolare questo allineamento però non basta avere la posa attuale del robot, in quanto questo se in movimento potrebbe già essere disallineato, motivo per cui si ricorre ad una *predizione della posa*, la posa viene predetta usando ogni velocità candidata che si sta valutando, da qui si calcola poi l'effettivo sfasamento che si ha rispetto al target, il target heading è quindi indicato come  $180 - \theta$  dove  $\theta$  indica l'angolo di disallineamento.

Si può notare ciò nell'immagine 3.12 :

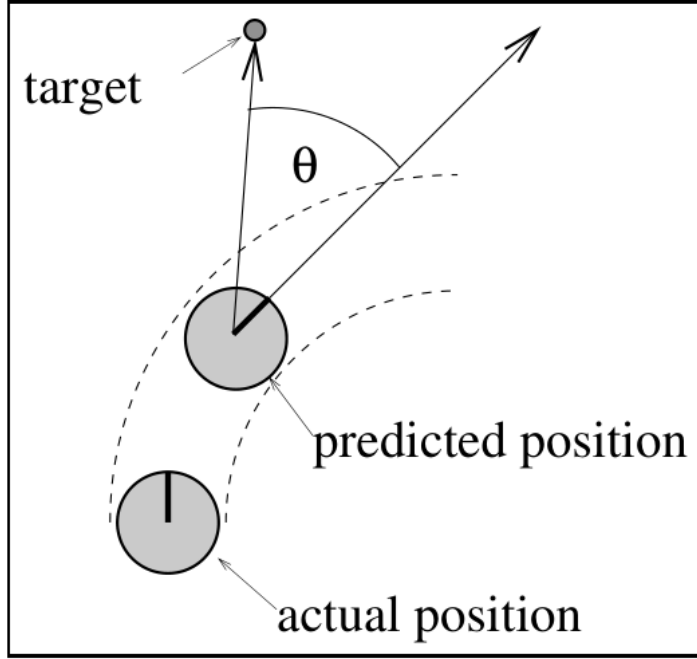


Figura 3.12: Angolo  $\theta$  rispetto al target [11]

Si passi poi al **secondo criterio**, che invece è la **distanza di sicurezza (clearance)** che indica la distanza dall'ostacolo più vicino :

$$dist(v, \omega) \quad (3.5)$$

L'ultimo criterio è la **velocità (velocity)** e viene usata per valutare le traiettorie, favorendo quelle che permettono al robot di avanzare più rapidamente:

$$velocity(v, \omega) \quad (3.6)$$

Mettendo tutto insieme si ottiene la funzione obiettivo da massimizzare :

$$G(v, \omega) = \sigma(\alpha \cdot heading(v, \omega) + \beta \cdot dist(v, \omega) + \gamma \cdot velocity(v, \omega)) \quad (3.7)$$

I pesi  $\alpha, \beta$  e  $\gamma$  sono dei pesi di bilanciamento, che variano poi in base all'applicazione.

Inoltre per rendere maggiormente fluide e regolari le traiettorie ed evitare dunque bruschi cambi di direzione, si applica uno *smoothing* ovvero uno smussamento.

### **3.3.2 Implementazione in ROS**

All'interno di ROS l'algoritmo viene implementato tramite il pacchetto `dwa_local_planner` che fornisce un controller il quale pilota il robot nel piano, usando una mappa il planner crea una traiettoria cinematica per far muovere il robot dalla partenza fino all'arrivo, durante il percorso il planner costruisce una funzione di costo rappresentata come griglia, viene quindi assegnato un costo o per meglio dire un peso ad ogni cella. Quello che il controllore dovrà fare è usare la funzione per determinare le velocità da inviare al robot.

# Capitolo 4

## Esperimenti e risultati

Nel seguente capitolo si andranno a mettere in pratica le informazioni descritte nei capitoli precedenti, utilizzando alcuni strumenti messi a disposizione da ROS. Nella prima parte della trattazione si potranno osservare le simulazioni fatte in ambiente virtuale e relativi risultati, terminate queste si passerà ad una visualizzazione reale di quello che accade, attraverso delle prove svolte nel laboratorio.

### 4.1 Setup della simulazione

#### 4.1.1 Robot utilizzato

Il robot utilizzato all'interno di queste simulazioni è un **TurtleBot3** di tipo **Waffle** a trazione differenziale, così è come si presenta nell'immagine di riferimento :

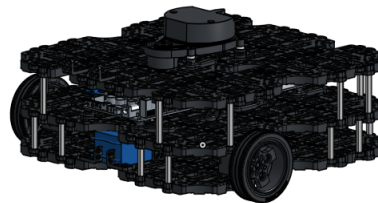
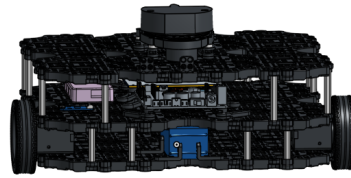


Figura 4.1: Robot TurtleBot3 Waffle visto dall'alto e lateralmente con un modello tridimensionale

Andando ad analizzarlo più nel dettaglio :

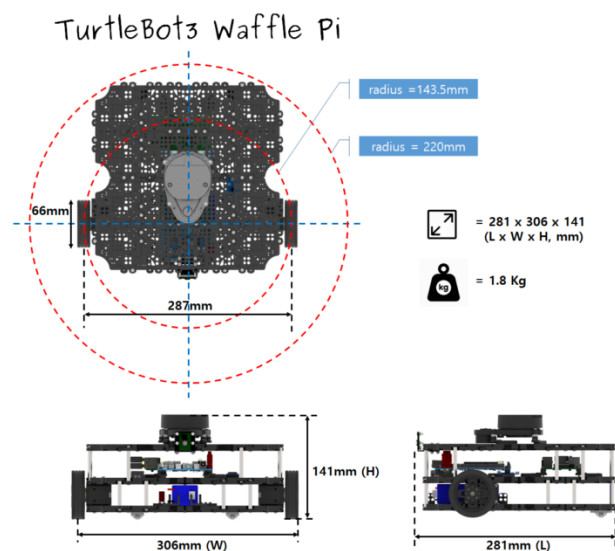


Figura 4.2: Visione più dettagliata del robot, fonte : <https://emanual.robotis.com/docs/en/platform/turtlebot3/features/>

Nella tabella sottostante sono riportate alcune caratteristiche hardware del robot in questione :

| <b>Elemento</b>                 | <b>Waffle Robot</b>                                          |
|---------------------------------|--------------------------------------------------------------|
| Massima velocità di traslazione | $0.26 \frac{m}{s}$                                           |
| Massima velocità di rotazione   | $1.82 \frac{rad}{s}$                                         |
| Carico massimo                  | $30 \text{ kg}$                                              |
| Dimensioni(L x W x H)           | $281 \text{ mm} \times 306 \text{ mm} \times 141 \text{ mm}$ |
| Peso                            | $1.8 \text{ kg}$                                             |
| Soglia di salita                | $10 \text{ mm}$ o meno                                       |
| Tempo di funzionamento previsto | $2 \text{ h}$                                                |
| Tempo di caricamento previsto   | $2 \text{ h } 30 \text{ m}$                                  |
| SBC <sup>1</sup>                | Raspberry Pi 4                                               |
| MCU <sup>2</sup>                | 32-bit ARM Cortex®-M7 with FPU                               |
| Controllore da remoto           | RC-100B + BT-410 Set (Bluetooth 4, BLE)                      |
| Bottoni e Interruttori          | Push buttons x 2, Reset button x 1, Dip switch x 2           |
| Connessione con PC              | USB                                                          |

Tabella 4.1: Specifiche hardware

È utile anche conoscere le parti specifiche di cui è composto :

|                           | Part Name                                           | Burger | Waffle Pi |
|---------------------------|-----------------------------------------------------|--------|-----------|
| <b>Chassis Parts</b>      | Waffle Plate                                        | 8      | 24        |
| .                         | Plate Support M3x35mm                               | 4      | 12        |
| .                         | Plate Support M3x45mm                               | 10     | 10        |
| .                         | PCB Support                                         | 12     | 12        |
| .                         | Wheel                                               | 2      | 2         |
| .                         | Tire                                                | 2      | 2         |
| .                         | Ball Caster                                         | 1      | 2         |
| .                         | Camera Bracket                                      | 0      | 1         |
| <b>Motors</b>             | DYNAMIXEL ( <a href="#">XL430-W250-T</a> )          | 2      | 0         |
| .                         | DYNAMIXEL ( <a href="#">XM430-W210-T</a> )          | 0      | 2         |
| <b>Boards</b>             | <a href="#">OpenCR1.0</a>                           | 1      | 1         |
| .                         | *Raspberry Pi                                       | 1      | 1         |
| .                         | USB2LDS                                             | 1      | 1         |
| <b>Remote Controllers</b> | BT-410 Set (Bluetooth 4, BLE)                       | 0      | 1         |
| .                         | RC-100B (Remote Controller)                         | 0      | 1         |
| <b>Sensors</b>            | ** <a href="#">LDS-01</a> or <a href="#">LDS-02</a> | 1      | 1         |
| .                         | <a href="#">Raspberry Pi Camera v2.1</a>            | 0      | 1         |
| <b>Memory</b>             | MicroSD Card                                        | 1      | 1         |
| <b>Cables</b>             | Raspberry Pi Power Cable                            | 1      | 1         |
| <b>Cables</b>             | Raspberry Pi Power Cable                            | 1      | 1         |
| .                         | Li-Po Battery Extension Cable                       | 1      | 1         |
| .                         | DYNAMIXEL to OpenCR Cable                           | 2      | 2         |
| .                         | USB Cable                                           | 2      | 2         |
| .                         | Camera Cable                                        | 0      | 1         |
| <b>Powers</b>             | SMPS 12V5A                                          | 1      | 1         |
| .                         | A/C Cord                                            | 1      | 1         |
| .                         | LIPO Battery 11.1V 1,800mAh                         | 1      | 1         |
| .                         | LIPO Battery Charger                                | 1      | 1         |

|                      |                 |    |     |
|----------------------|-----------------|----|-----|
| <b>Miscellaneous</b> | PH_M2x4mm_K     | 8  | 8   |
| .                    | PH_T2x6mm_K     | 4  | 8   |
| .                    | PH_M2x12mm_K    | 0  | 4   |
| .                    | PH_M2.5x8mm_K   | 16 | 16  |
| .                    | PH_M2.5x12mm_K  | 0  | 20  |
| .                    | PH_T2.6x12mm_K  | 16 | 0   |
| .                    | PH_M2.5x16mm_K  | 4  | 4   |
| .                    | PH_M3x8mm_K     | 44 | 140 |
| .                    | NUT_M2          | 0  | 4   |
| .                    | NUT_M2.5        | 20 | 24  |
| .                    | NUT_M3          | 16 | 96  |
| .                    | Rivet_1         | 14 | 22  |
| .                    | Rivet_2         | 2  | 2   |
| .                    | Spacer          | 4  | 4   |
| .                    | Silicone Spacer | 0  | 4   |
| .                    | Bracket         | 5  | 6   |
| .                    | Adapter Plate   | 1  | 1   |

Figura 4.3: Lista componenti

### 4.1.2 Creazione dell'ambiente

La creazione dell'**ambiente simulativo** è avvenuta attraverso l'utilizzo dei software messi a disposizione da ROS(*Gazebo* e *RViz*) e dei vari pacchetti di navigazione.

#### Progettazione ambiente su gazebo

Per poter procedere in questa pratica inizialmente vi è la necessità di scegliere la tipologia di robot coinvolta, una volta scelta va inserita all'interno del file di lancio, oppure specificato ogni qual volta si sceglie di lanciare il mondo di gazebo; all'interno di questo progetto il tipo di robot è stato specificato nel file rinominato : *start\_navigation.launch*, in particolare la suddetta riga appare scritta nel seguente modo :

```

...
<arg name="model" value="waffle"/>
<!--env name="TURTLEBOT3_MODEL" value="$(arg model)" /-->
...

```

A questo punto, si può avanzare e aprire gazebo per poter effettivamente *creare un ambiente*, inizialmente si avrà di fronte una schermata che presenta solamente il robot ad un punto di spawn definito (sempre all'interno dei vari file di launch), per aggiungere elementi delimitanti (come muri e/o ostacoli) bisogna entrare nella modalità di ***"building editor"*** :

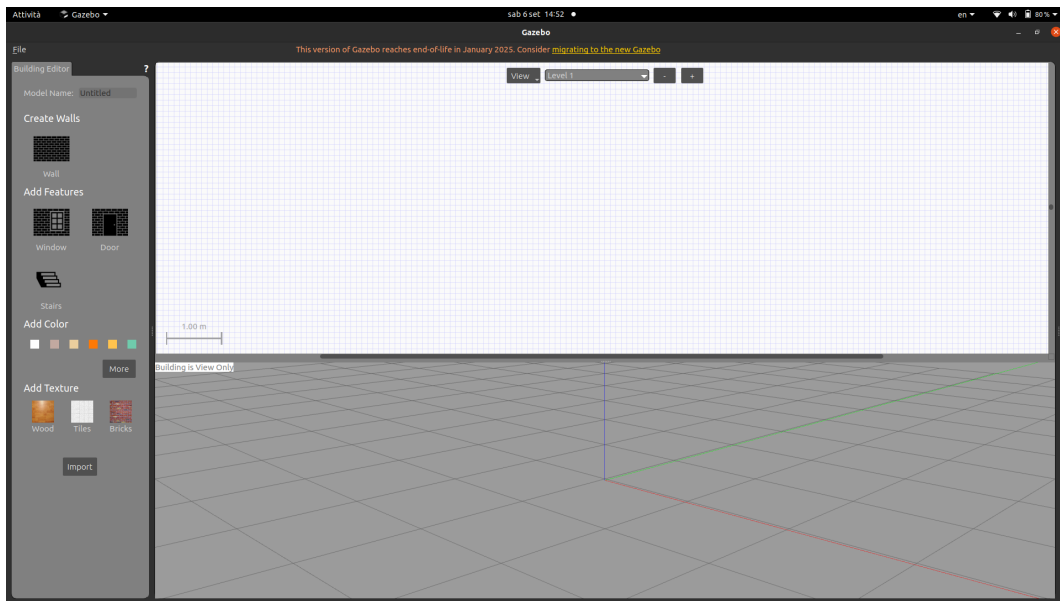


Figura 4.4: Building Editor di Gazebo

Come si può notare, la schermata viene principalmente divisa in tre parti, la prima ovvero quella superiore è volta all'inserimento dei vari elementi che verranno scelti dalla lista di sinistra, e il tutto verrà riportato nell'ambiente 3D rappresentato nel frame inferiore.

Le scelte relative agli elementi da aggiungere non sono limitate solamente a quelle che si possono vedere di default, ma si estendono anche con database o plugin scaricabili.

L'obiettivo iniziale è quello di creare uno spazio con ostacoli e limiti, che tuttavia permetta l'applicazione di variazioni, in modo tale da poter visualizzare il comportamento del robot in contesti maggiormente vincolanti, motivo per cui l'ambiente iniziale non deve risultare troppo complesso e articolato :

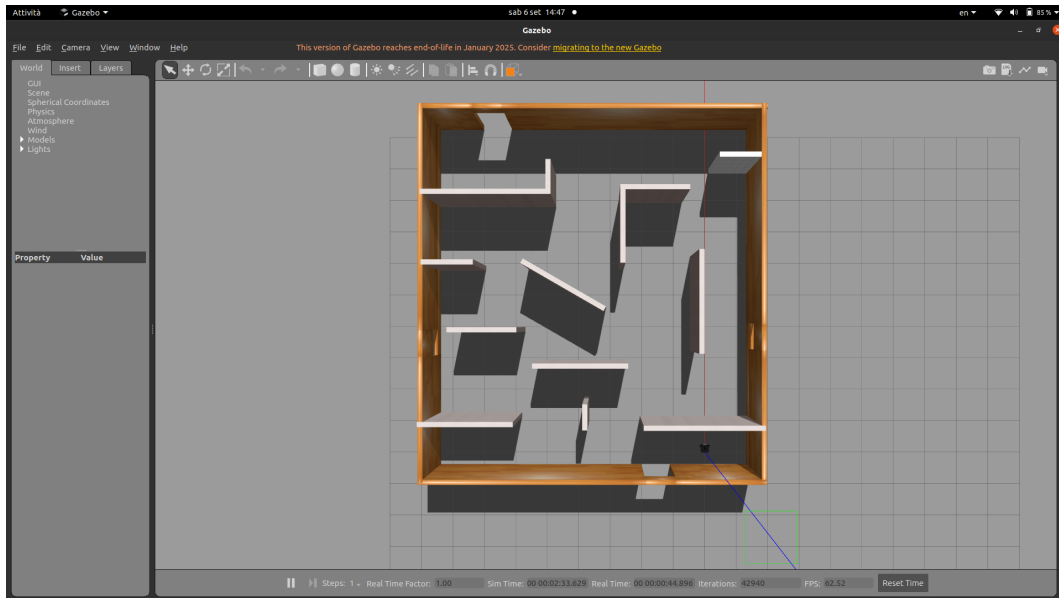


Figura 4.5: Anteprima dell'ambiente iniziale

Dopo aver impostato il mondo bisogna andare a salvarlo, in modo tale da poterci lavorare sopra senza troppi problemi.

## Generazione delle mappe con RViz

Una volta conclusa la delineazione del contesto su gazebo, è doveroso trasferirsi su RViz, questo perché permette la visualizzazione in tempo reale delle informazioni relative al robot e all'ambiente, fornendo una rappresentazione grafica dei dati prodotti dal sistema come l'odometria, i sensori e le mappe. Per poter ricostruire la mappa a partire dal mondo di Gazebo, quello che bisogna fare è far muovere il robot che attraverso il laser individuerà la presenza dei muri e degli ostacoli.

Prima si apre RViz, poi si sfrutta una combinazione tra *SLAM* e *turtlebot3 teleop*.

```
$ roslaunch turtlebot3_slam turtlebot3_slam.launch
```

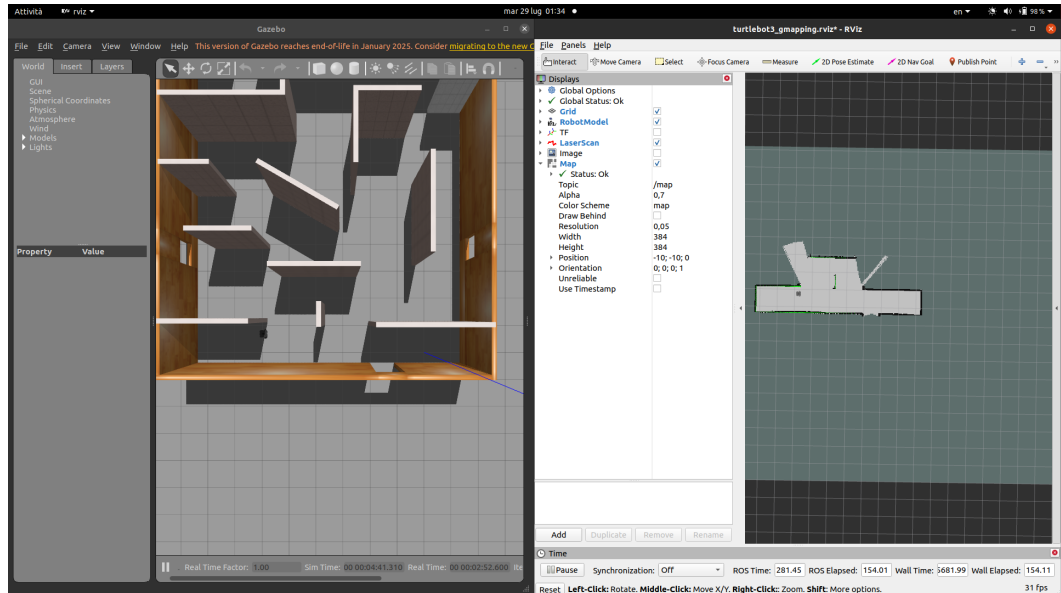


Figura 4.6: Utilizzo di SLAM per costruzione dell'ambiente in RViz

Per poter far muovere il robot si usa la tastiera attraverso il seguente comando da terminale :

```
$ roslaunch turtlebot3_teleop turtlebot3_teleop_key.launch
```

```

/home/francesca/catkin_ws/src/turtlebot3/turtlebot3_teleo...
/home/francesca/catkin_ws/src/turtl...  francesca@francesca-Victus-by-HP-La...
ROS_MASTER_URI=http://localhost:11311

setting /run_id to b23eec42-8b3a-11f0-9f34-f9bfb139469
process[rosout-1]: started with pid [21651]
started core service [/rosout]
process[turtlebot3_teleop_keyboard-2]: started with pid [21654]

Control Your TurtleBot3!
-----
Moving around:
      w
    a  s  d
      x

w/x : increase/decrease linear velocity (Burger : ~ 0.22, Waffle and Waffle Pi :
~ 0.26)
a/d : increase/decrease angular velocity (Burger : ~ 2.84, Waffle and Waffle Pi
: ~ 1.82)

space key, s : force stop

CTRL-C to quit

```

Figura 4.7: Guida del robot da tastiera

In questo modo si delinea la seguente mappa

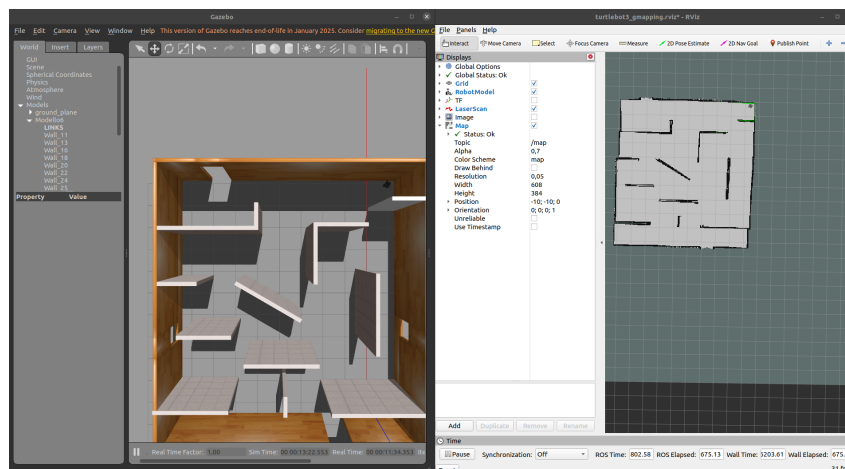


Figura 4.8: Fine slam

Per poter renderla permanente e quindi aprirla in qualsiasi momento, dovrà essere salvata, al salvataggio della mappa verranno creati e salvati due file : il primo riporta come estensione **.yaml** e il secondo **.pgm**.

Il file yaml possiede al suo interno delle informazioni e dei dati relativi alla mappa, quello generato dal per il progetto si presenta in questo modo :

```
image: mappaMondo2.pgm
resolution: 0.050000
origin: [-10.0, -10.0, 0.0]
negate: 0
occupied_thresh: 0.65
free_thresh: 0.196
```

Il file .pgm invece è un'immagine della mappa.

Su RViz si può scegliere quali feature e topic visualizzare, in modo tale da rendere più comoda e chiara la rappresentazione della navigazione, in particolare si possono anche andare ad aggiungere le proprietà relative ai propri topic, senza rischiare di aggiungere informazioni che non si hanno; questa immagine mostra come si presenta la schermata :

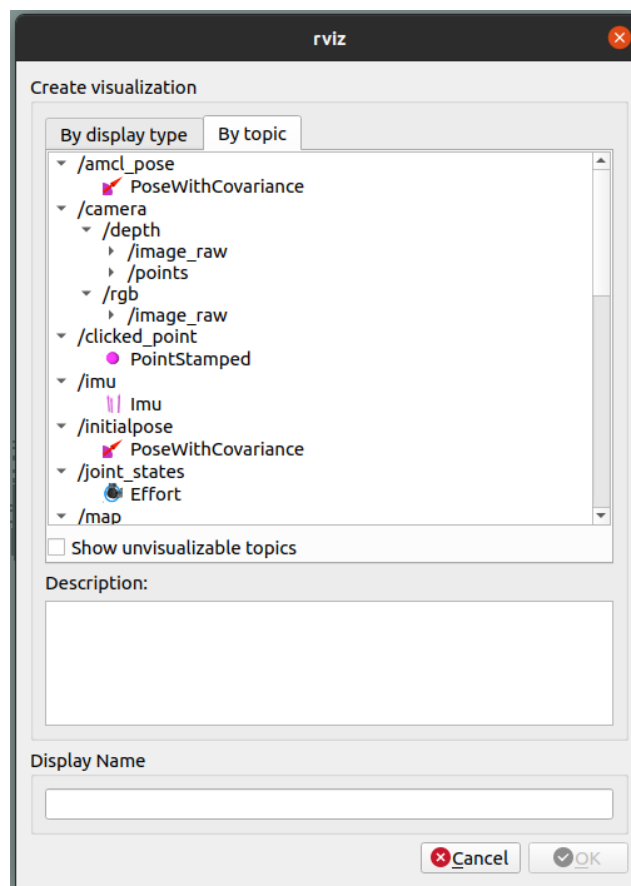


Figura 4.9: Schermata Rviz per aggiunta di feature

In questo progetto sono state rese visibili le seguenti caratteristiche :

- Global Options : sono le opzioni che riguardano la mappa e la sua rappresentazione, ad esempio il colore dello sfondo, le luci ecc...
- Global Status : comunica lo stato globale, indicando se ci sono problemi oppure risulta tutto correttamente funzionante.
- Grid : si può scegliere di rendere visibile la griglia sulla mappa.
- Map : sono le impostazioni relative alla mappa, in questo caso specifico le mappe sono tre, la prima è quella che ha il topic `/map`, poi si ha quella relativa al topic `/move_base/local_costmap/costmap` e l'ultima è `/move_base/global_costmap/costmap`.
- Robot Model : modello del robot.
- Path : rappresenta il percorso che viene creato, anche qui si hanno tre percorsi ovvero : `/move_base/DWAPlannerROS/global_plan`, `/move_base/DWAPlannerROS/local_plan`, `/move_base/NavfnROS/plan`.
- Odometry : informazioni sull'odometria
- Image : mostra quello che il robot vede dalla telecamera.
- LaserScan : informazioni sul laser.
- PoseArray

### 4.1.3 Documentazione necessaria e file di lancio

A questo punto arrivati si è in possesso di quasi tutti gli elementi per avanzare con la navigazione e poter iniziare le simulazioni, ciò che manca sono dei file che permettano di lanciare la mappa e il modellino.

In particolare sono stati creati due file di lancio (.launch) : **start\_localization.launch** e **start\_navigation.launch**.

## Start localization

Questo file serve per la localizzazione del robot, come prima accennato viene utilizzato un approccio che prende il nome di **AMCL**, che sarà proprio il nodo al centro della trattazione.

Nel file viene specificata la mappa di riferimento, in questo caso possiede il nome di mappaMondo2.yaml, vengono anche riportate le informazioni relative allo scan\_topic e alle coordinate in cui verrà fatto nascere il robot.

```
<launch>

  <!--include file="$(find turtlebot3_bringup)/launch/
    turtlebot3_remote.launch" /-->

  <arg name="map_file" default="$(find navigazione)/mappe/
    mappaMondo2.yaml"/>
  <!--<arg name="map_file" default="$(find t3_navigation)/maps/
    mappaFusa.yaml"/>-->
  <!--arg name="map_file" default="$(find t3_navigation)/maps/
    mappa_lab.yaml"/>-->
  <node name="map_server" pkg="map_server" type="map_server"
    args="$(arg map_file)" />

  <arg name="scan_topic" default="scan"/>
  <arg name="initial_pose_x" default="0.0"/>
  <arg name="initial_pose_y" default="0.0"/>
  <arg name="initial_pose_a" default="0.0"/>
```

E poi vi sono tutti i parametri del nodo amcl che sono il fulcro di questo file.

## Start navigation

All'interno di questo file invece, sono contenute tutte le informazioni relative alla navigazione del robot, questo implica la presenza di altri file minori che gestiscano i parametri del DWA, della local\_costmap, della global\_costmap e del Navfn; inoltre l'idea è quella di lanciare un solo file per tutto, quindi al suo interno viene resa implicita anche l'apertura del file prima descritto (start\_localization).

I documenti che definiscono i parametri sono :

- `costmap_common_params_waffle.yaml`
- `dwa_local_planner_params.yaml`
- `global_costmap_params.yaml`
- `local_costmap_params.yaml`
- `move_base_params.yaml`

### 4.1.4 Architettura stack di navigazione

Quando il file di lancio della navigazione viene lanciato, e di conseguenza vengono attivati anche gli altri sopra citati, si può osservare come si presenta lo stack di navigazione :

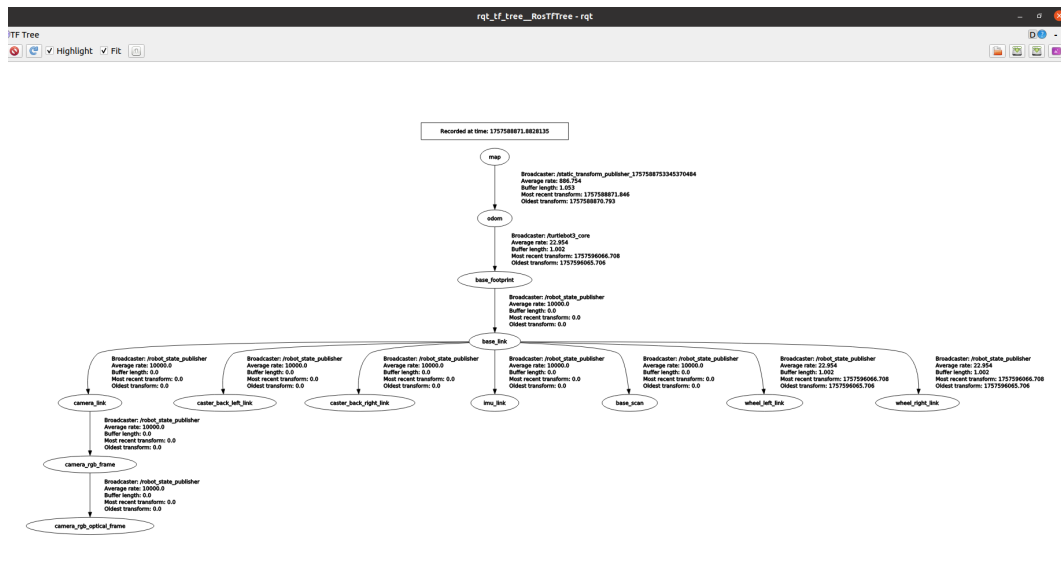


Figura 4.10: rqt\_graph

Tutto quello che è stato detto fino a questo momento in relazione allo stack, lo si può osservare nel grafico.

## 4.2 Traiettorie asservite e comportamento nei pressi degli ostacoli

È arrivato il momento di andare a testare il funzionamento del controllore su diversi scenari costruiti, per capire il comportamento dell'algoritmo DWA e la comunicazione che avviene all'interno dello stack.

Per la registrazione delle simulazioni viene utilizzato uno strumento di ROS che prende il nome di **rosbag**, con questo tool è possibile registrare una navigazione, indicando in che directory si vuole effettuare il salvataggio e soprattutto quali sono i topic che vengono monitorati durante tutta la registrazione, è possibile anche andare a riprodurre la simulazione registrata attraverso la funzione *play*.

Per una questione di uniformità e coerenza dei dati i topic che sono stati monitorati per questo progetto sono i seguenti :

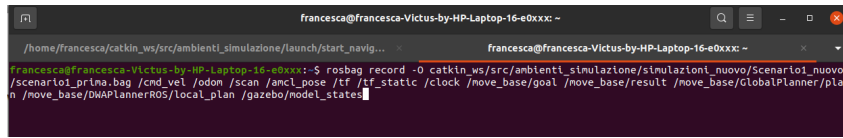


Figura 4.11: Topic utilizzati per le simulazioni

Alla fine delle varie prove, si avranno dei file **.bag**, per estrarre le informazioni necessarie alla creazione di grafici, al fine di un'analisi maggiormente approfondita, è stato utilizzato **Matlab**, in alternativa poteva essere usato anche python con varie librerie di supporto.

I **grafici** e le **metriche** sono il punto cruciale delle simulazioni, servono per raccogliere dati e procedere alla formulazione di diversi confronti, in particolare i grafici processati sono stati :

- Grafico della traiettoria;
- Velocità lineare;
- Velocità angolare;
- Distanza dall'obiettivo.

Mentre per quanto riguarda le metriche :

- Tempo impiegato;
- Distanza percorsa;
- Distanza minima dal goal.

Per ogni scenario, tranne per il primo, verrà fatta una "doppia simulazione", la prima sarà una simulazione dipendente da tutti gli scenari precedenti, la seconda invece avverrà resettando la mappa; questo approccio vuole evidenziare il funzionamento dello stack di navigazione e come avviene la gestione della mappa, infatti il DWA è un local planner che usa tutti i criteri visti in precedenza per trovare la strada migliore da percorrere, non avviene nessuna memorizzazione o "learning" tra una navigazione e un'altra, tuttavia se si osserva la prima simulazione dipendente dalle precedenti, si può notare che il

robot non fa più gli stessi "errori" che faceva in precedenza, proprio per la gestione dello stack, il DWA non contrinuisce in questo, e lo possiamo notare resettando semplicemente la mappa e osservando la seconda della coppia di simulazioni.

### 4.2.1 Primo Scenario

Il primo scenario creato è quello più semplice da un punto di vista della disposizione degli ostacoli, difatti è esattamente quello creato fin dall'inizio, non presenta ulteriori impedimenti.

#### Mappe Gazebo e RViz

Si visualizzi la mappa che viene utilizzata, sia all'interno di Gazebo che di RViz:

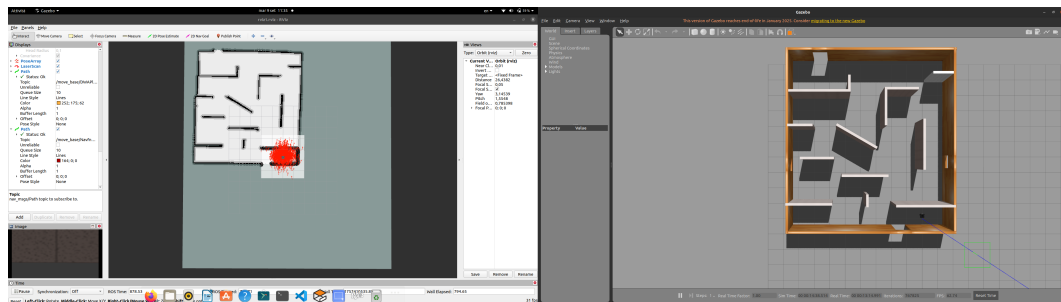


Figura 4.12: Primo scenario di simulazione

Per poterli visionare più nel dettaglio :

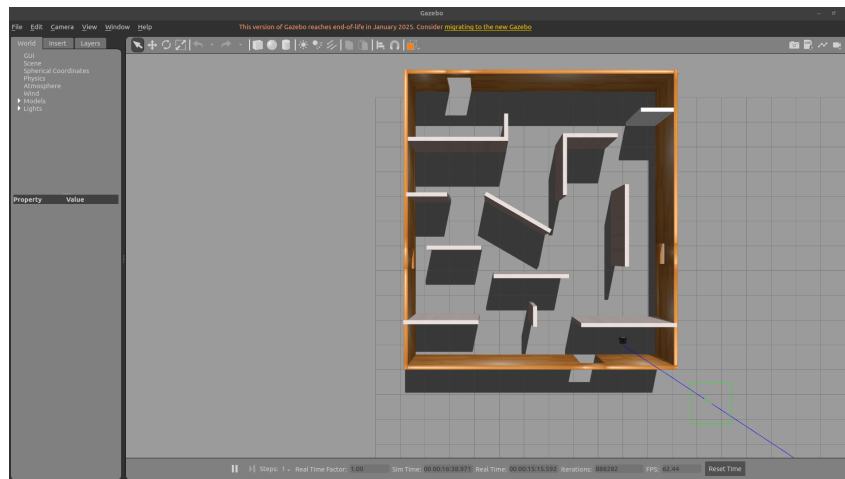


Figura 4.13: Primo scenario di simulazione in Gazebo

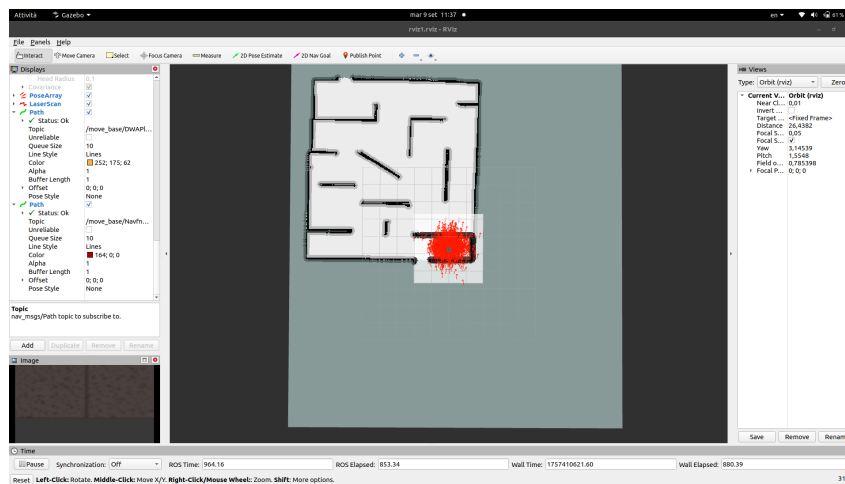


Figura 4.14: Primo scenario di simulazione in RViz

Il punto di partenza è in corrispondenza dello spawn del robot (nell'immagine 4.15 è cerchiato in verde), mentre il punto di arrivo (la posa è raffigurata da una freccia rossa), e quindi l'obiettivo è la porta che si trova sul lato più distante dal robot.

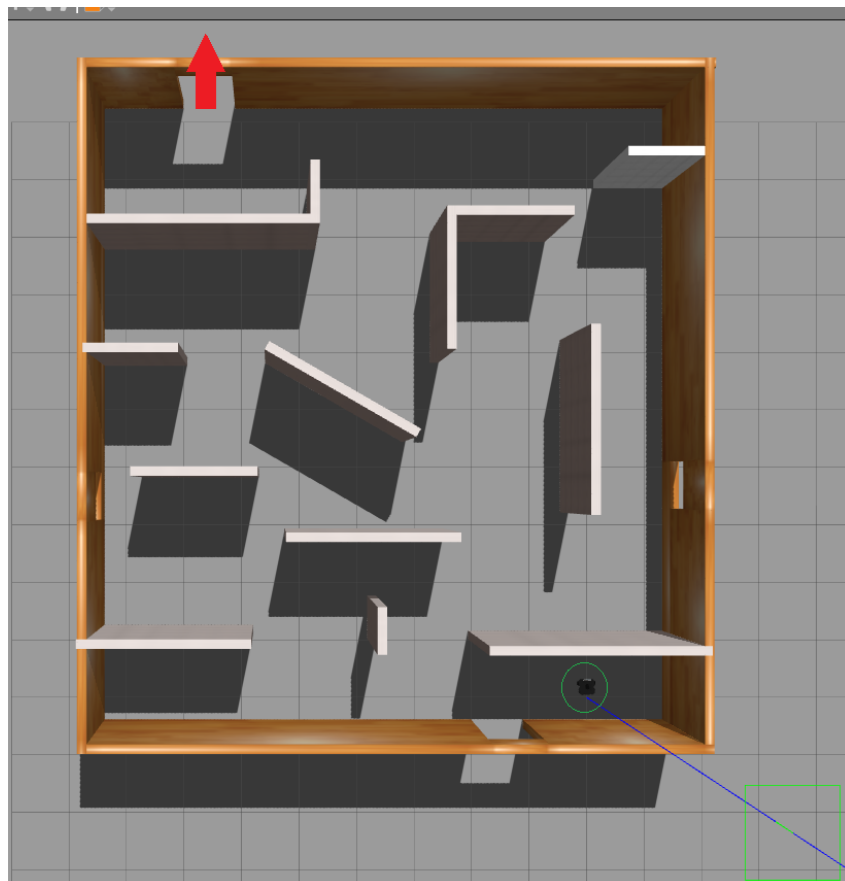


Figura 4.15: Partenza e obiettivo del primo scenario

Quando inizia la simulazione quello che è possibile visualizzare su RViz è il percorso globale che viene inizialmente suggerito e mentre il robot si muove vengono creati i vari percorsi secondo il local planner.



Figura 4.16: Percorso iniziale

Durante la navigazione, il robot comunica le informazioni, e all'interno del terminale è possibile visualizzare i messaggi che vengono inviati, come la ricezione del goal, o la creazione di nuovi percorsi, inoltre invia anche dei *warning* qualora qualcosa dovesse andare male e ci fosse bisogno di agire in qualche modo; per esempio nel caso riportato nell'immagine 3.2.2 c'è il bisogno di fare una ***rotate\_recovery*** (descritta nella tabella della sottosezione 3.2.2), questo è quello che appare :

```
/home/francesca/catkin_ws/src/ambienti_simulazione/launch/start_navigation.launch http://loc...
/home/francesca/catkin_ws/src/ambienti_simulazione/L... francesca@francesca-Victus-by-HP-Laptop-16-e0xxx: ~
[INFO] [1757410974.309928235, 1316.448000000]: Got new plan
[INFO] [1757410974.642905955, 1316.781000000]: Got new plan
[INFO] [1757410975.310401308, 1317.448000000]: Got new plan
[INFO] [1757410975.644208533, 1317.781000000]: Got new plan
[INFO] [1757410976.311765174, 1318.448000000]: Got new plan
[INFO] [1757410976.645515570, 1318.781000000]: Got new plan
[INFO] [1757410977.313651225, 1319.448000000]: Got new plan
[INFO] [1757410977.647107796, 1319.781000000]: Got new plan
[INFO] [1757410978.316172599, 1320.448000000]: Got new plan
[INFO] [1757410978.650300071, 1320.781000000]: Got new plan
[INFO] [1757410979.317456501, 1321.448000000]: Got new plan
[INFO] [1757410979.651635793, 1321.781000000]: Got new plan
[INFO] [1757410980.318441008, 1322.448000000]: Got new plan
[INFO] [1757410980.652181866, 1322.781000000]: Got new plan
[INFO] [1757410981.319756000, 1323.448000000]: Got new plan
[INFO] [1757410981.654002841, 1323.781000000]: Got new plan
[INFO] [1757410982.320831060, 1324.448000000]: Got new plan
[INFO] [1757410982.655104042, 1324.781000000]: Got new plan
[INFO] [1757410983.323586063, 1325.449000000]: Got new plan
[INFO] [1757410983.656318181, 1325.781000000]: Got new plan
[INFO] [1757410984.324247147, 1326.448000000]: Got new plan
[INFO] [1757410984.657620378, 1326.781000000]: Got new plan
[WARN] [1757410984.990923824, 1327.114000000]: Rotate recovery behavior started.
[INFO] [1757410991.800075782, 1333.915000000]: Got new plan
[INFO] [1757410992.134107059, 1334.248000000]: Got new plan
[INFO] [1757410992.467344468, 1334.581000000]: Got new plan
[INFO] [1757410993.135457750, 1335.248000000]: Got new plan
[INFO] [1757410993.468797035, 1335.581000000]: Got new plan
[INFO] [1757410993.803018889, 1335.914000000]: Goal reached
```

Figura 4.17: Rotate recovery

Si noti dal messaggio informativo finale, che in questo caso l'azione di recupero ha portato al raggiungimento dell'obiettivo.

## Metriche

A questo punto arrivati è doveroso capire come si è svolta la navigazione del robot in termini di costi temporali e di distanze.

Si noti che per non contaminare i dati e rendere tutto più uniforme possibile, il tempo totale viene fatto partire da quando il robot inizia a muoversi, questo perché da quando viene lanciata la simulazione fino a quando viene dato l'obiettivo possono passare dei secondi che variano da simulazione a simulazione, stessa cosa vale con la chiusura.

Queste sono le metriche :

```
Distanza percorsa: 15.01 m
Tempo totale: 89.10 s
Distanza minima dal goal: 0.28 m
-----
```

## Grafici simulazioni

Il primo grafico è relativo alla *traiettoria*, prende le informazioni dalla posa del robot in ogni momento e le riporta in un grafico bidimensionale con assi X e Y :

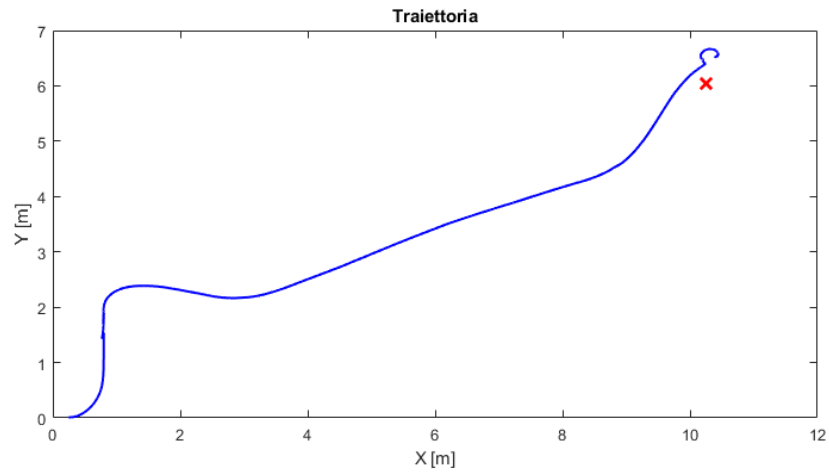


Figura 4.18: Grafico della traiettoria

*Grafico della velocità lineare :*

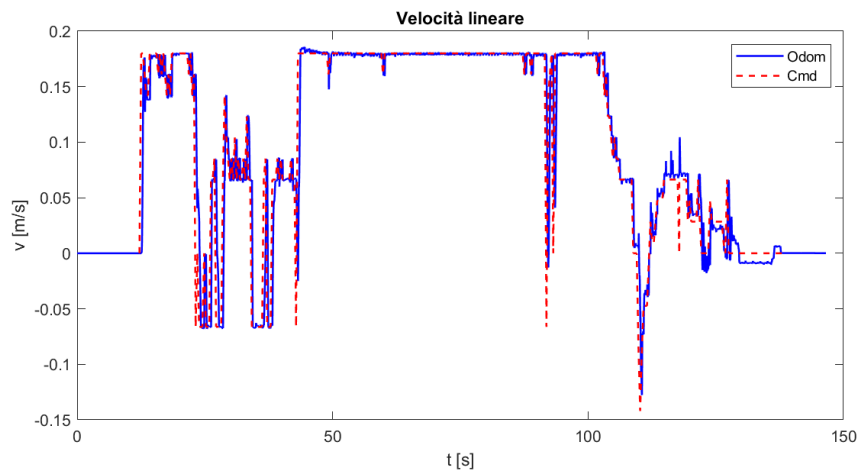


Figura 4.19: Grafico delle velocità lineari

*Grafico della velocità angolare :*

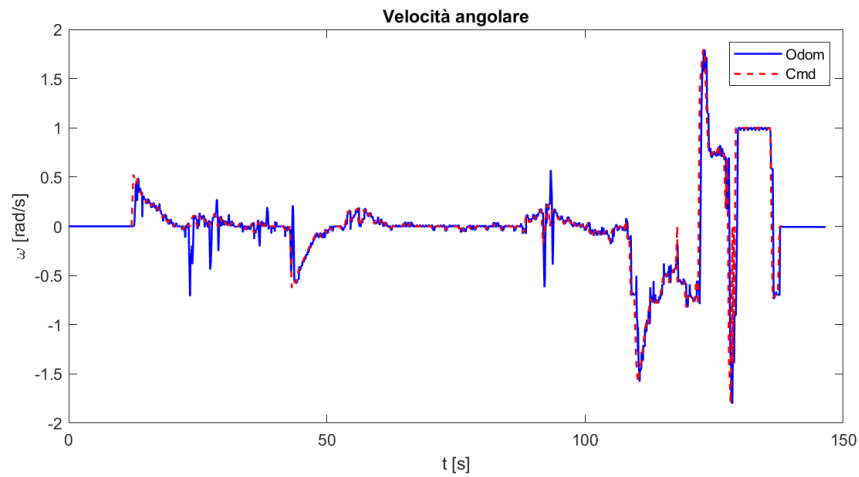


Figura 4.20: Grafico delle velocità angolari

*Grafico della distanza dall'obiettivo :*

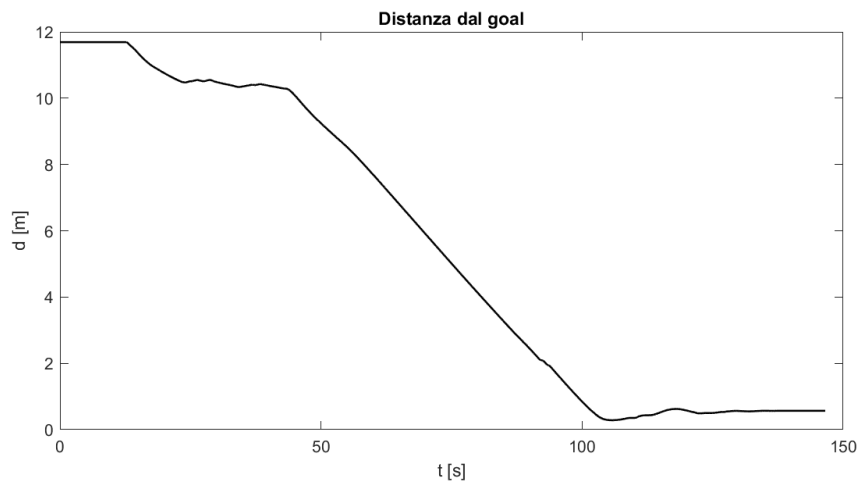


Figura 4.21: Grafico della distanza dall'obiettivo

La distanza dopo alcuni istanti di tempo si stabilizza ad un valore, questo avviene perché all'interno dei vari file .yaml presentati in precedenza, è riportata una voce che è *goal\_tolerance*, la quale indica la distanza massima che il robot può avere rispetto al goal.

## 4.2.2 Secondo Scenario

Il secondo scenario, vede l'aggiunta di un ostacolo di forma cilindrica, con coordinate :

- $x = 4,313332$

- $y = 2,215178$
- $z = 0,499997$
- $\text{roll} = 0,000004$
- $\text{pitch} = 0,000004$
- $\text{yaw}^3 = 0,00$

L'ostacolo è stato posizionato in modo da trovarsi in corrispondenza della traiettoria tracciata inizialmente per il raggiungimento dell'obiettivo, così da valutare come avviene l'evitamento.

La partenza e l'arrivo sono invece uguali a quelli del precedente scenario.

## Mappe Gazebo e RViz

Con il nuovo ostacolo la mappa si presenta ne seguente modo:

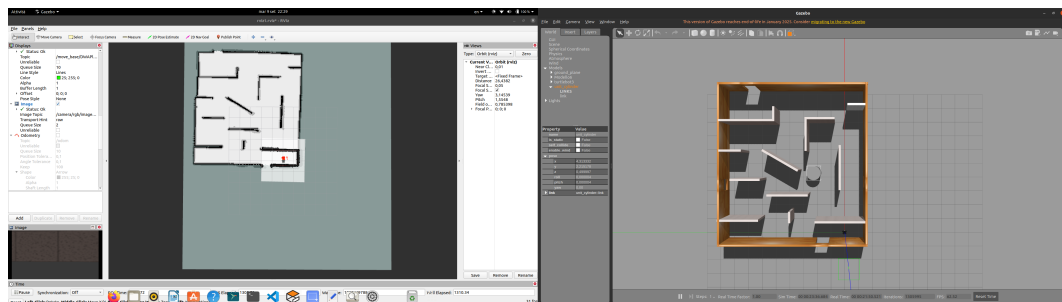


Figura 4.22: Secondo scenario

Si distingue la visualizzazione in Gazebo e in RViz:

---

<sup>3</sup>Roll, pitch e yaw sono i tre assi di rotazione nello spazio in 3 dimensioni, in particolare il primo è la corrispondenza inglese del termine rollio e indica la rotazione laterale, pitch corrisponde al beccheggio ed è una rotazione avanti/indietro, mentre imbardata (yaw) è la rotazione lungo l'asse verticale.

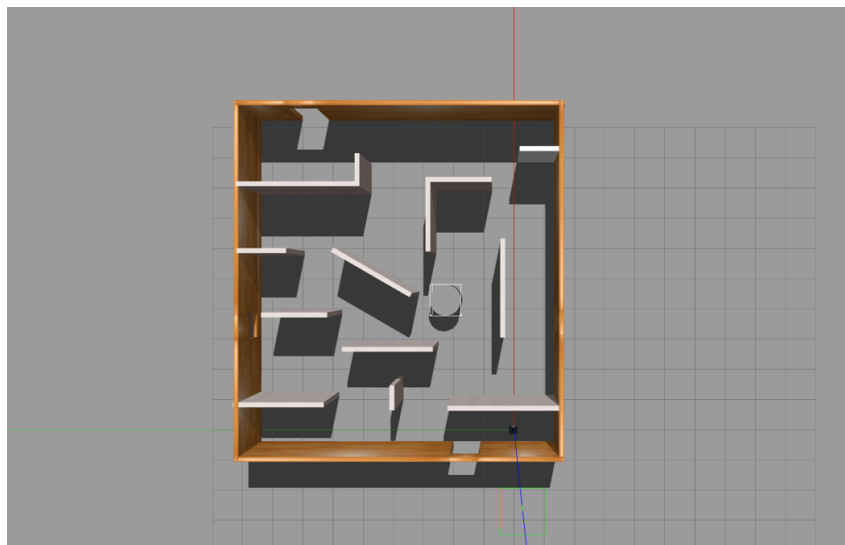


Figura 4.23: Secondo scenario in gazebo



Figura 4.24: Secondo scenario in RVizS

## Metriche

Come specificato nell'introduzione, sono state fatte due simulazioni, la prima che mantiene il riferimento della simulazione precedente, mentre la seconda si ottiene resettando la mappa.

Le differenze si notano fin dall'analisi delle metriche, difatti la prima avendo nello stack di navigazione già memorizzato la mappa risulterà più rapida nelle

situazioni conosciute ed esplorate già nel precedente scenario, mentre per la seconda la parte iniziale sarà leggermente più lenta.

#### **Prima simulazione :**

```
Distanza percorsa: 15.37 m
Tempo totale: 104.62 s
Distanza minima dal goal: 0.32 m
-----
```

#### **Seconda simulazione :**

```
Distanza percorsa: 14.89 m
Tempo totale: 108.12 s
Distanza minima dal goal: 0.26 m
-----
```

#### **Grafici simulazioni**

Quello che è stato possibile notare nelle metriche però è solo la parte più "superficiale", molti concetti possono essere compresi meglio guardando i grafici, e in particolare quelli delle velocità, per non renderne la visione troppo pesante e dispersiva verrà riportata l'immagine contenente i quattro grafici per ogni simulazione, e per eventuali considerazioni verranno presi singolarmente :

#### **Prima simulazione :**

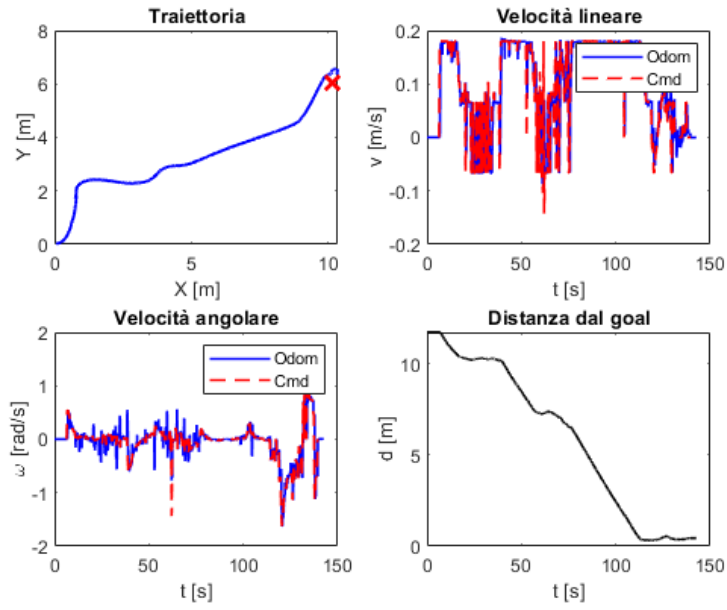


Figura 4.25: Grafici secondo scenario prima simulazione

Seconda simulazione :

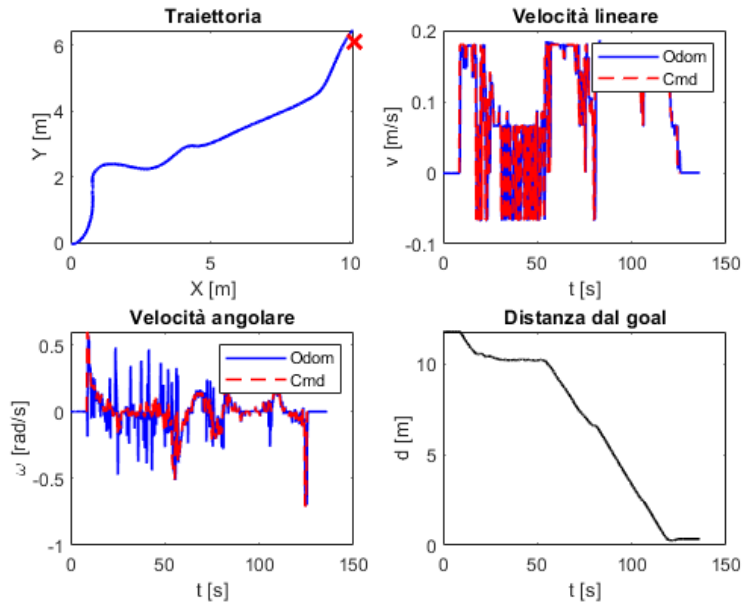


Figura 4.26: Grafici secondo scenario seconda simulazione

## Confronti

Se si sovrappongono le traiettorie si può notare che in realtà nonostante la differenza di tempo, le due traiettorie sono quasi sovrapposte, a parte il pri-

mo tratto in cui la prima sembra essere più immediata, ciò fa comprendere l'efficienza dell'algoritmo DWA:

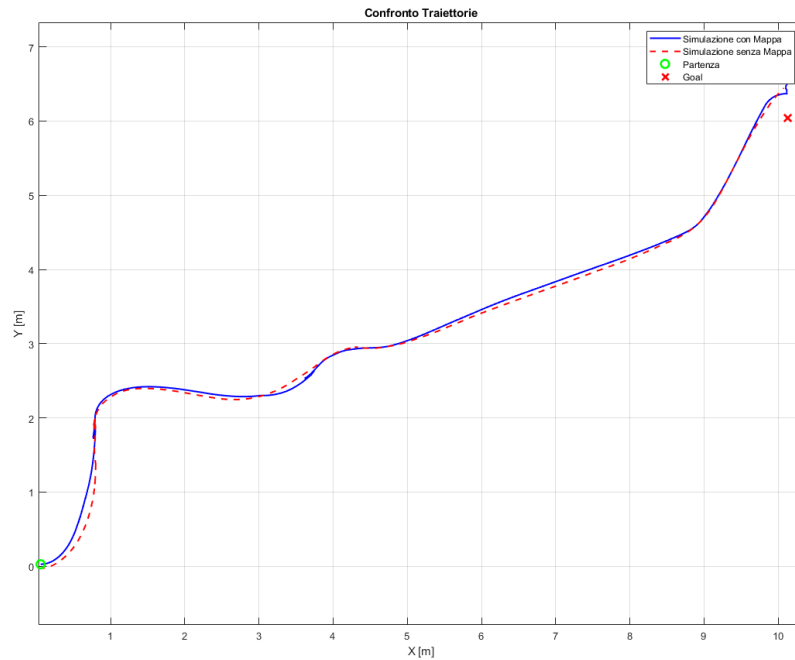


Figura 4.27: Grafici delle simulazioni a confronto

Per quanto riguarda le velocità invece, nello specifico quelle angolari, nella prima simulazione dove il robot ha già informazioni precedenti, sono meno presenti dei picchi (si trovano solo verso la parte finale), mentre nella seconda sono frequenti dei picchi anche nella prima parte.

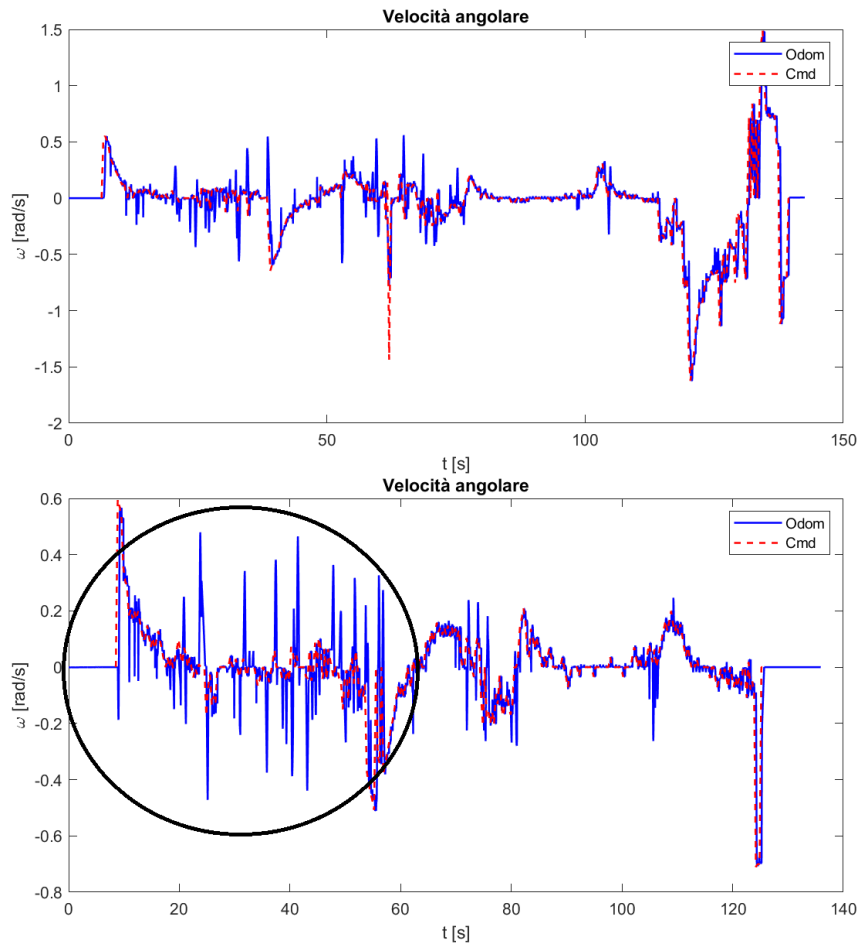


Figura 4.28: Confronto velocità angolari

### 4.2.3 Terzo Scenario

Il terzo scenario vede l'aggiunta di un altro ostacolo, oltre a quello già presente nello scenario precedente, questa volta cubico, posto più avanti rispetto al primo.

#### Mappe Gazebo e RViz

La mappa con il nuovo ostacolo risulterà nel seguente modo :

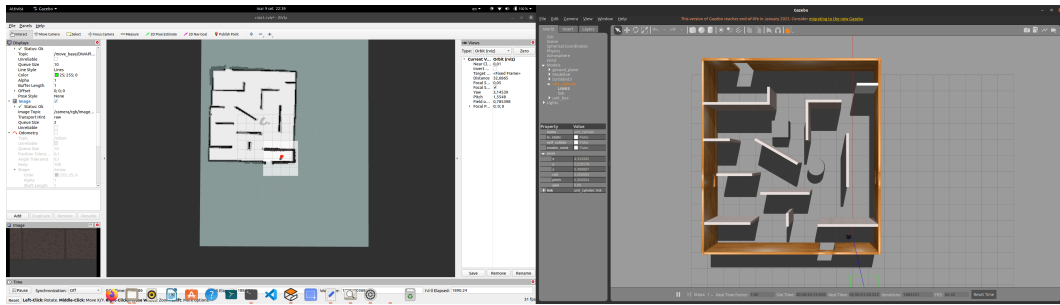


Figura 4.29: Terzo scenario

Visione dettagliata di Gazebo e RViz :

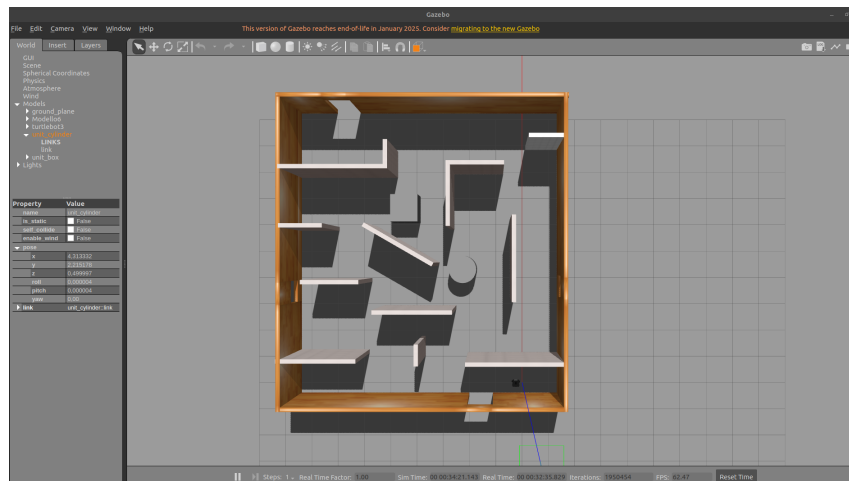


Figura 4.30: Terzo scenario Gazebo

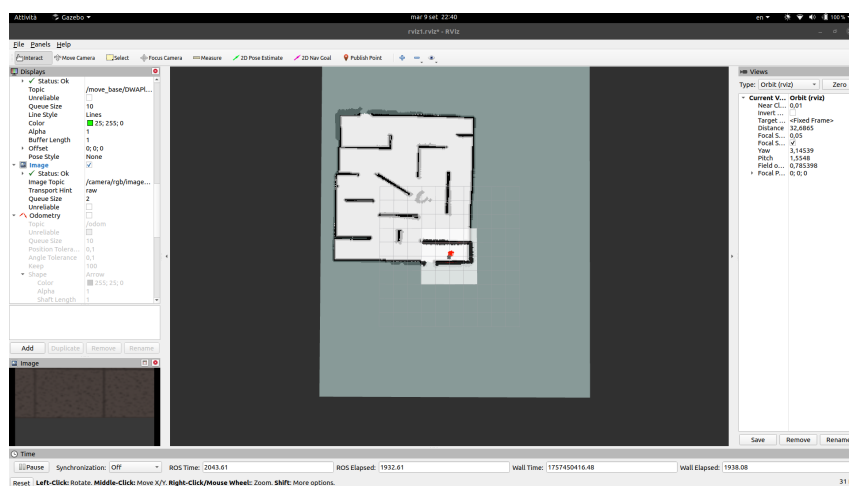


Figura 4.31: Terzo scenario RViz

Anche qui obiettivo e punto di partenza sono gli stessi.

## Metriche

### Prima simulazione :

```
Distanza percorsa: 16.21 m
Tempo totale: 133.41 s
Distanza minima dal goal: 0.26 m
-----
```

### Seconda simulazione :

```
Distanza percorsa: 14.87 m
Tempo totale: 106.21 s
Distanza minima dal goal: 0.39 m
-----
```

Osservando questi dati si potrebbe pensare ad un risultato contro-intuitivo, proprio per quello che è stato detto in precedenza, in realtà si può fare un'osservazione ancora più profonda, nella simulazione senza mappa il robot ha impiegato meno tempo perché ha trovato un percorso migliore, e ciò lo si può vedere anche dalla distanza anch'essa risultante minore; questo miglioramento potrebbe essere dovuto al fatto di non essere "vincolato" dalle informazioni relative alle precedenti simulazioni, dunque un'esplorazione in questo caso è risultata più efficiente.

## Grafici simulazioni

### Prima simulazione :

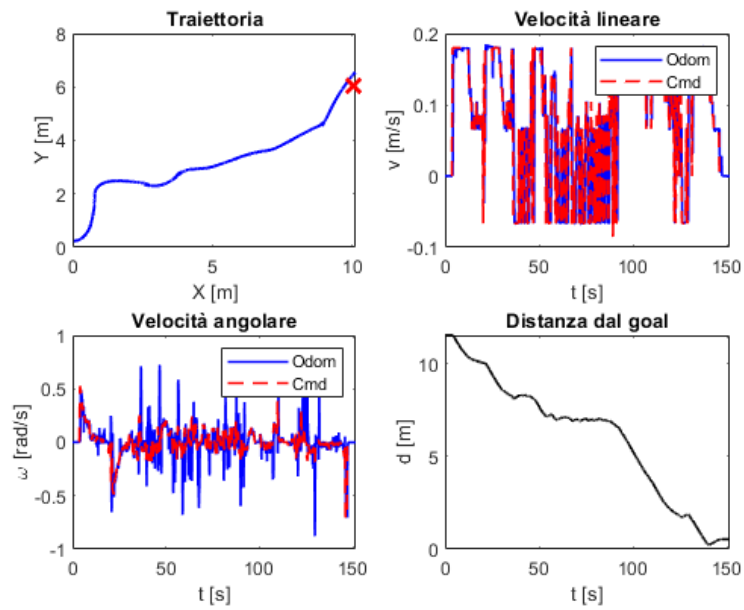


Figura 4.32: Grafici terzo scenario prima simulazione

Seconda simulazione :

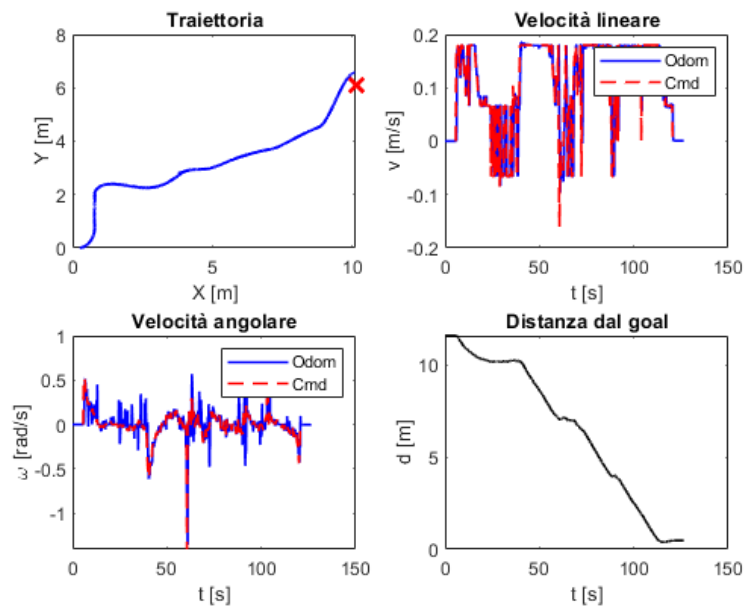


Figura 4.33: Grafici terzo scenario seconda simulazione

## Confronti

Anche qui confrontare la traiettoria potrebbe essere molto utile, soprattutto per far notare come il secondo percorso risulti essere migliore.

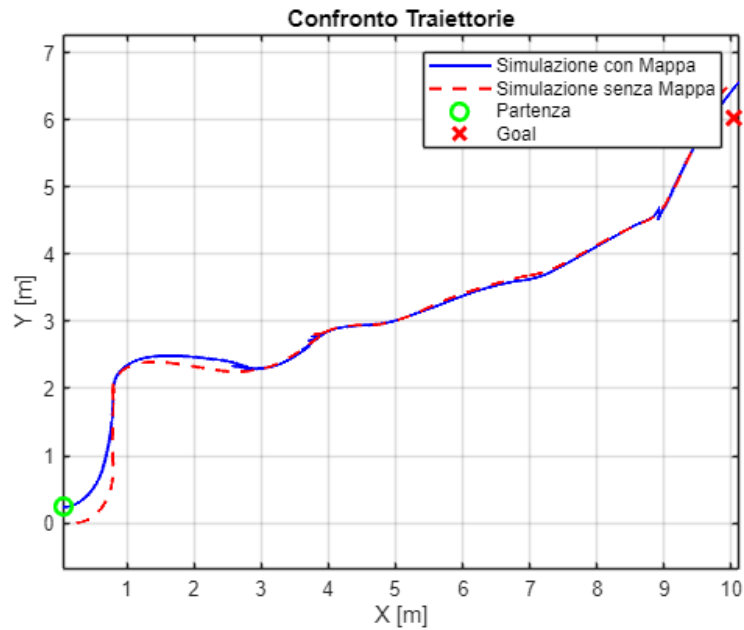


Figura 4.34: Grafici delle simulazioni a confronto

Per quanto riguarda le velocità lineari e angolari, il robot che non ha la mappa dalle precedenti simulazioni, e che quindi va in esplorazione, compie più movimenti.

#### 4.2.4 Quarto Scenario

Viene aggiunto un ostacolo di tipo cubico con le seguenti coordinate :

- $x = 9,015550$
- $y = 3,442060$
- $z = 0,500000$
- $\text{roll} = 0,00$
- $\text{pitch} = 0,00$
- $\text{yaw} = 0,0$

## Mappe Gazebo e RViz

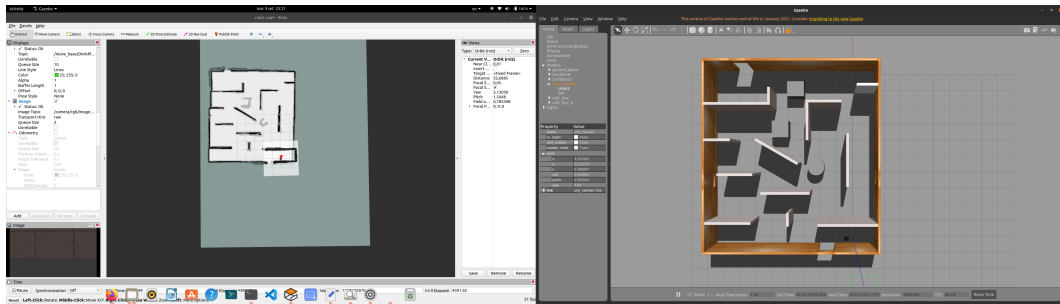


Figura 4.35: Quarto scenario

Nel dettaglio :

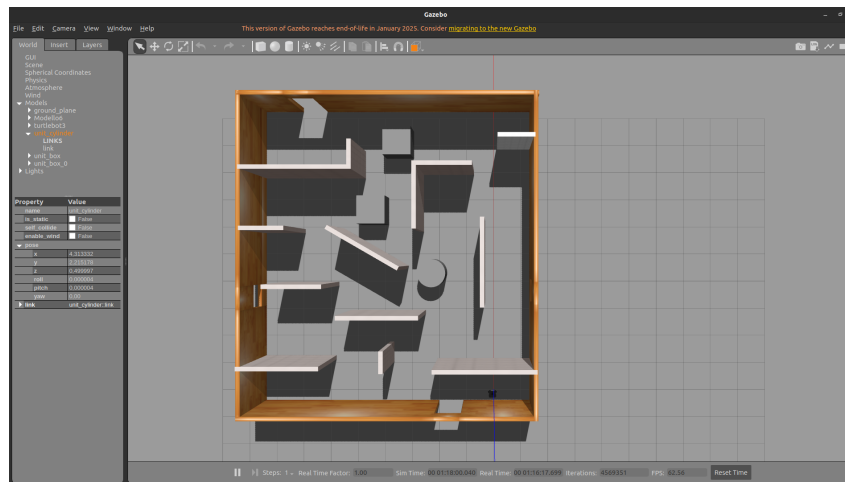


Figura 4.36: Quarto scenario Gazebo

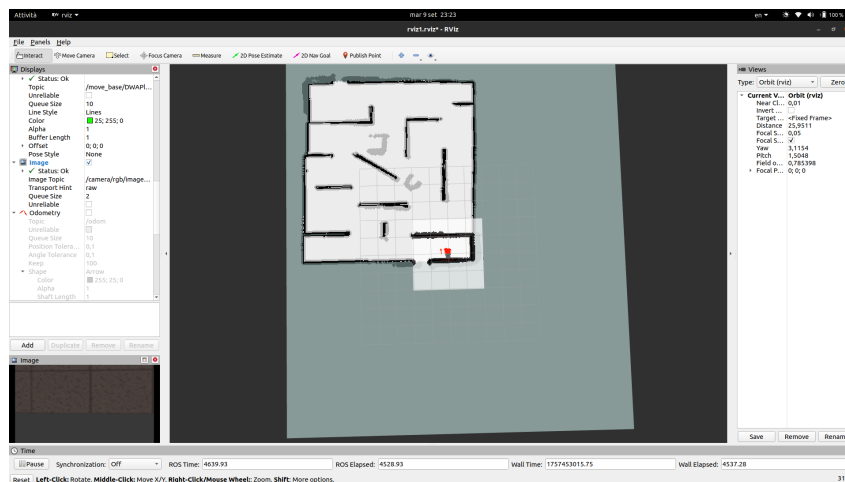


Figura 4.37: Quarto scenario RViz

## Metriche

### Prima simulazione :

Distanza percorsa: 14.50 m  
Tempo totale: 98.84 s  
Distanza minima dal goal: 0.44 m  
-----

### Seconda simulazione :

Distanza percorsa: 14.77 m  
Tempo totale: 103.13 s  
Distanza minima dal goal: 0.28 m  
-----

## Grafici simulazioni

### Prima simulazione :

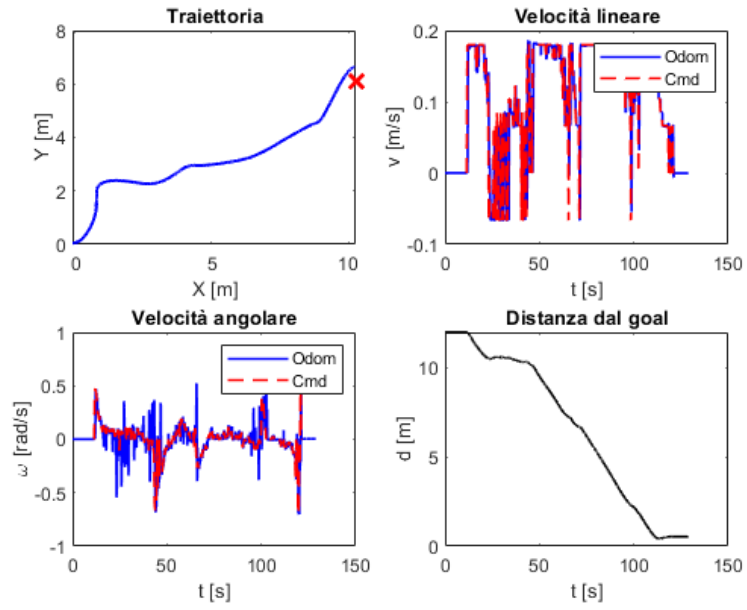


Figura 4.38: Grafici quarto scenario prima simulazione

Seconda simulazione :

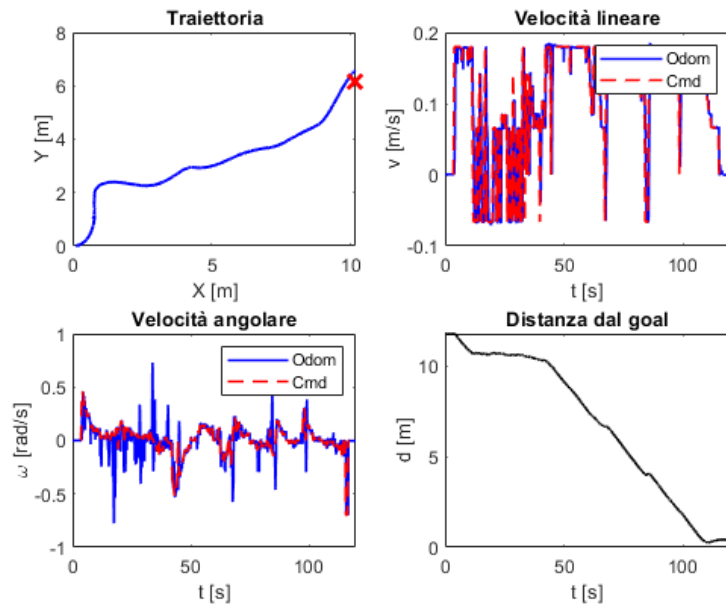


Figura 4.39: Grafici quarto scenario seconda simulazione

## 4.2.5 Quinto Scenario

Nell'ultimo scenario si è tentato di mettere in difficoltà maggiormente l'algoritmo, andando a posizionare l'ostacolo in un punto più fastidioso.

## Mappe Gazebo e RViz

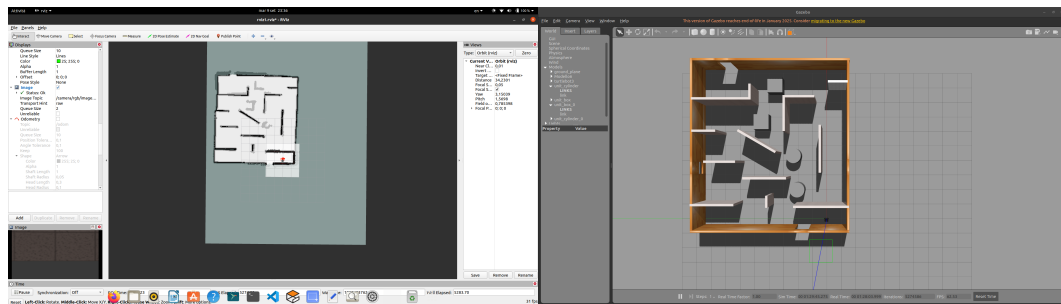


Figura 4.40: Quinto scenario

### Considerazioni

In questa simulazione il robot non riesce ad andare avanti, si blocca al primo ostacolo che trova, il quale è l'ultimo ad essere stato inserito.

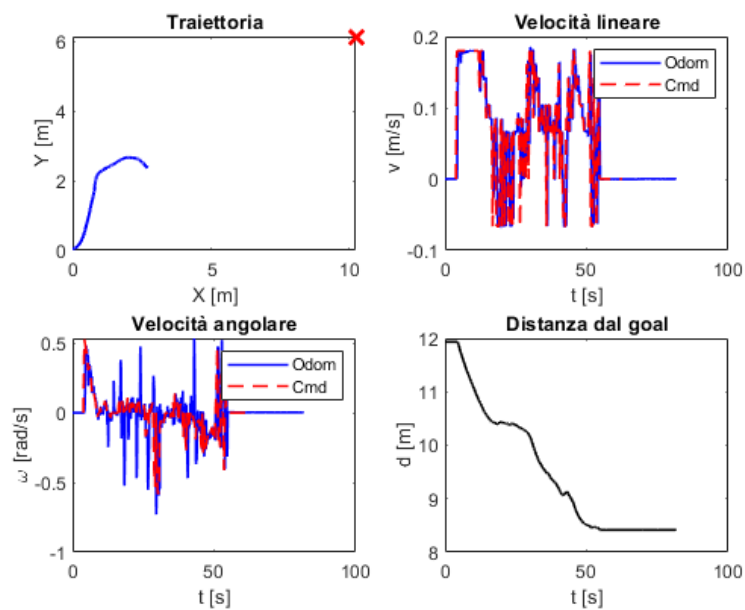


Figura 4.41: Grafici quinto scenario

Dal breve periodo di movimento si può notare come il dispositivo ruota e tenta di aggiustarsi in continuazione per capire come uscire dalla situazione in cui risulta "incastrato", dopo un certo lasso di tempo però, dopo aver provato tutte le opzioni a disposizione, il robot si arresta, mandando su terminale un messaggio di aborto dell'operazione.

### 4.2.6 Confronto e considerazioni

Nell'immagine si può visualizzare il confronto delle traiettorie nelle simulazioni che mantengono la mappa precedente :

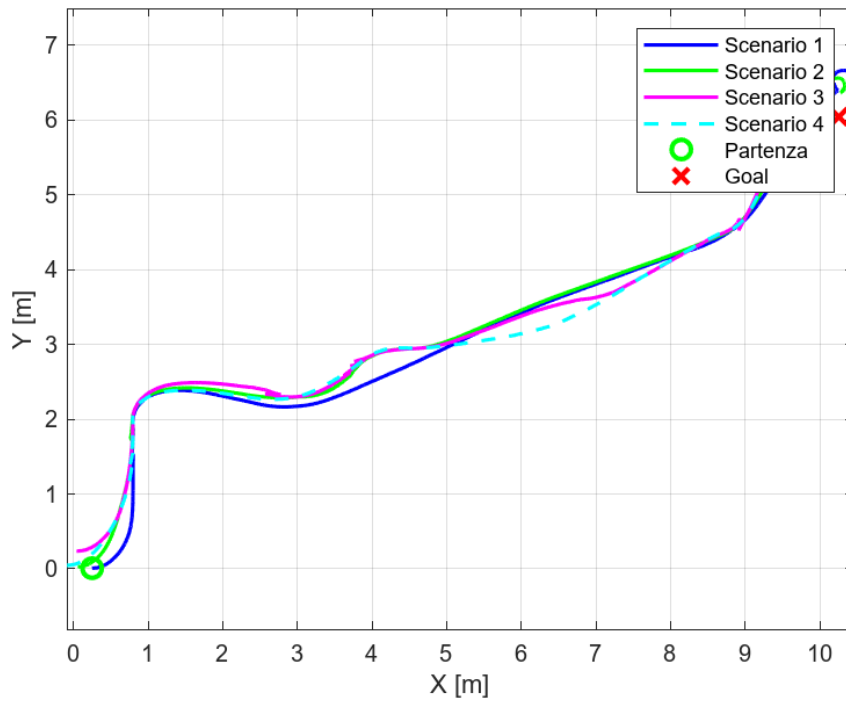


Figura 4.42: Grafici traiettoria a confronto

Confronto delle traiettorie, nelle simulazioni che non mantengono la mappa precedente :

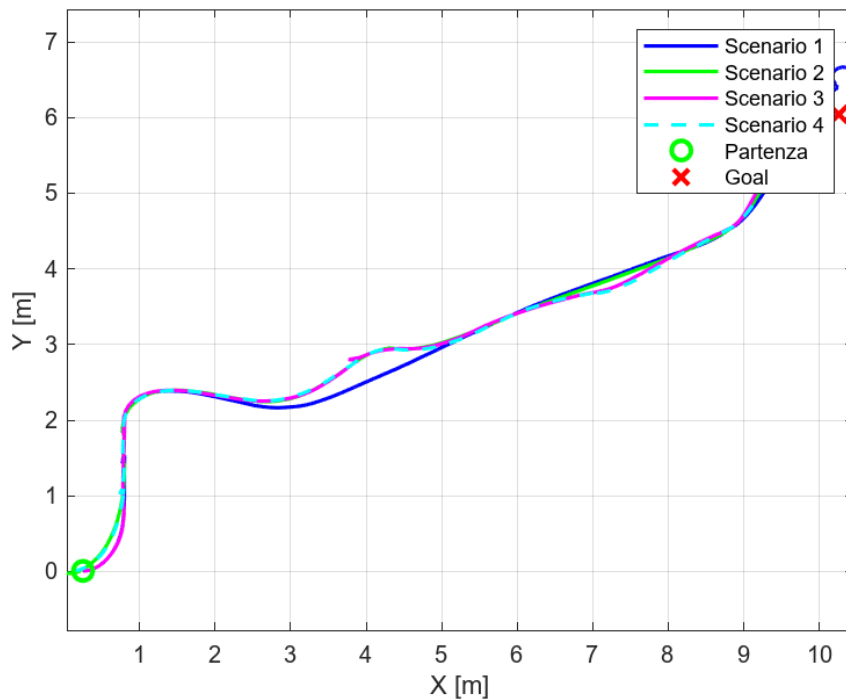


Figura 4.43: Grafici traiettoria a confronto

## 4.3 Esperimenti in laboratorio

In seguito alla riuscita delle prove simulative, il secondo step è stato quello di attuare le prove anche all'interno del laboratorio universitario, utilizzando il turtlebot waffle messo a disposizione e una serie di strumenti per la creazione dei percorsi.

### 4.3.1 Connessione del robot

Analogamente a quanto avviene nelle simulazioni, la fase iniziale consiste nella configurazione dell'ambiente di prova in cui verrà valutata la risposta del robot; in questo caso è necessario connettere il robot e il computer allo stesso server in modo che possano comunicare senza problemi, sarà dunque un'operazione che coinvolgerà gli indirizzi ip.

```
$ ssh pi@{IP_ADDRESS_OF_RASPBERRY_PI}
$ roslaunch turtlebot3_bringup turtlebot3_robot.launch
```

Se non è già presente all'interno del file di lancio va specificato anche il robot utilizzato, ma con questo progetto il problema non persiste perché è tutto dichiarato dentro il file, che insieme agli altri è analogo a quello già visto per la parte simulativa.

Una volta collegato tutto, è il momento di disporre le varie barriere; anche in questo caso inizialmente si è voluto creare un contesto non troppo complesso, in modo tale da aggiungere gradi di difficoltà andando avanti con le simulazioni. L'esplorazione viene fatta anche qui con SLAM e vengono salvate le mappe nella cartella apposita, così da poter essere poi riutilizzate.

### 4.3.2 Scenari creati

#### Primo scenario

La mappa di partenza risulta in questo modo :

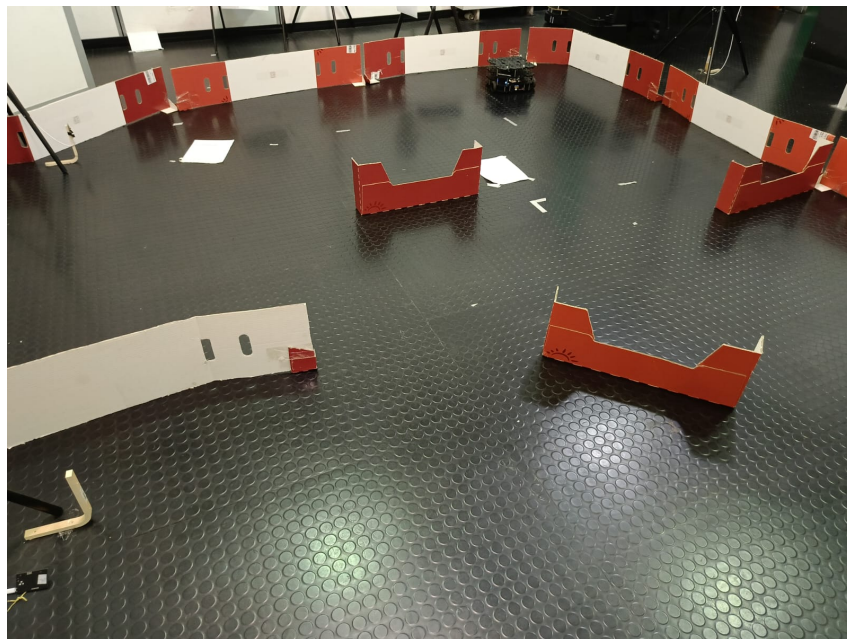


Figura 4.44: Allestimento del primo scenario sperimentale in laboratorio

Si noti che l'immagine è stata scattata dal punto in cui il robot dovrà arrivare, quindi dall'obiettivo e lo si può capire dalla mappa generata

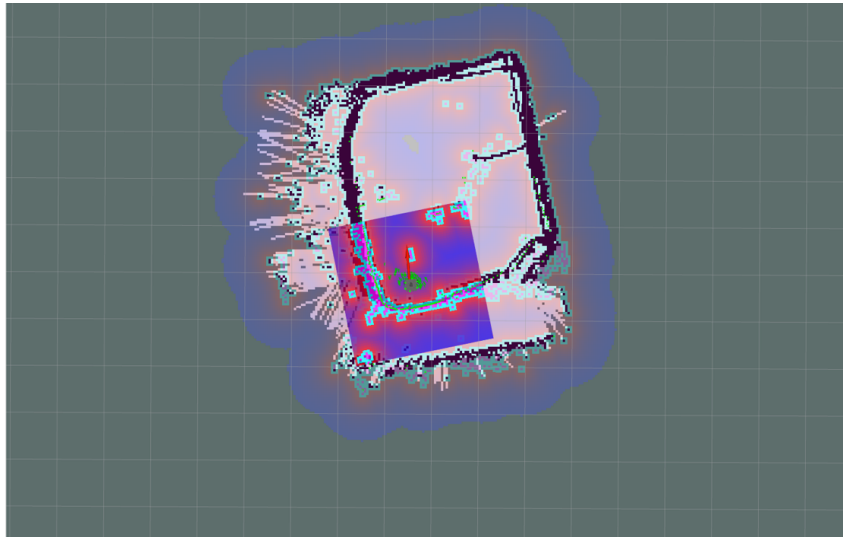


Figura 4.45: Allestimento del primo scenario sperimentale in laboratorio, RViz

Come prima basta far partire la registrazione della simulazione con i topic richiesti e infine attraverso gli script già creati analizzare i risultati.

```
Distanza percorsa: 5.81 m  
Tempo totale: 14.14 s  
Distanza minima dal goal: 0.29 m  
-----
```

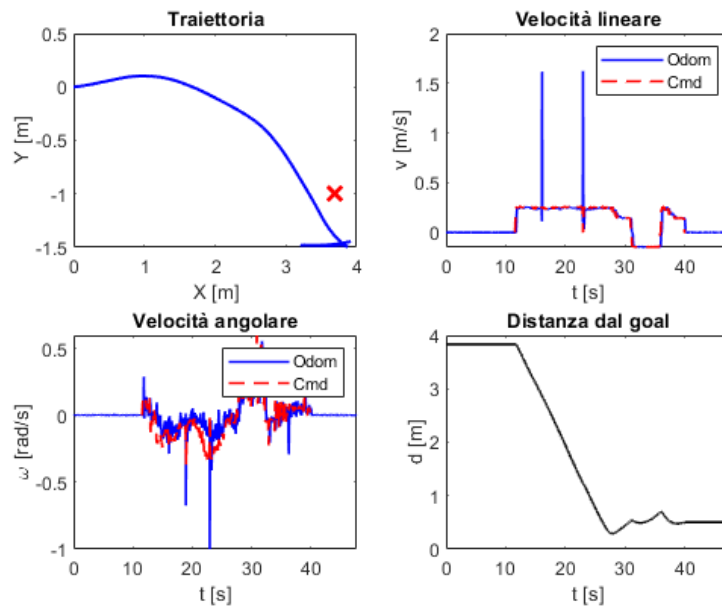


Figura 4.46: Primo scenario laboratorio grafici

La prima simulazione risulta essere per il robot non troppo impegnativa e lo si può capire dalle metriche, ma anche e soprattutto dai grafici che mantengono un profilo lineare per tutta la percorrenza, senza bruschi cambi di direzione o frenate.

## Secondo scenario

Passando al secondo scenario, è stato aggiunto un solo ostacolo, nella zona finale della mappa, prima del raggiungimento dell'obiettivo :

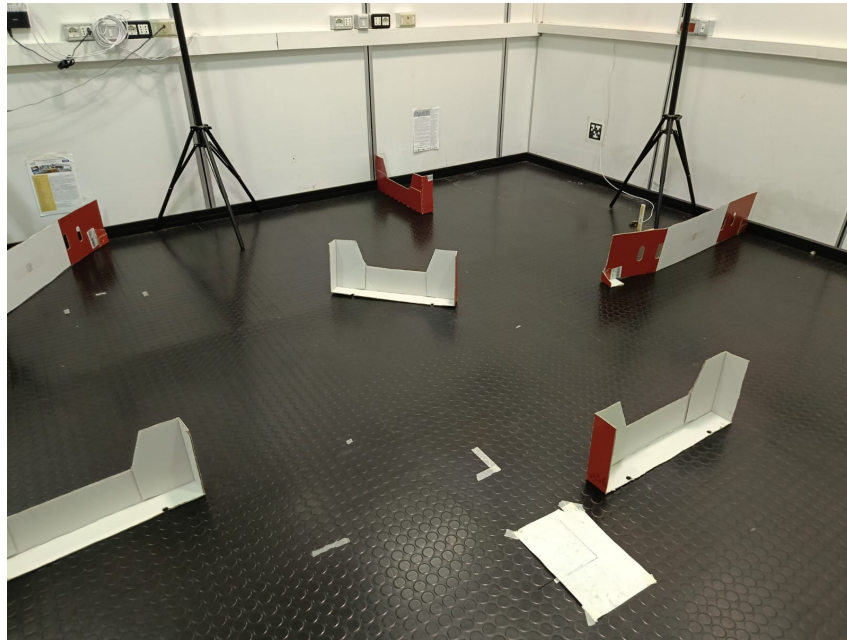


Figura 4.47: Allestimento del secondo scenario sperimentale in laboratorio

Anche qui il comportamento del robot non risulta troppo disturbato dall'impedimento appena inserito, ovviamente dai grafici si nota un movimento leggermente alterato rispetto al primo, dovuto proprio all'aggiunta dell'ostacolo, nonostante ciò percorre meno metri, probabilmente per via della memorizzazione della mappa da parte dello stack di navigazione :

Distanza percorsa: 4.62 m

Tempo totale: 24.67 s

Distanza minima dal goal: 0.66 m

-----

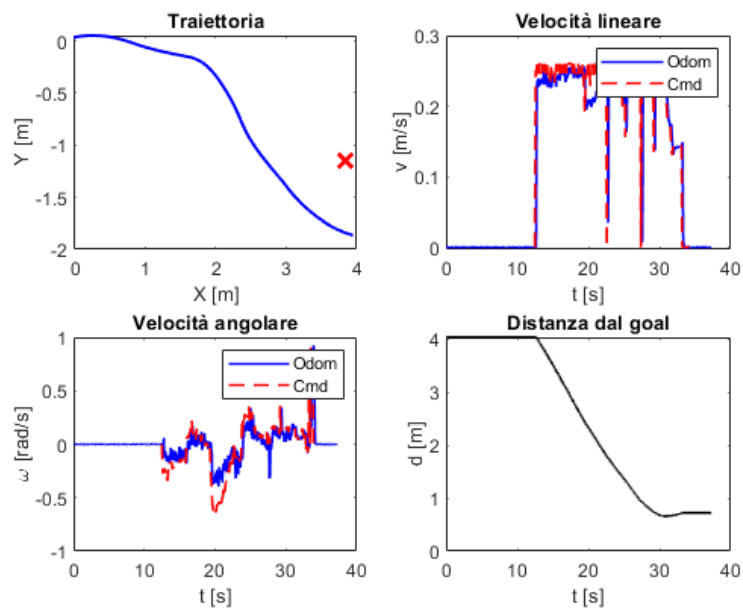


Figura 4.48: Grafici secondo scenario

E lo si può osservare anche con un confronto diretto tra le traiettorie dei due scenari :

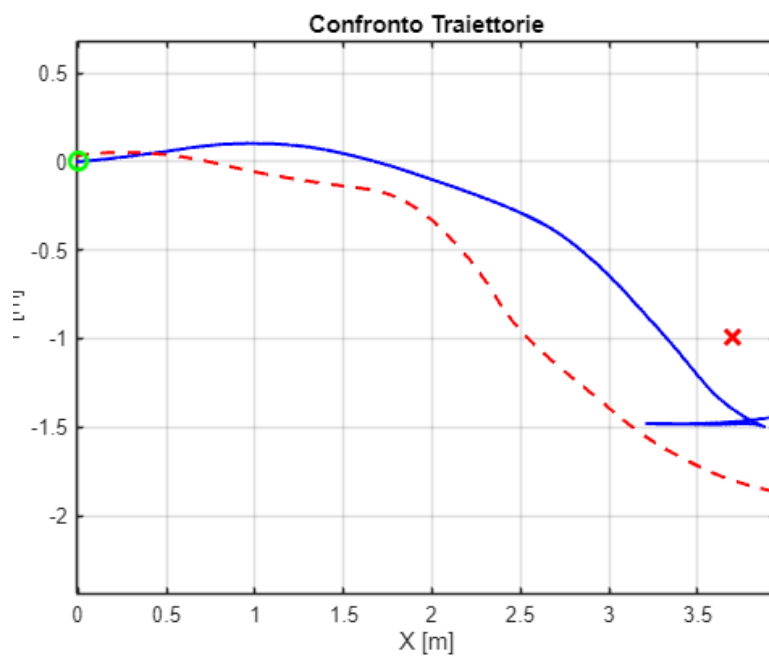


Figura 4.49: Confronto traiettorie

### Terzo scenario

Nel terzo scenario la situazione comincia a complicarsi, difatti l'ostacolo è stato posizionato vicino a quello precedente in modo tale da andare a bloccare il passaggio per il raggiungimento del punto desiderato, e questo si riflette anche nei risultati ottenuti e nei grafici.

La visione dello scenario è la seguente :

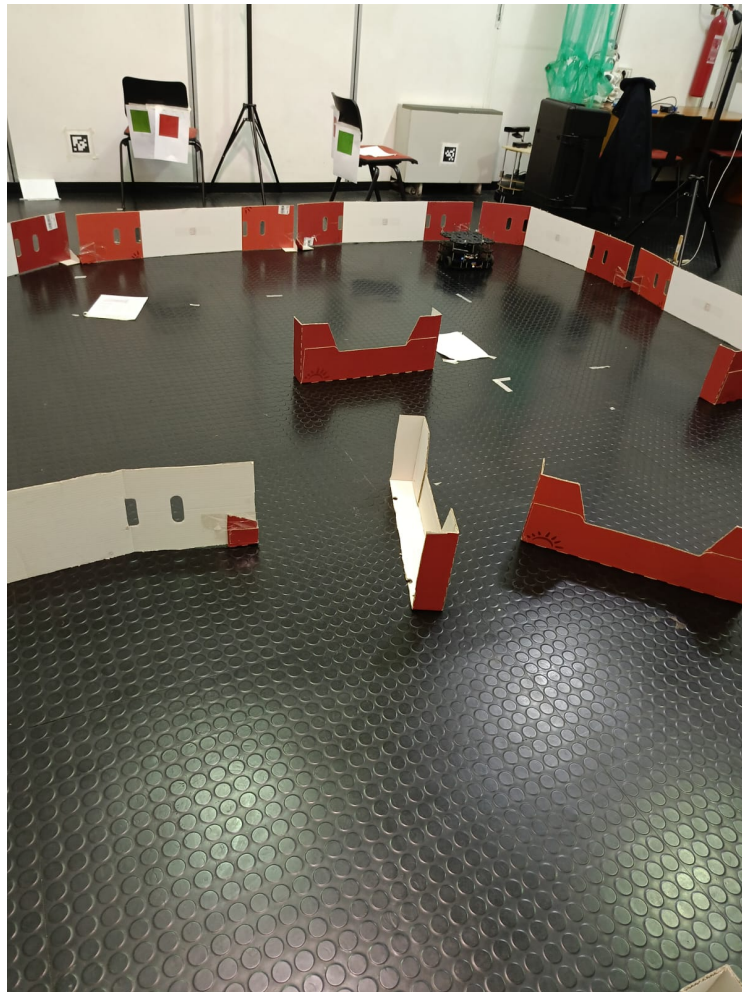


Figura 4.50: Allestimento del terzo scenario sperimentale in laboratorio

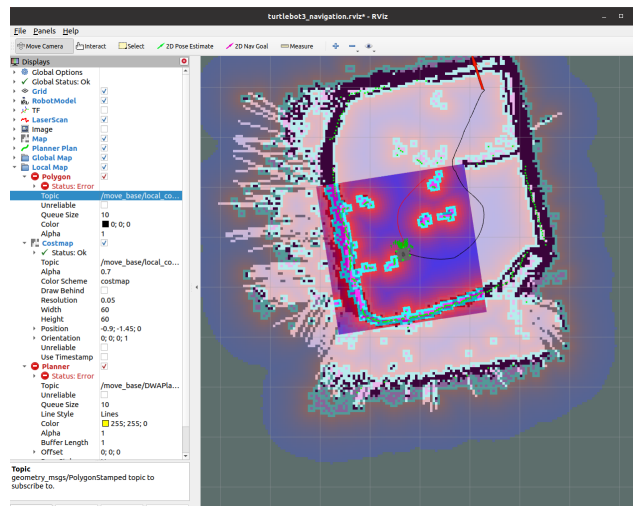


Figura 4.51: Disposizione terzo scenario

Metriche e grafici :

Distanza percorsa: 14.18 m

Tempo totale: 73.56 s

Distanza minima dal goal: 0.57 m

-----

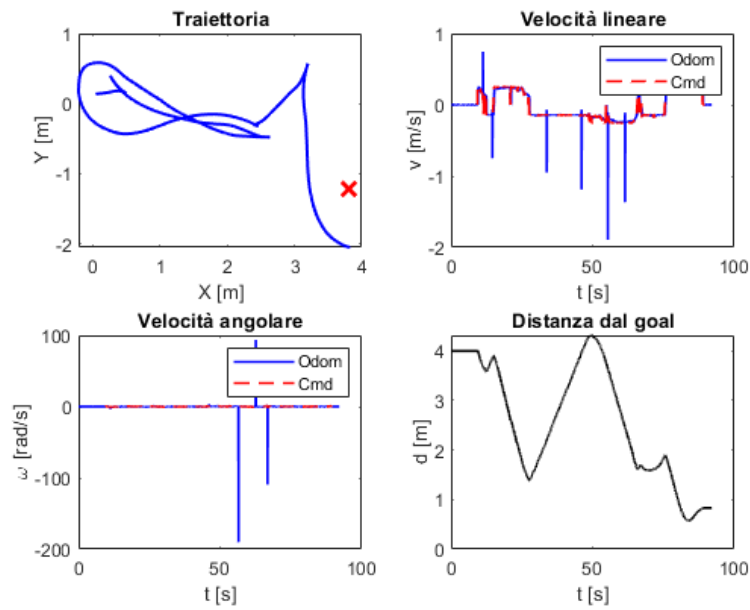


Figura 4.52: Grafici terzo scenario

Come si può notare il robot percorre una traiettoria particolare, difatti ad un certo istante torna indietro per cercare di ricalibrare il percorso, in modo tale da poter arrivare all'obiettivo senza collidere con l'ostacolo.

La manovra di retromarcia si può osservare in maniera particolare all'interno dei grafici di *velocità lineare e angolare* dove sono ben visibili i picchi di sottoelongazione.

#### Quarto scenario

L'ultimo scenario, analogamente a quello simulativo, è uno scenario molto più complesso che porta il robot a non riuscire ad arrivare al suo obiettivo.

L'ostacolo viene questa volta inserito di fronte alla posizione iniziale del robot:

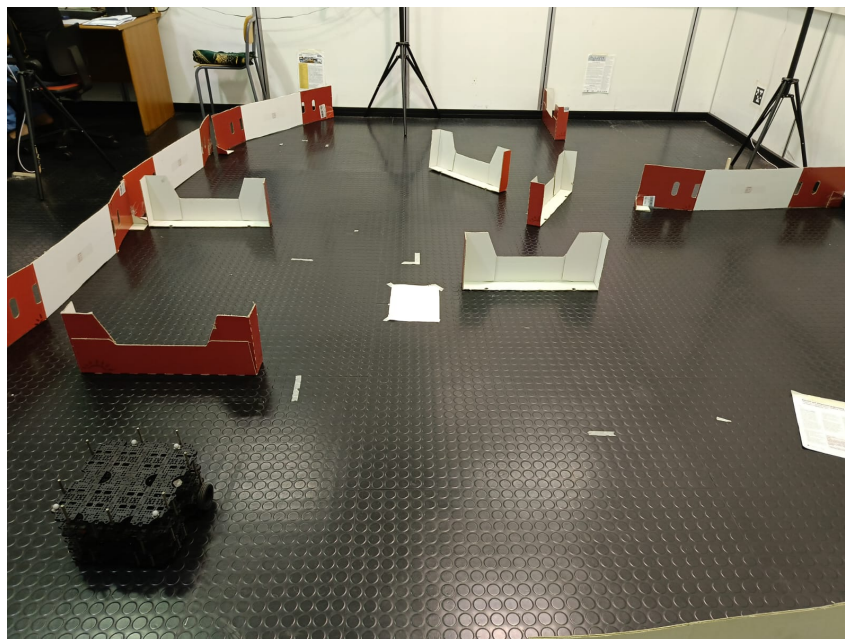


Figura 4.53: Disposizione quarto scenario

Il robot fa fatica a trovare percorsi ottimali e si blocca, questi sono i grafici risultanti :

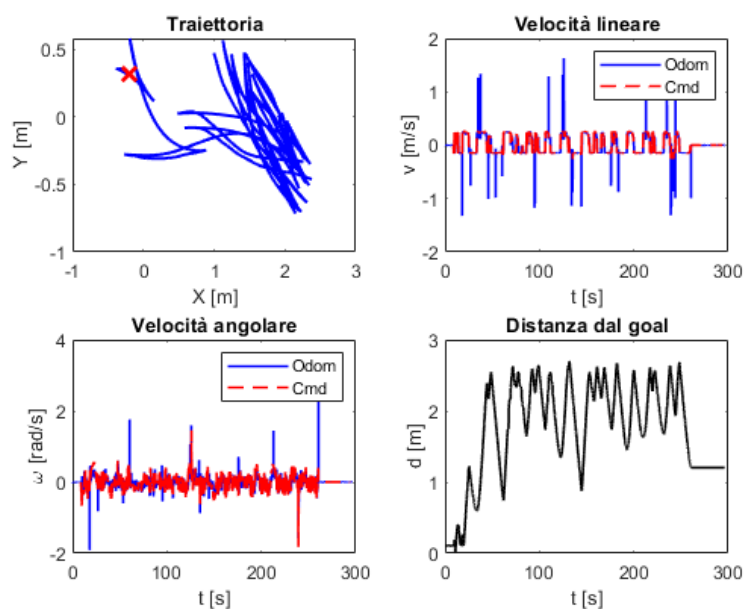


Figura 4.54: Grafici quarto scenario

# Capitolo 5

## Conclusioni

In conclusione vorrei porre l'attenzione sui risultati ottenuti, lo scopo del suddetto elaborato era quello di esplorare, analizzare e testare i comportamenti di un'intera architettura e algoritmi di navigazione, e soprattutto come questi si interfaccino con un robot che deve navigare all'interno di un ambiente contenente degli ostacoli; è stato possibile osservare un'ottima efficacia dell'algoritmo DWA nella maggior parte dei percorsi testati, che ha portato (sia nella casistica delle simulazioni virtuali che in quelle reali), al raggiungimento dell'obiettivo.

Altrettanto utile sono stati i risultati che hanno portato ad una conclusione "negativa", che quindi non hanno permesso di raggiungere il punto prestabilito, analizzare i grafici, e i resoconti di queste simulazioni, permette di trovare i punti deboli e in qualche modo di gettarne la base per la costruzione di algoritmi più flessibili sotto diversi punti di vista.

In particolare il controllore DWA per come è stato progettato potrebbe trovare diverse difficoltà qualora un ostacolo apparisse improvvisamente, o ancora meglio se questo fosse in movimento; è importante poi prendere in considerazione anche altre motivazioni che potrebbero portare al fallimento di una simulazione, come per esempio la qualità, tipologia e accuratezza dei sensori e l'incertezza di localizzazione con il metodo AMCL.

## 5.1 Proposte future

È importante riuscire a sviluppare sistemi che permettano sempre più la riuscita di un'accurata navigazione, l'utilizzo di questi potrebbe portare a svolte in svariati ambiti, come per esempio per assistere medici all'interno delle sale operatorie o persone che non possono più fare determinati movimenti, oppure addirittura per salvare vite esplorando ambienti pericolosi per l'uomo.

Questo elaborato ha analizzato solo una piccola parte di quello che è un mondo molto ampio e in continua crescita e che se impiegato nel giusto modo potrebbe fare la differenza.

# Bibliografia

- [1] Gerald Cook. *"Mobile robots : navigation, control and remote sensing"*
- [2] Roland Siegwart, Illah Reza Nourbakhsh, and Davide Scaramuzza. *"Introduction to Autonomous Mobile Robots"*. MIT Press, 2nd edition, 2011.
- [3] Nilsson, Nils J. *"Shakey the robot."* (1984).
- [4] Vedeshin, LA. *"The First Soviet Experiments on Remote Sensing and Contact Study of the Moon (on the 50th Anniversary of the Moon Landing of Lunokhod 1 Self-Propelled Vehicle)"*
- [5] Team, Rover. *"Characterization of the Martian surface deposits by the Mars Pathfinder rover, Sojourner."*
- [6] Doroftei, Ioan, Victor Grosu, and Veaceslav Spinu. *"Omnidirectional mobile robot-design and implementation."* (2007): 5518.
- [7] Malu, Sandeep Kumar, and Jharna Majumdar. *"Kinematics, localization and control of differential drive mobile robot."* *Global Journal of Research In Engineering* 14.1 (2014): 1-9.
- [8] Nurmaini, Siti, Kemala Dewi, and Bambang Tutuko. *"Differential-drive mobile robot control design based-on linear feedback control law."* *IOP Conference Series: Materials Science and Engineering*. Vol. 190. No. 1. IOP Publishing, 2017.

- [9] Fiorini P, Shiller Z. "*Motion Planning in Dynamic Environments Using Velocity Obstacles. The International Journal of Robotics Research. 1998;17(7):760-772. doi:10.1177/027836499801700706*"
- [10] Joseph, Lentin. *ROS Robotics projects. Packt Publishing Ltd, 2017.*
- [11] Thrunyz, Dieter Foxy Wolfram Burgard Sebastian. "*The dynamic window approach to collision avoidance.*" (1997).
- [12] Joseph, Lentin, and Jonathan Cacace. *Mastering ROS for Robotics Programming: Design, build, and simulate complex robots using the Robot Operating System. Packt Publishing Ltd, 2015.*